

Radiance Transfer Prediction for Real-Time Rendering using Neural Networks and Geometric Algebra

PhD Thesis



Author: Efstratios Geronikolakis (Student ID: 1192)

Supervisor: Prof. George Papagiannakis

Co-Supervisor: Dr. Manos Kamarianakis, Dr. Antonis
Protopsaltis

Department of Computer Science
School of Sciences & Engineering
University of Crete
Heraklion, Greece

November 12, 2025

Abstract

Precomputed Radiance Transfer (PRT) has long been a cornerstone of real-time rendering, enabling high-fidelity global illumination under complex lighting. However, traditional PRT suffers from prohibitive precomputation costs, limited scalability to high-resolution meshes, and impracticality for dynamic or animated scenes—severely constraining its use in interactive applications. Three key challenges arise in particular: (i) the expensive computation of per-vertex spherical harmonic (SH) coefficients, (ii) the lack of adaptability to changing geometry and lighting, and (iii) the memory and latency constraints that hinder deployment in real-time pipelines.

Recent advances in neural rendering and neural radiance field (NeRF) research have demonstrated the potential of deep learning networks to model complex light transport efficiently. Yet, these methods are typically restricted to image-space rendering and remain poorly suited for SH-based lighting representations or integration within mesh-based pipelines. In parallel, Conformal Geometric Algebra (CGA) has gained attention in computer graphics for its ability to represent geometric entities and transformations in a unified algebraic framework, offering a mathematically consistent way to encode spatial and lighting relationships directly on meshes. To address these limitations, we introduce Neural-GASh, a neural rendering framework that reformulates PRT within a geometric deep learning paradigm. By encoding vertex–normal pairs as CGA multivectors and utilizing neural networks to predict radiance transfer coefficients, we bypass the costly precomputation while maintaining physical interpretability.

The contributions of this work are threefold. First, we show that CGA-based encodings improve training stability and expressiveness by utilizing multivectors that capture spatial and angular relationships more richly than Euclidean inputs. Second, we show that graph neural networks (GNNs), leveraging mesh connectivity, capture self-occlusion and shadowing effects that MLP-based surrogates fail to reproduce. Third, we show how to integrate the trained models into Unity for real-time rendering, for both static and animated assets. Experimental evaluation across diverse 3D meshes confirms that Neural-GASh reduces precomputation by orders of magnitude while preserving high-quality illumination, including complex shadowing. The framework scales to tens of thousands of vertices in real-time, and the Unity prototype demonstrates its feasibility for interactive use cases. In summary, this thesis establishes GNN-based PRT as a scalable and effective alternative to classical precomputation, laying groundwork for future neural rendering pipelines in graphics and virtual reality.

Περίληψη

Ο αλγόριθμος Precomputed Radiance Transfer (PRT) αποτελεί εδώ και χρόνια ακρογωνιαίό λίθο της απόδοσης φωτός σε πραγματικό χρόνο, επιτρέποντας υψηλής πιστότητας καθολικό φωτισμό υπό πολύπλοκες συνθήκες. Ωστόσο, ο πρωτότυπος PRT παρουσιάζει σημαντικά μειονεκτήματα, όπως υπερβολικά μεγάλο προ-υπολογιστικό κόστος, περιορισμένη επεκτασιμότητα σε τρισδιάστατα μοντέλα υψηλής ανάλυσης, και αδυναμία εφαρμογής του σε δυναμικές σκηνές — γεγονός που περιορίζει σημαντικά τη χρήση του σε διαδραστικές εφαρμογές. Ειδικότερα, τρεις βασικές προκλήσεις προκύπτουν: (i) ο δαπανηρός υπολογισμός των συντελεστών σφαιρικών αρμονικών (SH) ανά κορυφή, (ii) η έλλειψη προσαρμοστικότητας σε μεταβαλλόμενη γεωμετρία και φωτισμό, και (iii) οι περιορισμοί μνήμης και καθυστέρησης που δυσχεραίνουν την ενσωμάτωσή του σε προγραμματιστικές ροές πραγματικού χρόνου. Οι πρόσφατες εξελίξεις στο πεδίο της νευρωνικής απόδοσης φωτός και στα πεδία νευρωνικής λαμπρότητας (NeRFs) έχουν δείξει τη δυναμική των νευρωνικών δικτύων στην αποδοτική μοντελοποίηση πολύπλοκων αλληλεπιδράσεων φωτός. Παρ' όλα αυτά, οι περισσότερες από αυτές τις μεθόδους περιορίζονται σε απόδοση φωτός βασισμένη σε εικόνες και δεν προσαρμόζονται εύκολα σε αναπαραστάσεις φωτισμού που βασίζονται σε SH ή σε προγραμματιστικές ροές βασισμένες σε τρισδιάστατα μοντέλα. Παράλληλα, η Σύμμορφη Γεωμετρική Άλγεβρα (CGA) έχει προσελκύσει το ενδιαφέρον της επιστημονικής κοινότητας των γραφικών λόγω της ικανότητάς της να αναπαραστή γεωμετρικές οντότητες και μετασχηματισμούς μέσα σε ένα ενιαίο αλγεβρικό πλαίσιο, προσφέροντας έναν μαθηματικά συνεπή τρόπο κωδικοποίησης χωρικών και σχέσεων φωτισμού απευθείας πάνω στα πλέγματα. Για να αντιμετωπιστούν αυτοί οι περιορισμοί, παρουσιάζουμε το Neural-GASh, ένα πλαίσιο νευρωνικής απόδοσης φωτός που επαναδιατυπώνει τον PRT μέσα από το πρίσμα γεωμετρικής άλγεβρας και βαθιάς μάθησης. Κωδικοποιώντας τα ζεύγη κορυφών-κανονικών διανυσμάτων ως πολυδιανύσματα CGA και χρησιμοποιώντας νευρωνικά δίκτυα για την πρόβλεψη των συντελεστών μεταφοράς λαμπρότητας, παρακάμπτουμε το χρονοβόρο προ-υπολογιστικό στάδιο, διατηρώντας ταυτόχρονα τη φυσική ερμηνευσιμότητα του μοντέλου. Η συνεισφορά της παρούσας εργασίας είναι τριπλή. Πρώτον, δείχνουμε ότι οι κωδικοποιήσεις βασισμένες στη CGA βελτιώνουν τη σταθερότητα εκπαίδευσης και την εκφραστικότητα του μοντέλου, καθώς τα πολυδιανύσματα αποτυπώνουν πιο πλούσιες χωρικές και γωνιακές σχέσεις σε σχέση με δεδομένα εισόδου βασισμένα στην Ευκλείδεια Γεωμετρία. Δεύτερον, αξιοποιώντας την τοπολογία του τρισδιάστατου μοντέλου, αποδεικνύουμε ότι τα Νευρωνικά Δίκτυα Γράφων (GNNs) μπορούν να αποδώσουν φαινόμενα αυτοαπόκρυψης (self-occlusion) και σκιών (shadowing) που απλά πολυστρωματικά νευρωνικά δίκτυα προώθησης (MLPs) αδυνατούν

να αναπαράγουν. Τρίτον, παρουσιάζουμε πώς τα εκπαιδευμένα μοντέλα μπορούν να ενσωματωθούν στην μηχανή γραφικών Unity για απόδοση φωτός σε πραγματικό χρόνο, τόσο σε στατικά όσο και σε δυναμικά μοντέλα. Η πειραματική μας αξιολόγηση σε μια ποικιλία τρισδιάστατων μοντέλων επιβεβαιώνει ότι το Neural-GASh μειώνει τον χρόνο προϋπολογισμού κατά τάξεις μεγέθους, διατηρώντας παράλληλα φωτισμό υψηλής ποιότητας και ρεαλιστική σκίαση. Το πλαίσιο κλιμακώνεται σε δεκάδες χιλιάδες κορυφές σε πραγματικό χρόνο, και η εφαρμογή του στην Unity αποδεικνύει την πρακτική του χρησιμότητα σε διαδραστικές εφαρμογές. Συνοψίζοντας, η παρούσα διατριβή καθιερώνει μια εκδοχή του PRT βασισμένη σε GNNs ως μια επεκτάσιμη και αποτελεσματική εναλλακτική στις κλασικές μεθόδους προϋπολογισμού, θέτοντας τις βάσεις για μελλοντικές ροές νευρωνικής απόδοσης φωτός στα γραφικά και την εικονική πραγματικότητα.

Declaration of Authorship

I, Efstratios Geronikolakis, declare that this thesis titled, "Radiance Transfer Prediction for Real-Time Rendering using Neural Networks and Geometric Algebra", and the work presented in it are my own. I confirm that:

- This work was done wholly or mainly while in candidature for a research degree at this University.
- Where any part of this thesis has previously been submitted for a degree or any other qualification at this University or any other institution, this has been clearly stated.
- Where I have consulted the published work of others, this is always clearly attributed.
- Where I have quoted from the work of others, the source is always given. With the exception of such quotations, this thesis is entirely my own work.
- I have acknowledged all main sources of help.
- Where the thesis is based on work done by myself jointly with others, I have made clear exactly what was done by others and what I have contributed myself.

Signature: Efstratios Geronikolakis

Date: November 12, 2025

Acknowledgements

I would like to express my sincere gratitude to my supervising professor, George Papagiannakis, for his continuous guidance, encouragement, and invaluable direction throughout the course of this project.

I am also deeply thankful to my co-supervisors, Manos Kamarianakis and Antonis Protopsaltis, for their insightful feedback, constructive advice, and constant support during the development and writing of this thesis.

Moreover, I would like to express my profound gratitude to Prodromos Kolyvakis for introducing me to the fascinating field of Deep Learning and Neural Networks, and for his invaluable assistance and insightful ideas during moments when progress seemed difficult.

Finally, I owe my deepest appreciation to my parents and, above all, to my wife, Maria Varytimidou, whose unwavering support and encouragement sustained me through the most challenging moments of this journey.

I would also like to acknowledge the financial support provided by the Manassaki Fellowship, established in memory of Maria Michail Manassaki, which greatly contributed to the successful completion of this research.

To my family...

Contents

1	Introduction	5
1.1	The rendering equation	5
1.2	Importance of GI	6
1.3	Approximating the light equation: PRT	8
1.4	Limitations of PRT	10
1.5	Toward a Neural Formulation of PRT	12
1.6	Thesis Overview	12
2	State-of-the-art	14
2.1	Precomputed Radiance Transfer	14
2.1.1	Extensions to Precomputed Radiance Transfer	14
2.2	Neural Rendering	18
2.2.1	Advances in Deep Learning for Radiance Transfer	19
2.3	Gaussian Splatting	22
2.3.1	Advancements in the field of 3DGS	24
2.4	Real-Time Rendering in XR and VR Environments	25
2.5	Identified Research Gaps	26
3	Methodology	29
3.1	Motivation and Overview	30
3.2	Problem Formulation	33
3.3	Geometry Encoding with CGA	33
3.4	Neural Variants of PRT	34
3.4.1	Deployment Strategy	34
3.4.2	Architectural Variants	34
3.5	Summary	35
4	Neural-GASh (MLP-Based)	36
4.1	Euclidean vs CGA Input	38
4.1.1	From Euclidean to CGA Representation	39
4.2	Data formation	40
4.3	Network architecture	42
4.3.1	Raw vertex-normal pairs vs CGA Multivectors	42
4.4	Network training and validation	43
4.5	Unity Integration	45
4.6	Results	46
4.7	Gaussian Splatting & Neural-GASh	51
4.8	Neural-GASh vs State-of-the-art	54

4.9	Limitations	57
5	Neural-GASh (GNN-Based)	59
5.1	MLP-based with richer dataset	60
5.2	GNN-based	60
5.2.1	GNN (32D)	61
5.2.2	GNN (32D + 25D Visibility)	62
5.2.3	Teacher-Student Architecture	64
5.2.4	Two-Stage GNN	66
5.2.5	Design trade-offs: single vs. two-stage GNN	67
5.3	Networks training and validation	69
5.3.1	Coefficients GNN	69
5.3.2	Visibility GNN	71
5.4	Unity Integration	74
5.4.1	Flask Server	74
5.4.2	Unity Pipeline	76
5.5	Results	78
5.5.1	Qualitative Analysis	78
5.5.2	Quantitative Analysis	81
5.5.3	Flask Server Latency Analysis	90
5.5.4	Animated Geometry	93
5.5.5	Multiple Objects & Scenes	94
5.5.6	Scalability to Large Meshes	96
6	Discussion & Future Work	98
6.1	Limitations	99
6.2	Future Work	100
6.3	Conclusion	100
	Bibliography	102
A	Appendix	114
A.1	Pythonic PRT basic algorithm implementation	114
A.2	Platform	114
A.3	Methodology	115
A.3.1	3D Mesh Loading	116
A.3.2	High-Dynamic-Range (HDR) Skybox	117
A.3.3	Main Implementation	118
A.3.4	Implementation Variants	121

B	Appendix	123
B.1	Traditional PRT in Python	123
B.2	GNN Training	133
B.3	Flask Server	144
B.4	Unity	148
C	Appendix	155
C.1	Introduction to Conformal Geometric Algebra (CGA)	155
C.2	Vector Objects in CGA	155
C.3	Products in $\mathbb{R}_{4,1}$	156
C.4	Rotations, Translations, and Dilations	157
C.5	Remarks	158

List of Figures

1	Rendering results obtained using different GI algorithms.	6
2	Comparison of a computer-generated living room scene rendered (a) without GI, and (b) with baked GI. In the non-GI version, all baked data were cleared and no active light sources were used, resulting in a flat, unlit appearance. In the GI version, the scene was illuminated using a combination of area lights and a mixed directional light, followed by light baking. The inclusion of GI introduces indirect illumination, soft shadows, and interreflections, producing a more realistic and visually coherent result [40, 8]. . .	7
3	The main PRT pipeline as introduced by Sloan et al. [75]	9
4	The representation of a simple scene, lit by the PRT algorithm using CGA. [56]	15
5	An example of PRTT usage. [18]	16
6	The main concept of the PRT for Reflectance and Lighting Estimation. [79]	17
7	Example results from Gaussian Splatting. (a) Rendering with visible Gaussians highlights the overdraw and transparency artifacts produced by the representation. (b) Rendering with invisible Gaussians produces a clean and realistic reconstruction of the scene, where the Gaussian primitives are imperceptible. [43] . . .	23
8	NeRF (left) versus 3DGS (right). [11]	23
9	A scene that was reconstructed with the 3DGS technique. [43] . .	24
10	Scaling of unshadowed vs. shadowed PRT precomputation time with respect to vertex count. While both grow with complexity, shadowed PRT exhibits significantly steeper growth due to visibility tests.	32
11	The neural radiance field architecture utilized in Neural-GASh. . .	37
12	The neural network architecture, used in the neural radiance field of Neural-GASh.	42

13	Shading results of a cat mesh using (a) Neural-GASh, i.e., model is using 32 CGA multivectors as input, and (b) a network with identical architecture as Neural-GASh, using 6D euclidean coordinates as input (for position and normal of each vertex). In (a), the red light source illuminates only the edge of the cat’s head, consistent with its frontal position. In contrast, in (b), the entire back of the cat (including the tail) is unrealistically illuminated by the same frontal red light source. This represents a challenging edge case in large-scale 3D meshes, which is more accurately addressed by incorporating the 32D CGA representation in our neural network input.	43
14	The main structure of the Neural-GASh pipeline in Unity for the teapot model.	46
15	Traditional PRT and Neural-GASh time comparison.	48
16	MSE and deviation of the predicted vertex coefficients for the five 3D meshes.	48
17	The garden scene from the Mip-NeRF 360 project [3]. On the left is the 3D Gaussian Splatting reconstruction, generated from a dataset of 185 images. On the right is the reconstruction using a 3D point cloud, generated with the SuGaR method [29], with texture extracted from the Gaussians.	52
18	Neural-GASh results for the Mip-NeRF 360 scene [3], reconstructed with the SuGaR method [29], with each instance involving the rotation of light and light coefficients using CGA logic.	53
19	Lighting results on the reconstructed scene from Mip-NeRF 360 [3], reconstructed with the SuGaR method [29], utilizing different radiance maps. The time in seconds needed to switch from one lightmap to another, was between 0.3-0.6 seconds. The lighting changes are performed during program runtime.	54
20	Comparison between (a) the traditional PRT baseline and (b) the MLP-based Neural-GASh result. The self-shadow beneath the arm, highlighted in red in (a), is missing in (b), revealing the MLP model’s limitation in capturing occlusion-dependent light transport.	59
21	Comparison between (a) the traditional PRT baseline and (b) the GNN-based Neural-GASh output using only CGA-derived input features. The self-shadow region, highlighted in red, appears misaligned in (b), indicating that CGA features alone are insufficient for accurate shadow placement without incorporating explicit topological information.	62

22	Comparison between (a) the traditional PRT baseline and (b) the GNN-based Neural-GASh output augmented with visibility samples. The self-shadow region, highlighted in green, is more accurately reproduced in (b), demonstrating that incorporating visibility information enhances topological awareness and corrects the misalignment observed in Figure 21.	64
23	Illustration of the Teacher-Student architecture applied to Neural-GASh. The Teacher model (left), a larger GNN trained with full input including visibility samples, supervises a Student model (right) restricted to the CGA multivector input. Adapted from [69]. . . .	65
24	Conceptual illustration of the neural radiance field formulation adopted in the GNN-based Neural-GASh.	67
25	A flowchart of the coefficients GNN workflow.	71
26	A flowchart of the visibility GNN workflow.	73
27	Abstract pipeline of the GNN-based Neural-GASh and its communication with the Flask server.	75
28	Detailed pipeline of GNN-based Neural-GASh.	78
29	The HDRI map that was used to provide the results in Table 8. The HDRI is sourced from <i>Poly Haven</i> [62].	82
30	The HDRI map that was used to provide the results in Table 9. The HDRI is sourced from <i>Poly Haven</i> [62].	82
31	The HDRI map that was used to provide the results in Table 10. The HDRI is sourced from <i>Poly Haven</i> [62].	82
32	Time comparison in seconds (s) of the traditional PRT algorithm and the GNN-based version of Neural-GASh.	86
33	Mean Squared Error (MSE) of coefficients between the traditional PRT algorithm and GNN-based version of Neural-GASh, across the 9 coefficients indices.	88
34	Standard Deviation (STD) of coefficients between the traditional PRT algorithm and GNN-based version of Neural-GASh, across the 9 coefficients indices.	88
35	Prediction accuracy of the proposed visibility network across different 3D models.	89
36	Per-stage latency breakdown of the Flask server for seven different meshes. Each line corresponds to a mesh, with time (in milliseconds, log scale) plotted across the nine pipeline stages. The AO stage (8) is optional, and performance-sensitive applications can omit it.	92

37	Example of the GNN-based Neural-GASh applied on animated geometry. The animation progresses from left to right and the shadows are highlighted within the red boxes. The HDRI map in Figure 29 is utilized to light the object.	94
38	Comparison of the surgery scene rendered with the GNN-based Neural-GASh (a) and Unity’s built-in global illumination (b). Under the simplest Unity GI configuration, Neural-GASh achieves superior results in both visual quality and performance. In particular, the objects under the table are more accurately lit in (a) as well as the cloth, as its shadow is more visible. Neural-GASh reproduces self-shadowing effects on the doctor, which are absent in the Unity rendering.	95
39	The pyGANDALF scene editor, providing an engine-like interface for real-time visualization and prototyping. [60]	115
40	An example of a LDR image (left) and a HDR image (right). In the HDR image, the snow keeps its texture, the trees in shadow are still visible, and both bright and dark regions have detail. [30]	117
41	A cross-HDRI image (a) and the skybox (b) we create based on it. The HDRI is sourced from <i>Poly Haven</i> [62].	118
42	A cross-HDRI image with its sections marked. The HDRI is sourced from <i>Poly Haven</i> [62].	119
43	The main pipeline of our PRT implementation in Python.	119
44	Comparison of PRT variants on the monkey mesh. (a) Unshadowed transfer, where illumination is accumulated without accounting for occlusion. (b) Shadowed transfer, where ray-triangle visibility tests capture self-occlusion, producing darker regions in crevices and shadowed areas. Red boxes highlight regions with notable differences due to shadow integration.	122

List of Tables

1	Comparative summary of representative GI and neural rendering methods.	27
2	Precomputation times of the Python PRT implementation for meshes of varying vertex counts. Shadowed PRT incurs significantly higher cost due to ray-triangle visibility queries.	31
3	Precomputation times for various meshes using Traditional PRT and Neural-GASh. The bold result indicates real-time performance for meshes with fewer than 1000 vertices.	47
4	Results comparison of five different meshes under traditional PRT and Neural-GASh.	50
5	Visual comparison of textured meshes rendered with traditional PRT (left) and Neural-GASh (right). The results show that Neural-GASh reproduces the same global illumination, and diffuse shading characteristics as classical PRT, while operating in real time. The near-identical appearance between the two columns demonstrates that the neural model preserves lighting fidelity even on textured, complex geometries.	51
6	Comparison of PRT methods and their capabilities with Neural-GASh.	55
7	Precomputation times for various meshes using Traditional PRT and GNN-based Neural-GASh.	80
8	Results comparison of six different meshes under a dark lit environment.	83
9	Results comparison of six different meshes under a brightly lit environment.	84
10	Results comparison of six different meshes under a normal lit environment.	85
11	End-to-end round-trip latency measured on the Flask server for various mesh models. All values are reported in milliseconds and represent the average of ten inference requests per mesh. The total latency includes data transmission, neural inference, and ambient occlusion processing.	91

List of Abbreviations

Abbreviation	Definition
3DGS	3D Gaussian Splatting
AO	Ambient Occlusion
AR	Augmented Reality
BRDF	Bidirectional Reflectance Distribution Function
CGA	Conformal Geometric Algebra
ECS	Entity Component System
GA	Geometric Algebra
GI	Global Illumination
GNN	Graph Neural Network
GATv2	Graph Attention Network v2
HDR	High Dynamic Range
HDRI	High Dynamic Range Imaging
LDR	Low Dynamic Range
MLP	Multilayer Perceptron
NeRF	Neural Radiance Field
ONNX	Open Neural Network Exchange
PRT	Precomputed Radiance Transfer
PRTT	PRT Textures
SH	Spherical Harmonics
VR	Virtual Reality
XR	Extended Reality

List of Publications

The work presented in this dissertation builds upon and extends the research outcomes of the following publications, to which the author has contributed either as main author or as co-author.

Conference Publications

- Geronikolakis, E., Kamarianakis, M., Protopsaltis, A., and Papagiannakis, G., “**Neural-GASh: A CGA-based Neural Radiance Prediction Pipeline for Real-Time Shading,**” *Computer Graphics International 2025 (CGI 2025) – ENGAGE 2025 Workshop*, Springer LNCS Proceedings, 2025. (**Best GA Computing Paper Award**) [22]
- Zikas, P., Kamarianakis, M., Kartsonaki, I., Lydatakis, N., Kateros, S., Kentros, M., Geronikolakis, E., Evangelou, G., Apostolou, A., Catilo, P. A. A., and Papagiannakis, G., “**Covid-19 – VR Strikes Back: Innovative medical VR training,**” In *ACM SIGGRAPH 2021 Immersive Pavilion (SIGGRAPH ’21)*. Association for Computing Machinery, New York, NY, USA. DOI: 10.1145/3450615.3464546. [93]
- Papagiannakis, G., Zikas, P., Lydatakis, N., Kateros, S., Kentros, M., Geronikolakis, E., Kamarianakis, M., Kartsonaki, I., and Evangelou, G., “**MAGES 3.0: Tying the knot of medical VR,**” in *ACM SIGGRAPH 2020 Immersive Pavilion (SIGGRAPH ’20)*, Association for Computing Machinery, New York, NY, USA, Article 6, pp. 1–2. DOI: 10.1145/3388536.3407888, 2020. [58]

Journal Publications

- Zikas, P., Protopsaltis, A., Lydatakis, N., Kentros, M., Geronikolakis, S., Kateros, S., Kamarianakis, M., Evangelou, G., Filippidis, A., Grigoriou, E., Angelis, D., Tamiolakis, M., Dodis, M., Kokiadis, G., Petropoulos, J., Pateraki, M., Papagiannakis, G. (2023). **MAGES 4.0: Accelerating the World’s Transition to VR Training and Democratizing the Authoring of the Medical Metaverse.** *IEEE Computer Graphics and Applications*, **43**(2), 43–56. <https://doi.org/10.1109/mcg.2023.3242686>. [95]
- Zikas, P., Kateros, S., Lydatakis, N., Kentros, M., Geronikolakis, E., Kamarianakis, M., Evangelou, G., Kartsonaki, I., Apostolou, A., Birrenbach,

T., Exadaktylos, A.K., Sauter, T.C., and Papagiannakis, G., “**Virtual reality medical training for COVID-19 swab testing and proper handling of personal protective equipment: Development and usability**,” *Frontiers in Virtual Reality*, vol. 2, p. 740197, 2022. [94]

- Bertrand, S., Vassiliadi, M., Zikas, P., Geronikolakis, E., Papagiannakis, G. (2021). **From Readership to Usership: Communicating Heritage Digitally Through Presence, Embodiment and Aesthetic Experience**. *Frontiers in Communication*, 6.
<https://doi.org/10.3389/fcomm.2021.676446>. [6]

Book Chapters

- Geronikolakis, E., Zikas, P., Kateros, S., Lydatakis, N., Georgiou, S., Kentros, M., and Papagiannakis, G., “**A True AR Authoring Tool for Interactive Virtual Museums**,” in *Visual Computing in Cultural Heritage*, F. Liarakapis, A. Voulodimos, N. Doulamis, and A. Doulamis (eds.), Springer-Nature, Series on Cultural Computing, 2020. [24]
- Geronikolakis, E., Tsioumas, M., Bertrand, S., Loupas, A., Zikas, P., Papagiannakis, G. (2018). **New Cross/Augmented Reality Experiences for the Virtual Museums of the Future**. In: Ioannides, M. *et al.* (eds) *Digital Heritage. Progress in Cultural Heritage: Documentation, Preservation, and Protection. EuroMed 2018. Lecture Notes in Computer Science*, vol. 11196. Springer, Cham. Also presented at EuroMed 2018. [20]
- Papagiannakis, G., Geronikolakis, E., Pateraki, M., López-Mencheró, M. V., Tsioumas, M., Sylaiou, S., Liarakapis, F., Grammatikopoulou, A., Dimitropoulos, K., Grammalidis, N., Partarakis, N., Margetis, G., Drossis, G., Vassiliadi, M., Chalmers, A., Stephanidis, C., Magnenat-Thalmann, N., **Mixed Reality, Gamified Presence, and Storytelling for Virtual Museums**. In: Lee, N. (ed) *Encyclopedia of Computer Graphics and Games*. Springer, Cham, 2018 [57]
- Papaefthymiou, M., Kanakis, M. E., Geronikolakis, E., Nochos, A., Zikas, P., and Papagiannakis, G., “**Rapid Reconstruction and Simulation of Real Characters in Mixed Reality Environments**,” in *Digital Cultural Heritage*, Lecture Notes in Computer Science, vol. 10605, Springer, Cham, 2018. [55]

Preprints and Technical Reports

- Geronikolakis E., and Papagiannakis, G., “**An XR rapid prototyping framework for interoperability across the reality spectrum,**” *arXiv preprint arXiv:2101.01771*, 2021. [23]

All the above publications are also included in the bibliography at the end of this thesis.

1 Introduction

Global illumination (GI) is a fundamental concept in computer graphics, referring to a family of algorithms that simulate the transport of light within a scene by accounting for both direct and indirect illumination. Unlike simple local illumination models, such as the Blinn-Phong algorithm [7], which consider only the light that travels directly from an emissive source to a surface, GI techniques aim to approximate the full range of light transport phenomena, including reflection, refraction, scattering, and interreflections between surfaces. This broader treatment of light behavior is essential for producing images that achieve physical plausibility and perceptual realism, as illustrated in Figure 2.

1.1 The rendering equation

The theoretical foundation of GI is commonly traced to Immel’s [37] and Kajiya’s [40] seminal work on the rendering equation, which provides a mathematical framework for describing the equilibrium of radiance in a scene. Solving this equation is, however, computationally intractable in closed-form for most practical cases. As a result, a variety of numerical techniques have been developed to approximate its solution, as shown in Figure 1, including Monte Carlo methods such as path tracing [13], bidirectional path tracing [47], photon mapping [39], and more recently, real-time approaches leveraging hardware acceleration [15] and spatio-temporal denoising [50]. These methods have progressively advanced the ability to render scenes with physically accurate shadows, caustics, color bleeding, and subtle diffuse interreflections—features that are indispensable for achieving realism in synthetic imagery.

$$L_o(x, \hat{v}) = L_e(x, \hat{v}) + \int_{\Omega^+} f_r(x, \hat{\omega}, \hat{v}) L_i(x, \hat{\omega}) (\hat{\omega} \cdot \hat{n}) d\hat{\omega}, \quad (1)$$

Equation 1 expresses the rendering equation as introduced by Immel and Kajiya, which describes the equilibrium of radiance at a point on a surface. It states that the outgoing radiance L_o in a given direction $\hat{v}$ is the sum of the emitted radiance L_e from the surface and the reflected radiance due to incident light arriving from all directions $\hat{\omega}$ in the hemisphere Ω^+ above the surface normal $\hat{n}$.

The integrand combines three essential components:

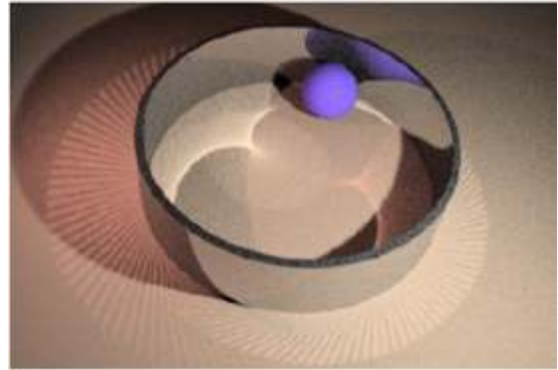
- $L_i(x, \hat{\omega})$: the *incident radiance* arriving at x from direction $\hat{\omega}$,
- $f_r(x, \hat{\omega}, \hat{v})$: the *Bidirectional Reflectance Distribution Function (BRDF)* describing how light is scattered,

- $(\hat{\omega} \cdot \hat{n})$: the geometric term accounting for the foreshortening of grazing-angle illumination.

This formulation underpins virtually all GI algorithms. Since the integral does not generally admit a closed-form solution, a wide variety of numerical methods have been proposed to approximate it, most notably Monte Carlo techniques such as path tracing, bidirectional path tracing, and photon mapping.



(a) Path Tracing [13]



(b) Bidirectional Path Tracing [12, 47]



(c) Photon Mapping [39]

Figure 1: Rendering results obtained using different GI algorithms.

1.2 Importance of GI

The importance of GI becomes evident when considering the limitations of models that neglect indirect illumination. Without GI, scenes often exhibit unrealistic sharp contrasts, flat shading, and missing secondary shadows, ultimately undermining both visual fidelity and viewer immersion. In contrast, incorporating global light transport enhances depth perception, material differentiation, and the overall aesthetic quality of the image, thereby aligning more closely with the way humans perceive the real world.

A notable illustration of the impact of GI is presented in Figure 2, which depicts a computer-generated scene of a living room. The image on the left shows



Figure 2: Comparison of a computer-generated living room scene rendered (a) without GI, and (b) with baked GI. In the non-GI version, all baked data were cleared and no active light sources were used, resulting in a flat, unlit appearance. In the GI version, the scene was illuminated using a combination of area lights and a mixed directional light, followed by light baking. The inclusion of GI introduces indirect illumination, soft shadows, and interreflections, producing a more realistic and visually coherent result [40, 8].

the scene rendered without GI, resulting in a flat appearance with unrealistic shading and the absence of secondary lighting effects. In contrast, the image on the right presents the identical scene rendered with GI, where light interactions such as interreflections and soft shadows contribute to a far more convincing and visually compelling result.

The inclusion of GI fundamentally transforms the visual experience by enhancing realism and visual quality. By accurately simulating natural lighting conditions, it significantly improves the fidelity of computer-generated imagery, which is particularly important in applications such as film, video games, and Virtual Reality (VR). At the same time, GI contributes to atmospheric and mood setting, as lighting strongly influences the perception of a scene and can shape the viewer’s emotional response. Furthermore, it enhances detail and depth by revealing subtle lighting variations that highlight textures, material properties, and geometric features, thereby strengthening the perception of three-dimensionality. Finally, GI ensures consistency and immersion by providing coherent and physically plausible lighting across the entire scene, which reinforces believability and allows the viewer to engage more fully with the virtual environment.

Beyond its significance in computer-generated imagery for film, games, and VR, GI also plays a crucial role in domains such as architectural visualization, where accurate light prediction informs design decisions, and scientific and medical visualization, where light transport can influence interpretability of volumetric or surface data. As rendering increasingly shifts from offline production pipelines to real-time interactive systems such as video games, VR, and Augmented Reality (AR), the challenge becomes achieving physically accurate GI while operating under the strict frame-time budgets these platforms impose. Unlike offline rendering,

where computations may take minutes or hours per frame, real-time applications must generate convincing results within milliseconds, typically sustaining frame rates of 30–120 frames per second to ensure responsiveness and visual comfort. This constraint continues to drive research into efficient sampling strategies, novel acceleration structures, and perceptually motivated approximations that enable GI to be approximated at interactive rates without sacrificing visual fidelity.

1.3 Approximating the light equation: PRT

Although GI provides the theoretical foundation for physically accurate rendering, its direct implementation is hindered by the intractability of the rendering equation. The integral formulation introduced by Kajiya cannot be solved analytically for realistic scenes, which necessitates the use of numerical approximations. Over the years, a wide variety of algorithms have been developed to approximate this equation with different trade-offs in terms of accuracy and efficiency. Among these, PRT has become a particularly influential method, as it enables the approximation of complex light transport phenomena while still supporting real-time rendering.

Among the many GI algorithms that have been developed (such as [84, 14, 46]), is Precomputed Radiance Transfer (PRT) as shown in Figure 3. In general, the higher the realism sought in GI, the more computationally expensive the algorithm becomes, making efficiency a critical factor in practical applications. PRT was introduced by Sloan [75] as a solution to this problem, enabling interactive and real-time rendering while still preserving many of the perceptual qualities associated with global light transport. It was later extended to handle more complex lighting environments and material properties through clustered principal components [74].

The core idea behind PRT is to precalculate how light is transported within a scene and to store this information in a form that can be reused during rendering. By precomputing these transfer functions, the algorithm avoids the need to solve the full rendering equation in real time, thus greatly improving performance. During rendering, the precomputed data is combined with the actual incident lighting to reconstruct the appearance of the scene, allowing for complex light interactions—such as soft shadows, diffuse interreflections, and subtle shading variations—to be captured efficiently. As a result, PRT can adapt to dynamic lighting conditions with significantly reduced computational cost, making it particularly attractive for applications such as video games and interactive simulations where both performance and visual fidelity are paramount.

Technically, PRT relies on a global transport simulator to capture how low-

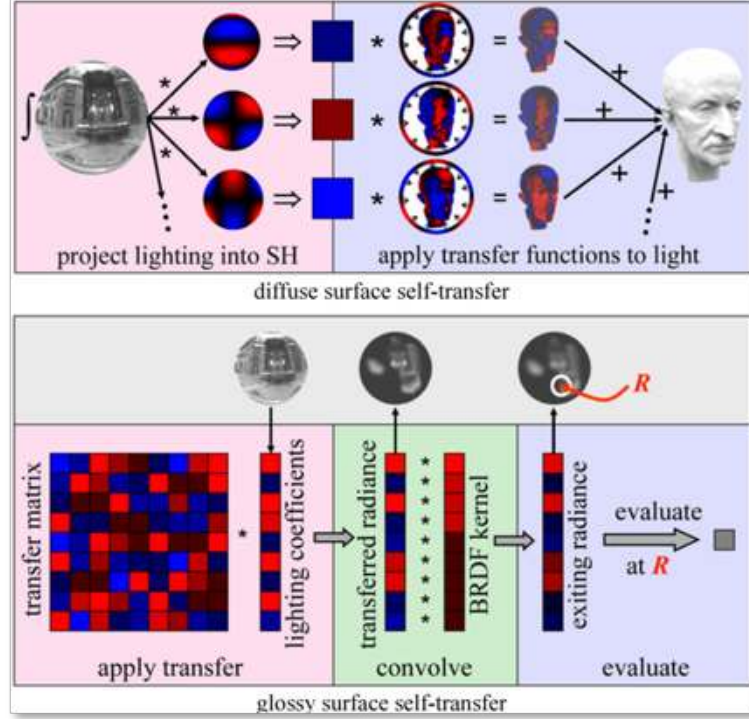


Figure 3: The main PRT pipeline as introduced by Sloan et al. [75]

frequency lighting transfers across an object’s surface. The transfer functions are then expressed in a compact mathematical representation using low-order *Spherical Harmonics* (*SH*). This formulation reduces the otherwise high-dimensional shading integral to a simple dot product of vectors in the case of diffuse surfaces, or to matrix operations in the case of glossy reflectors. The use of SH not only makes the computations efficient but also allows rigid transformations of objects relative to the lighting environment to be handled naturally.

In PRT, many of these terms that appear in Equation 1 (especially the visibility function and the product $f_r(\mathbf{x}, \omega_i, \omega_o)(\omega_i \cdot \mathbf{n})$) are precomputed. This results in the so-called transfer function, $T(\mathbf{x}, \omega_i)$, which encapsulates the light response of each point on the object.

To efficiently store the transfer functions, PRT leverages a basis function representation, most notably using SH. SH provide an orthonormal basis for functions defined over the sphere, making them well-suited for representing the angular variation of lighting. The transfer function at each point is projected onto a limited number of SH basis functions:

$$T(\mathbf{x}, \omega_i) \approx \sum_{l=0}^L \sum_{m=-l}^l c_{lm}(\mathbf{x}) Y_{lm}(\omega_i) \quad (2)$$

Here, $c_{lm}(\mathbf{x})$ are the coefficients computed during the precomputation phase, and $Y_{lm}(\omega_i)$ are the SH basis functions. This compression significantly reduces the

data needed for each point while still capturing the essential characteristics of the lighting environment.

The offline phase comprises several critical steps. First, a geometric sampling step is performed, in which the scene is sampled at an appropriate resolution—typically at the mesh vertices—to ensure that variations in geometry and material properties are accurately captured. Next, visibility determination takes place, during which visibility for each sample point is assessed by casting rays over the hemisphere. This process calculates occlusion effects, which are essential for rendering realistic soft shadows. Following this, the BRDF and diffuse response are incorporated, integrating local material properties and the associated BRDF into the transfer function to account for different surface reflection characteristics. Lastly, projection onto basis functions is performed, where the computed transfer functions are projected onto a selected basis (e.g., SH), producing a compact set of coefficients, $c_{lm}(\mathbf{x})$, that efficiently represent the function. Given its computational complexity, this phase is typically executed offline, enabling a detailed and high-fidelity representation of the light interactions of the scene.

During runtime, the precomputed coefficients are combined with the current lighting environment—also represented in the same basis—to reconstruct the lighting for each point via a simple dot product:

$$L_o(\mathbf{x}, \omega_o) \approx \sum_{l=0}^L \sum_{m=-l}^l c_{lm}(\mathbf{x}) L_{lm} \quad (3)$$

This efficient computation enables real-time updates as lighting conditions change dynamically, thereby supporting complex GI effects without expensive per-frame calculations.

1.4 Limitations of PRT

The traditional PRT method, while efficient in capturing GI effects, has several inherent limitations that restrict its applicability in dynamic and interactive environments. One of its primary constraints is the assumption of static geometry, as any modifications to the scene, such as object movement, deformation, or destruction, invalidate the precomputed data, making it unsuitable for dynamic settings. Additionally, while PRT allows for dynamic lighting, its reliance on low-order SH limits the representation of high-frequency details, such as sharp shadows and specular highlights, causing visual artifacts or blurring effects. Higher-order expansions improve detail but significantly increase computational and memory costs, reducing efficiency in real-time applications. Another major limitation is the inability of PRT to accommodate dynamically changing objects and mate-

rials, meaning that any alterations to the reflectance properties of a surface or newly introduced objects are not accurately reflected in the GI calculations.

Furthermore, the precomputation phase is computationally intensive and requires substantial processing time and memory storage, especially in complex environments with high geometric detail. For example, in the case of a relatively simple geometry consisting of approximately 672 vertices, the PRT precomputation phase requires around 100 seconds. In contrast, for more complex models containing 49,671 vertices, the required precomputation time increases dramatically to roughly 11,000 seconds (approximately three hours) [22]. As the geometric complexity and vertex count continue to grow, the precomputation time escalates rapidly, rendering PRT increasingly impractical for detailed or large-scale scenes. These constraints become even more pronounced in VR applications, where real-time adaptability is essential for user immersion. Traditional PRT struggles to account for parallax effects, viewpoint-dependent lighting changes, and user-driven interactions, all of which are crucial in VR rendering. Lastly, VR frame rate demands (often exceeding 90 FPS) impose additional performance constraints, making static precomputed solutions less viable.

Finally, to the best of our knowledge, no complete implementation of PRT currently exists within the Unity game engine. The only related identified attempt [1] remains incomplete and, moreover, inherits the inherent limitations of PRT, with the computationally intensive precomputation phase representing the most significant bottleneck.

Despite these limitations, PRT remains a key method for achieving efficient and realistic GI using precomputed radiance data. By harnessing precomputed data and low-frequency lighting representations, PRT exemplifies how innovation can bridge the gap between performance and visual fidelity in computer graphics.

However, these constraints also expose a clear research gap: existing PRT methods do not provide a scalable solution that can adapt to dynamic geometry, evolving materials, and the stringent real-time performance requirements of modern interactive applications. Addressing this gap requires new approaches that retain PRT efficiency while extending its flexibility to dynamic, real-time, and high-frequency lighting conditions.

To address the limitations of PRT in dynamic settings, learning-based methods have emerged as promising alternatives. Multilayer Perceptrons (MLPs) offer a simple means to approximate radiance transfer directly from geometric features, reducing reliance on precomputed data. Graph Neural Networks (GNNs) go a step further by naturally exploiting the mesh connectivity of 3D objects, capturing spatial relationships that fully connected architectures cannot. These neural approaches provide a path toward more adaptive and scalable solutions

that preserve the efficiency of PRT while extending its applicability to real-time and interactive environments. Significant contributions that apply deep learning to GI include methods such as *NeuralPRT* [64], *DeepPRT* [48], and *PlenOctrees* [90], among others, which will be reviewed in the subsequent sections.

Recent neural rendering approaches, such as Neural Radiance Fields (NeRFs) [3] and 3D Gaussian Splatting (3DGS) [43], have achieved remarkable realism by learning scene appearance directly from data. However, these methods primarily operate in image space and remain static once trained, lacking adaptability to changing geometry or lighting conditions. This limitation underscores the need for geometry-aware neural representations that operate directly on 3D meshes and support real-time interaction.

1.5 Toward a Neural Formulation of PRT

To overcome the static and computationally expensive nature of traditional PRT, this thesis introduces Neural-GASh [22], a neural, geometry-aware reformulation of radiance transfer that operates directly on mesh vertex data. Unlike previous methods such as NeuralPRT [64], which operates on 2D rendered images, and DeepPRT [48], which trains on image-based coefficient maps, both of which lack compatibility with real-time applications and game engine integration, the proposed Neural-GASh framework employs Conformal Geometric Algebra (CGA) as its input representation. This formulation enables a unified encoding of vertex position and orientation directly on 3D meshes, allowing seamless deployment within the Unity game engine and across a broad range of Extended Reality (XR) platforms. This representation allows the model to capture both spatial and angular relationships consistently across arbitrary geometries.

Neural-GASh further departs from previous approaches by incorporating GNNs that exploit mesh connectivity, allowing the model to reason over local neighborhoods and infer occlusion relationships that per-vertex models cannot capture. This combination of CGA-based encodings and topology-aware learning provides a scalable, physically grounded, and real-time-capable alternative to traditional precomputation-based PRT. As such, the proposed framework bridges classical physically based rendering with modern geometric deep learning, establishing a path toward real-time, neural GI suitable for XR environments.

1.6 Thesis Overview

In this thesis, we focus on the PRT algorithm, with the aim of analyzing its characteristics and addressing its limitations in dynamic and real-time rendering scenarios. While PRT efficiently captures GI effects, its reliance on precomputa-

tion restricts adaptability to changing geometry, materials, and lighting, which are fundamental requirements in interactive applications such as video games and VR. To overcome these challenges, we investigate the use of neural networks (MLPs) and deep learning—specifically GNNs—as a means to predict radiance transfer directly from geometric representations. GNNs are particularly well-suited for this task, as they naturally exploit the graph structure of 3D meshes while maintaining scalability and efficiency. By bridging PRT with GNN-based learning, our goal is to achieve both physical accuracy and real-time performance, enabling seamless deployment within modern game engines.

The remainder of this thesis is organized as follows. Section 2 presents a review of the relevant state of the art, covering prior work on PRT, recent advances in neural rendering, and the emerging technique of Gaussian Splatting, which currently yields some of the most realistic results in novel view synthesis. Section 3 details the methodology developed to extend traditional PRT into a neural, geometry-aware, and XR-compatible lighting framework, outlining the motivation behind this work and the specific limitations it aims to address. Section 4 introduces our first approach to combining real-time shading with PRT through Neural-GASh, a NeRF that integrates neural networks with the core principles of PRT, along with an evaluation of its results. Building upon this, Section 5 presents the evolved version of Neural-GASh, incorporating advanced deep learning architectures and establishing compatibility with game engines, with extensive results demonstrating its capabilities. Finally, Section 6 provides the discussion and conclusions, highlighting the limitations of the current work and outlining promising directions for future research.

2 State-of-the-art

The study of GI has been a central theme in computer graphics for decades, with a wide variety of algorithms proposed to approximate solutions to the rendering equation. Techniques range from offline methods such as path tracing and photon mapping, which prioritize physical accuracy, to interactive and real-time methods that balance visual quality with strict performance constraints. Within this spectrum, PRT has emerged as a key technique, enabling the approximation of complex light transport for static scenes through precomputation and compact representations such as SH.

In recent years, the field has been further transformed by the rise of neural rendering, where machine learning methods are leveraged to approximate or accelerate traditional rendering pipelines. Approaches such as NeRFs and, more recently, Gaussian Splatting (GS), have demonstrated remarkable capabilities in synthesizing novel views and capturing realistic lighting effects with unprecedented efficiency. These advancements highlight the growing intersection between computer graphics and deep learning, opening new possibilities for extending classical methods like PRT to modern real-time applications.

2.1 Precomputed Radiance Transfer

PRT [75] is a family of techniques that approximate GI by representing light transport through precomputed basis functions such as SH. This formulation allows complex lighting effects to be captured efficiently at runtime through a compact per-vertex coefficient representation. Over the years, numerous extensions have been developed to overcome PRT’s inherent static assumptions and computational costs, enabling its use in more dynamic and data-driven rendering pipelines.

2.1.1 Extensions to Precomputed Radiance Transfer

To address these limitations, several extensions and reformulations of the PRT framework have been proposed.

The incorporation of Conformal Geometric Algebra (CGA) into the PRT framework (Figure 4) marks a paradigm shift in how light transport simulations are conceptualized and implemented [56]. As discussed in the recent survey by Hitzer et al. [35], CGA provides a powerful algebraic foundation for unifying geometric representation and transformation, making it particularly suited for real-time graphics applications. By representing SH rotations with CGA rotors, the algorithm minimizes the number of computations required, optimizing per-

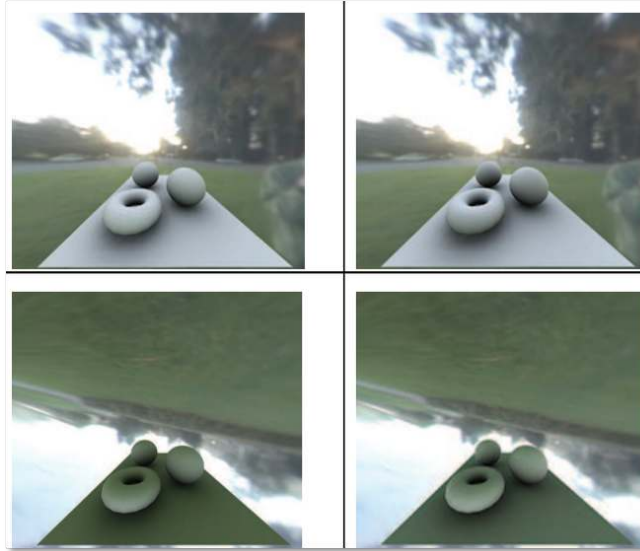


Figure 4: The representation of a simple scene, lit by the PRT algorithm using CGA. [56]

formance without compromising accuracy. This efficiency enhancement is particularly advantageous in real-time rendering scenarios, where computational speed is paramount. However, despite its strides in efficiency, the CGA-enhanced PRT model is not without its constraints. Chief among these limitations is its inability to support translation and scaling of SH coefficients. Consequently, the algorithm is ill-suited for scenarios requiring dynamic transformations of objects or changes in scale. Furthermore, like its predecessor, this adapted PRT model does not support deformations or animations. While it excels in rendering static scenes with complex lighting interactions, its applicability is limited in dynamic environments where objects undergo continuous changes.

Despite these limitations, the integration of Conformal Geometric Algebra into the PRT algorithm represents a significant step forward in the quest for efficiency and realism in GI. By leveraging the power of CGA to enhance the results, this innovative approach offers a glimpse into the future of real-time rendering, where performance and visual quality converge in perfect harmony.

In the ongoing pursuit of efficiency and fidelity in GI, the introduction of PRT Textures (PRTT) [18] represents a significant advancement (Figure 5). Designed to efficiently store transfer values, PRTT extend the capabilities of the PRT framework, thereby enabling higher rendering quality while maintaining computational efficiency.

PRTT represent a significant expansion of the PRT paradigm, facilitating the storage and retrieval of transfer values essential for simulating both diffuse and glossy reflections. This expansion not only broadens the scope of PRT applications but also enables the rendering of high-quality local effects, including glossy

renders and intricate interreflections. A standout feature of PRTT is their ability to achieve superior rendering quality, particularly on low tessellation meshes. By leveraging precomputed transfer values stored within textures, PRTT elevate the visual fidelity of digital scenes, imbuing them with a level of detail and realism previously unattainable. Furthermore, PRTT exemplify the delicate balance between computation effort and rendering quality. Despite the complexity of GI calculations, PRTT maintain real-time frame rates, ensuring smooth and immersive user experiences without compromising on visual excellence. Despite their advantages, PRTT are not without inherent limitations. In transfer textures, interreflections effectively bake the lighting and BRDF into the texture data. Consequently, changes to interreflections require re-computation, limiting the flexibility of the rendering process.

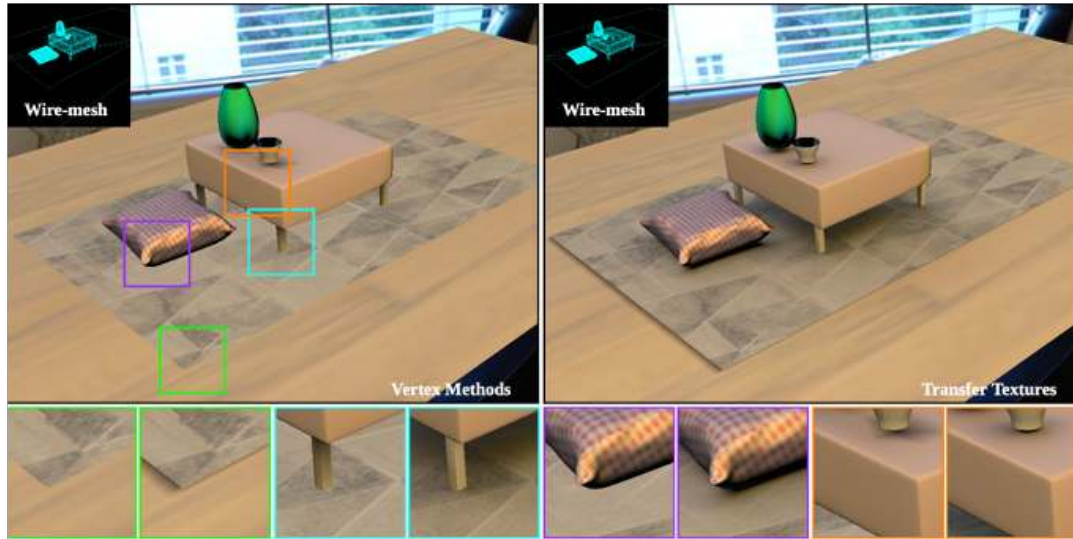


Figure 5: An example of PRTT usage. [18]

Despite this limitation, the introduction of PRTT represents a significant stride forward in the realm of GI. By encoding transfer values into textures, PRTT improve the efficiency of storage and retrieval, offering a practical means of balancing rendering quality and computational cost in real-time scenarios.

In the ever-evolving landscape of computer graphics, the PRT paradigm takes on new dimensions as it ventures into the realm of reflectance and lighting estimation [79] (Figure 6). This innovative application of PRT not only advances the field of relighting and content creation but also opens avenues for sophisticated light transport effects and enhanced rendering quality. At its core, PRT for reflectance and lighting estimation serves as a powerful tool for dissecting scenes into their constituent elements: reflectance properties and lighting conditions. This decomposition is fundamental for tasks such as relighting, where the ability to manipulate lighting independently from surface properties is paramount

for creative expression and content manipulation. Moreover, PRT for reflectance and lighting estimation goes beyond simple illumination models, accounting for advanced light transport effects such as occlusion and indirect lighting. By capturing the nuances of light propagation within a scene, this approach achieves a level of realism and fidelity crucial for immersive digital experiences. Central to this methodology is the utilization of PRT techniques to estimate reflectance and lighting from geometry and images. By leveraging precomputed data, PRT enables the reconstruction of surface reflectance properties and colored lighting environment maps, providing a comprehensive understanding of scene illumination.

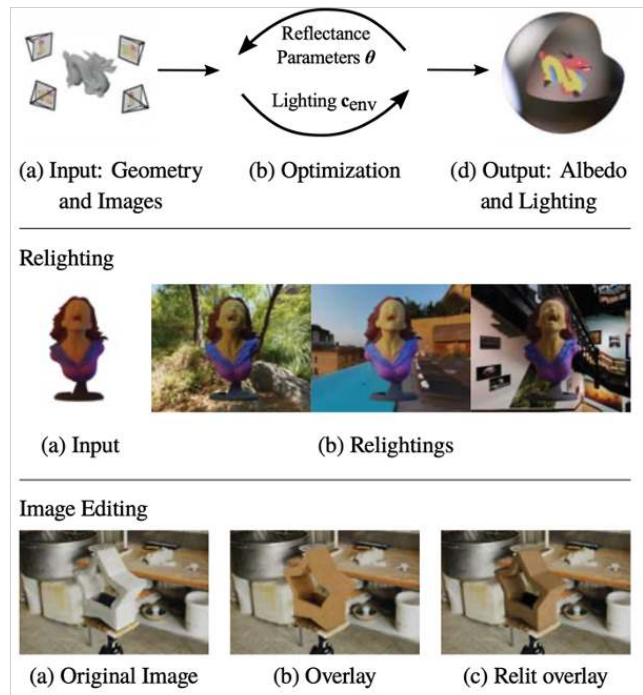


Figure 6: The main concept of the PRT for Reflectance and Lighting Estimation. [79]

One of the key strengths of PRT for reflectance and lighting estimation lies in its ability to improve both qualitative and quantitative results by incorporating indirect light. By considering the indirect contributions of light within a scene, this approach yields more accurate and visually pleasing renderings, enhancing the overall realism of digital content. However, despite its numerous advantages, the current approach faces challenges in representation. The reliance on SH basis for representation may not fully capture the complexity of certain scenes, leading to potential inaccuracies in estimation.

Moreover, the data-driven paradigm for PRT [5] sets the stage for future advancements in the field, laying the groundwork for machine-learned methods to augment and refine traditional rendering techniques. By embracing the power of data-driven approaches, PRT enters a new era of innovation and adaptability,

poised to meet the evolving demands of modern rendering pipelines. However, despite its promise, the data-driven paradigm for PRT has its limitations. Large assets, such as expansive game levels, may pose challenges in implementation due to computational constraints. Additionally, the current approach does not support the rendering of specular reflections, limiting its applicability in scenarios where such effects are prominent.

Looking ahead, addressing these limitations will be paramount for the widespread adoption and continued evolution of the data-driven paradigm for PRT. Future research efforts may focus on optimizing computational efficiency to accommodate larger scenes and refining algorithms to incorporate specular reflections, thus expanding the applicability and versatility of this innovative approach. The data-driven paradigm for PRT represents a significant leap forward in the evolution of real-time rendering techniques. By leveraging the power of data and machine learning, this approach offers a compelling solution to longstanding challenges in PRT development, ushering in a new era of efficiency, realism, and adaptability in digital content creation. As research and development in this area continue to progress, we anticipate further breakthroughs that will reshape the landscape of computer graphics and redefine the possibilities of interactive digital experiences.

Further optimizations of PRT for real-time rendering include MoMo-PRT [70], which employs carefully optimized data structures to approximate complex lighting effects such as indirect illumination and soft shadows. By relying on classical graphics techniques rather than neural methods, MoMo-PRT achieves interactive performance without specialized hardware. However, the reliance on precomputation continues to constrain its use in dynamic scenes, and challenges remain with respect to high-frequency lighting and efficient memory usage in large environments.

2.2 Neural Rendering

In recent years, neural networks have emerged as a powerful tool in computer graphics, offering novel solutions to long-standing challenges in real-time rendering, GI, and high-fidelity image synthesis. Traditional rendering techniques, including PRT, rely on complex mathematical models and precomputed data structures to approximate GI effects efficiently. However, these approaches often suffer from limitations such as static scene assumptions, high precomputation costs, and difficulties in handling dynamic lighting and geometry. Neural networks provide an alternative paradigm by leveraging data-driven models to learn complex light transport behaviors, enabling faster, more adaptable, and more scalable rendering techniques.

Deep learning-based methods have been applied to various aspects of computer graphics, including real-time shading, denoising, light transport approximation, and even full-frame neural rendering. One of the key advantages of neural networks in graphics is their ability to generalize from training data and interpolate complex functions, making them particularly useful for approximating high-dimensional problems such as radiance transfer. By learning scene-dependent or scene-independent representations, neural networks can significantly reduce the memory footprint and computational overhead required for traditional methods. Furthermore, modern GPU architectures, optimized for deep learning workloads, provide efficient inference capabilities that allow neural network-based models to be deployed in real-time applications such as game engines, virtual reality, and augmented reality.

2.2.1 Advances in Deep Learning for Radiance Transfer

DeepPRT [48], introduces a deep convolutional neural network designed to efficiently encapsulate the radiance transfer characteristics of freely deformable objects for real-time rendering. Traditional PRT methods often struggle with dynamic deformations due to their reliance on static precomputations. DeepPRT addresses this limitation by learning a compact representation that can adapt to various deformations, enabling real-time rendering of complex lighting effects such as soft shadows and interreflections on deformable objects. This approach leverages the power of neural networks to generalize across different deformation states, offering a more flexible and efficient solution for rendering deformable objects under dynamic lighting conditions.

NeuralPRT [64] explores the integration of neural networks with traditional PRT techniques to achieve real-time rendering of GI in static scenes under dynamic environment lighting. The authors investigate various neural network architectures inspired by PRT principles, aiming to efficiently learn light transport and enhance rendering quality without relying on high-end ray-tracing hardware. Their findings indicate that architectures incorporating light transport models and PRT-inspired designs significantly improve GI predictions, maintaining real-time performance and demonstrating the potential of neural approaches in advancing interactive graphics. A key limitation of this approach is that it primarily operates on images rather than directly manipulating 3D geometry. While traditional PRT methods compute radiance transfer at the vertex or voxel level, NeuralPRT instead learns a mapping from lighting conditions to final rendered images, bypassing explicit per-vertex calculations. This image-based approach limits its ability to handle dynamic geometry, as the model is trained on static 3D scenes and cannot easily adapt to structural changes in real time. Addition-

ally, since it encodes 3D light transport into a neural representation, the method sacrifices geometric flexibility in favor of efficient rendering, making it less suitable for applications requiring interactive object manipulation or real-time scene modifications.

The PlenOctrees [90] method accelerates NeRFs leveraging real-time rendering. Traditional NeRFs, while highly effective at synthesizing novel views with detailed geometry and lighting, suffer from extreme computational costs, often taking 30 seconds per frame to render, making them impractical for interactive applications. To address this, the authors introduce PlenOctrees, an octree-based representation that enables rendering at over 150 FPS, making NeRF-based models usable for real-time applications such as VR/AR and industrial visualizations. The key innovation lies in pre-tabulating the NeRF data into an octree structure and using SH to efficiently store and reconstruct view-dependent effects, removing the need for costly per-query neural network evaluations. Additionally, the paper demonstrates how direct octree optimization can further refine the representation while reducing NeRF training time. Experimental results show that PlenOctrees achieve rendering speeds over 3000 times faster than conventional NeRFs, with comparable or superior image quality. This work represents a significant step toward making neural rendering feasible for real-time applications and highlights the potential of hierarchical volumetric representations in neural graphics. Despite achieving state-of-the-art rendering performance and interactive frame rates, PlenOctrees exhibit several limitations that constrain their broader applicability. First, the octree-based representation incurs a significantly larger memory footprint compared to the more compact NeRF models, with uncompressed models averaging 1.93 GB on synthetic datasets and 3.53 GB on real-world datasets such as Tanks and Temples. Although compressed, lower-resolution variants (30–120 MB) can be used for online delivery, this reduction often comes at the expense of visual fidelity. Second, the method demonstrates reduced effectiveness when applied to unbounded or forward-facing scenes, as the underlying data distribution differs from that of bounded volumes. This limitation restricts the ability of PlenOctrees to generalize across diverse scene types. Finally, forward-facing scenes inherently do not support full six degrees of freedom (6-DOF) viewing, limiting interactivity; in such cases, alternative representations such as Multi-Plane Images (MPIs) are more appropriate.

Ruhe et al. [67] present a novel approach to integrating geometric algebra (Clifford algebra) into neural network architectures, allowing for a more structured and mathematically grounded representation of geometric transformations. Unlike traditional deep learning methods that rely on coordinate-based representations, this work leverages group action layers to model transformations inher-

ently, enabling more robust learning of symmetries and spatial relationships. The authors introduce specialized activation and normalization functions tailored to geometric algebra, ensuring stability and efficiency in training. Their approach is particularly effective in tasks involving 3D rigid body transformations and fluid dynamics simulations, where traditional neural networks struggle to maintain consistency and interpretability. By embedding geometric principles directly into neural architectures, this work opens new possibilities for improved generalization, efficiency, and interpretability in deep learning applications that involve complex spatial transformations. The primary limitation of Geometric Clifford Algebra Networks (GCANs) lies in the computational density of their underlying algebraic operations. This high compute density often results in slower run-times and increased memory consumption during gradient computation, which can hinder scalability. While, in principle, such dense operations could be advantageous on modern accelerators such as GPUs, realizing these benefits typically requires the development of custom GPU kernels or advanced compiler-based kernel fusion techniques. In the absence of these optimizations, training efficiency may lag behind comparable non-GCAN layers. Nevertheless, recent efforts have demonstrated notable progress, with convolutional GCAN layers achieving training speeds only marginally slower (approximately $1.2\times$) than standard PyTorch implementations, representing an order-of-magnitude improvement compared to earlier baselines.

DeepPR [28] is a novel framework for Diminished Reality (DR), enabling the seamless removal of real-world objects while reconstructing the occluded background with high fidelity. Unlike conventional inpainting techniques that often struggle with maintaining geometric consistency, DeepDR leverages a structure-aware generative network to jointly predict color and depth, ensuring visually and structurally plausible scene reconstructions. This method effectively preserves sharp edges and intricate textures, addressing common artifacts found in prior approaches. By integrating scene semantics into the inpainting process, DeepDR achieves real-time performance with minimal temporal inconsistencies, making it well-suited for interactive applications. Experimental evaluations demonstrate significant improvements over existing DR techniques, both in terms of qualitative appearance and quantitative metrics, marking a step forward in realistic background restoration. Despite achieving competitive results, DeepDR exhibits several notable limitations when applied to challenging real-world scenarios. For instance, in the presence of large diminished regions in front of highly irregularly textured objects (such as carpets), the method frequently produces structural artifacts, whereas competing approaches tend instead to oversmooth the background. Furthermore, DeepDR—like other comparable methods—struggles to

reconstruct highly ambiguous object boundaries, often resulting in color bleeding or implausible edges. An additional limitation arises from the characteristics of the InteriorNet dataset used for training: since the dataset lacks ground truth for object removal and the mask generation procedure does not account for shadows cast by removed objects, the model fails to learn accurate shadow removal. This leads to further ambiguities at shadow borders, as shaded and unshaded regions usually belong to the same semantic class, thereby amplifying the likelihood of artifacts.

Yao et al. [88] provide a detailed overview of recent advancements in NeRFs. The authors examine the fundamental architecture of NeRF and discuss various improvements and their applications in areas such as 3D scene reconstruction, novel view synthesis, human modeling, and robotics. The paper also reviews essential datasets and evaluation metrics used for training NeRF models. It concludes with an analysis of existing challenges and future research directions, offering valuable insights for researchers aiming to advance NeRF-based visual rendering technologies.

2.3 Gaussian Splatting

Beyond PRT and its neural extensions, recent advances in neural rendering have been driven by Gaussian Splatting (GS), which is a recently introduced technique for novel view synthesis and real-time rendering that has rapidly gained attention in the graphics community due to its ability to combine photo-realistic quality with interactive frame rates [89]. 3D Gaussian Splatting (3DGS) is a technique for representing and rendering scenes by modeling them as a collection of learnable 3D Gaussians, enabling high-quality real-time rendering. Unlike implicit methods such as NeRFs [3], which encode scene information within a neural network, 3DGS explicitly stores position, orientation, scale, and color information in a set of Gaussian primitives, allowing for direct rasterization without requiring volumetric sampling (Figure 7). Each Gaussian is parameterized by its mean, covariance, and opacity, enabling smooth blending and anti-aliased rendering. The rendering process involves projecting these Gaussians onto a two-dimensional image plane, where they are accumulated and composited to produce photorealistic images. This approach achieves significant efficiency gains by leveraging differentiable rasterization, which directly optimizes Gaussian parameters through gradient descent, making it suitable for real-time applications such as novel view synthesis, dynamic scene reconstruction, and interactive 3D editing. Kerbl et al. [43] introduced a real-time 3DGS pipeline that efficiently reconstructs high-quality scenes from multi-view images, demonstrating its superiority

over traditional neural rendering methods in terms of both speed and quality. The explicit nature of 3DGS enables superior editability and adaptability, making it a promising technique for next-generation graphics rendering, AR, and VR.

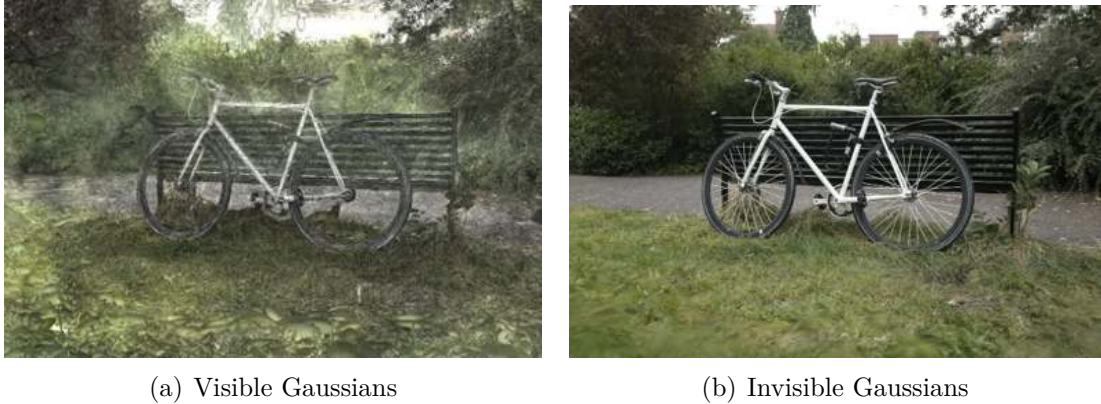


Figure 7: Example results from Gaussian Splatting. (a) Rendering with visible Gaussians highlights the overdraw and transparency artifacts produced by the representation. (b) Rendering with invisible Gaussians produces a clean and realistic reconstruction of the scene, where the Gaussian primitives are imperceptible. [43]

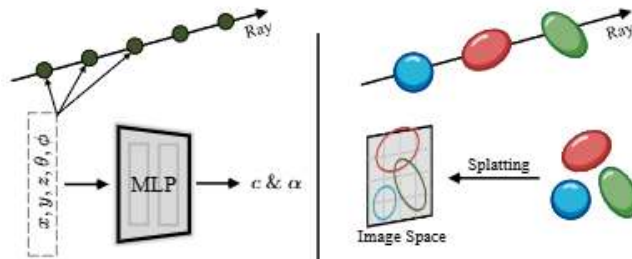


Figure 8: NeRF (left) versus 3DGS (right). [11]

While 3DGS represents a significant advancement over NeRFs by enabling real-time rendering with high visual fidelity, it has its limitations. A primary concern lies in memory usage: the representation relies on millions of Gaussian primitives, which results in a large memory footprint compared to NeRF’s more compact MLP-based formulation. This issue becomes particularly pronounced when scaling to large or highly detailed environments, where storage and transmission costs can become prohibitive. Although compression techniques have been proposed, they often trade off detail and accuracy, reducing the method’s efficiency in practice.

Another limitation of 3DGS is its reliance on static scenes. Similar to PRT and NeRF, the method assumes fixed geometry and materials during optimization. Once the Gaussians have been fitted to a scene, any modification of geometry, lighting, or material properties requires a costly re-optimization. As such, the technique currently struggles to handle dynamic scenes, deformable objects,

or user-driven interactions, which are essential in real-time applications such as games or VR.

Furthermore, while 3DGS achieves interactive frame rates, the rendering complexity is tied to the sheer number of Gaussian primitives. In practice, this can lead to bottlenecks in scenarios requiring real-time adaptability, especially on devices with limited GPU resources. Managing these primitives efficiently—for example, through pruning, level-of-detail techniques, or dynamic streaming—remains an open research challenge.



Figure 9: A scene that was reconstructed with the 3DGS technique. [43]

2.3.1 Advancements in the field of 3DGS

Recent studies have explored various aspects of 3DGS. For instance, Chen et al. [11] provide a comprehensive review of 3DGS. The survey explores the fundamental concepts behind 3DGS, examines its applications in areas such as real-time rendering and scene editability, and compares various state-of-the-art models across benchmark tasks. Additionally, the authors discuss existing challenges and outline potential future research directions, highlighting the significance of 3DGS in advancing 3D reconstruction and representation techniques. Liang et al. [49] extended 3DGS to dynamic scenes, incorporating a temporal component to model complex motions efficiently. Finally, Zhang et al. [92] propose a relightable view synthesis framework that combines 3DGS with high-order SH, enabling efficient GI with dynamic relighting. While capable of producing high-quality results, this hybrid approach is constrained to diffuse materials, lacks support for specular and view-dependent effects, and is limited in reproducing high-frequency details such as sharp shadows, owing to the low-frequency nature of SH. These advancements underscore the versatility of 3DGS in addressing challenges associated with real-time rendering, dynamic scene reconstruction, and mixed reality applications. The explicit nature of 3DGS allows for improved rendering speed, memory efficiency, and editability, making it a promising technique for next-generation 3D

representation and rendering.

Building upon the analysis of existing GI approaches, it becomes clear that despite their significant contributions, each method suffers from fundamental limitations that hinder their adoption in highly dynamic and interactive environments. Techniques such as PRT struggle with staticity and heavy precomputation costs, while methods like 3DGS and PlenOctrees, although powerful, face scalability issues in memory usage and adaptability to dynamic scenes. Similarly, neural approaches often introduce additional computational complexity or hardware dependencies that limit seamless integration into real-time applications. In this thesis, we propose a dedicated pipeline for the PRT algorithm that specifically addresses these challenges, with a primary focus on overcoming the static geometry assumption, mitigating the computational bottleneck of precomputation, and enabling robust multiplatform deployment. By bridging classical rendering theory with deep learning methods, our goal is to preserve the efficiency and physical grounding of PRT while extending its applicability to modern real-time scenarios such as video games, VR, and other interactive applications.

2.4 Real-Time Rendering in XR and VR Environments

Real-time rendering lies at the core of XR systems, encompassing Virtual, Augmented, and Mixed Reality (VR/AR/MR). In these environments, perceptual realism, visual latency, and interaction responsiveness directly determine user immersion and training efficacy. Achieving physically grounded illumination in XR remains a long-standing challenge, as rendering pipelines must balance high-quality GI with stringent real-time performance requirements.

Several works from our research group involve the development of interactive XR applications, some in the context of cultural heritage [20], building upon prior studies [57, 6] and experimental developments in augmented reality authoring and interaction [24, 55], among others. Extending this line of research into the medical training domain, the MAGES 4.0 framework [95], a direct evolution of MAGES 3.0 [58], introduced a modular VR authoring platform that democratizes the creation of medical training simulations through optimized real-time rendering and physically based shading. Similarly, Covid-19 VR Strikes Back [93] and its follow-up usability study [94] leveraged efficient rendering and behavioral analytics to deliver scalable VR-based medical training for pandemic response scenarios. These projects highlight the importance of lightweight and robust rendering solutions capable of maintaining realism and interactivity under strict computational budgets.

Several XR-targeted systems have already explored neural rendering and Gaus-

sian splatting in immersive contexts. For example, the VRSplat method [80] allows interactive manipulation of Gaussian kernels in VR with dynamic shading and real-time performance. On the NeRF side, VR-NeRF [87] presents a dual-display VR extension allowing real-time novel-view synthesis with optimized NeRF pipelines. Meanwhile, Re-ReND [66] enables real-time NeRF rendering on devices including AR/VR headsets by converting radiance fields into efficient scene representations.

These recent efforts indicate that NeRF and 3DGS techniques are beginning to find applications within XR and VR contexts. However, key challenges remain, including real-time lighting consistency, efficient GI across complex geometry and materials, and low-latency shading suitable for head-mounted displays. The Neural-GASh framework is motivated precisely by these gaps, aiming to introduce SH-based neural lighting into XR-compatible, geometry-aware rendering pipelines that support both static and skinned geometry in real time. Together, these developments highlight the ongoing convergence of neural rendering, GI, and XR technologies, underscoring the need for hybrid approaches—such as the Neural-GASh framework presented in this thesis—that unite physical accuracy with real-time adaptability to bridge the gap between precomputed lighting models and dynamic, interactive virtual environments.

2.5 Identified Research Gaps

Table 1 summarizes the key characteristics of representative GI and neural rendering methods reviewed in this section. The comparison illustrates the gradual evolution from precomputed, static approaches toward data-driven models that leverage neural representations. Despite this progress, existing methods remain limited in their ability to integrate geometric awareness, topology, and real-time deployment within a unified framework—motivating the development of the proposed Neural-GASh architecture.

Method	Input Domain	Model	Deformations	Topology	Engine Support	Limitations
PRT [75]	3D Mesh	SH Basis (Per Vertex)	No	No	Partial	Precomputation; static only
PRT+CGA [56]	3D Mesh	SH+CGA Rotors	No	No	Partial	No translation or scaling; static scenes
PRTT [18]	3D Mesh	Texture-Encoded SH	No	No	Yes	Lighting baked into textures
DeepPRT [48]	3D Mesh	CNN (Coeff. Textures)	Yes	No	No	Requires UV mapping; offline training
NeuralPRT [64]	2D Image	MLP/CNN	No	No	No	Operates in image space; limited geometric generalization
PlenOctrees [90]	Volumetric	Octree+SH	No	No	No	Memory; bounded scenes only
3DGS [43]	Point Cloud Multi-view	Explicit 3D Gaussians	No	No	Partial	Memory; static scenes

Table 1: Comparative summary of representative GI and neural rendering methods.

The preceding review highlights several persistent gaps in the field of GI and neural rendering, which directly motivate the design of the proposed framework.

Gap 1: Lack of geometry-aware neural GI methods. Existing neural approaches to radiance transfer differ in their operating domain and input representation. NeuralPRT [64] predicts shading in image space using G-buffer features and environment lighting, while DeepPRT [48] operates on meshes but performs training in UV or texture space, predicting SH coefficient textures that are later mapped back onto geometry. Although both successfully learn low-frequency light transport, neither operates directly in 3D geometric space. As a result, they cannot explicitly reason about spatial and angular relationships between vertices, nor generalize effectively to deformations or topological changes. This limitation prevents deployment in real-time 3D engines, where vertex-level computation and dynamic lighting updates are required.

Gap 2: Absence of topology-aware neural inference. No existing method integrates vertex-level neural inference with mesh connectivity in a geometry-aware manner. Current neural GI models—whether MLP, CNN, or volumetric (NeRF, 3DGS)—either ignore mesh topology or represent geometry implicitly through dense sampling. This omission leads to limited spatial coherence and

poor modeling of local shadowing, adjacency, and self-occlusion, all of which are naturally encoded in the mesh graph. Moreover, such methods remain largely decoupled from real-time rendering pipelines and game engine integration, restricting their use in practical XR or interactive environments.

Proposed Contribution: Neural-GASh. The proposed framework, Neural-GASh, directly addresses these challenges by combining CGA encodings, GNNs, and real-time engine deployment. CGA provides a unified 32-dimensional representation that embeds both position and orientation, preserving conformal properties such as angles, tangency, and incidence that Euclidean coordinates cannot represent linearly. This enables the GNN to reason jointly about spatial and angular relationships across connected vertices. Through message passing over the mesh graph, Neural-GASh infers per-vertex radiance transfer while implicitly modeling self-occlusion—without requiring explicit ray tracing. Finally, the model is fully integrated into the Unity engine, supporting both static and skinned meshes across XR platforms. This integration of CGA-based vertex features, topology-aware inference, and runtime deployment constitutes a novel contribution to learning-based GI and real-time rendering.

3 Methodology

This chapter presents the unified methodology developed to extend traditional PRT into a neural, geometry-aware, and XR-compatible lighting framework. The aim is to preserve the physical accuracy and theoretical rigor of traditional PRT while eliminating its most restrictive limitation—the expensive, per-mesh precomputation that confines it to static scenes. The proposed Neural-GASh pipeline merges the physical foundations of light transport with modern geometric deep learning, enabling real-time radiance prediction across diverse and dynamic geometries.

The methodology follows a progressive design that begins with the development of a modular and transparent Python implementation of diffuse PRT, serving as both a validation tool and a generator of ground-truth supervision for neural training. For implementation details, see Appendix A. Building upon this foundation, the proposed *Neural-GASh* framework reformulates PRT as a learning-based problem. Neural networks are trained to predict SH lighting coefficients directly from CGA-encoded geometry, effectively bypassing the precomputation bottleneck. Each surface element, represented by a vertex-normal pair, is embedded into CGA, encoding both spatial position and orientation within a 32-dimensional multivector structure. This representation enriches the geometric feature space and allows the neural models to infer lighting relationships that remain consistent under geometric transformations.

Two complementary architectures are introduced: an MLP that performs efficient per-vertex inference suitable for real-time deployment, and a GNN that incorporates mesh connectivity to capture non-local light interactions such as self-occlusion and shadowing. Together, these models establish a continuum between local feedforward regression and topology-aware neural light transport.

The trained neural models are integrated into Unity, completing a full real-time lighting pipeline capable of operating on both static and skinned meshes. This deployment validates the practicality of neural PRT for XR applications, achieving physically accurate results with interactive performance. The main contributions of this thesis therefore include: the first documented Python-based PRT implementation, the introduction of CGA-based neural encodings for light transport, the design of the Neural-GASh MLP and GNN architectures, and their successful deployment in a real-time rendering environment. Collectively, these components form a coherent methodology bridging physically based rendering and geometric deep learning, advancing the development of interpretable and efficient neural global illumination.

3.1 Motivation and Overview

Traditional diffuse PRT provides physically robust global illumination but is hindered by long precomputation times and its inability to handle deformable or animated geometry. The objective of this work is to preserve PRT’s physical fidelity while enabling efficient, real-time, and geometry-agnostic operation. To this end, a modular Python implementation of diffuse PRT was developed to compute reference SH transfer coefficients up to order $L=2$ (nine basis terms) for both unshadowed and shadowed transport. The baseline, fully integrated with PyTorch, serves as a ground-truth generator for neural supervision and as a physically accurate standard for validation and ablation studies.

To quantify the computational cost of our Python-based PRT implementation, we benchmarked both the unshadowed and shadowed variants across meshes of increasing geometric complexity. Table 2 reports precomputation times (in seconds) for vertex counts ranging from $\sim 3K$ to over 70K. The unshadowed case scales approximately linearly with the number of vertices, since each vertex independently accumulates cosine-weighted SH contributions from sampled directions. By contrast, the shadowed variant grows significantly more costly due to the additional visibility checks, which require ray–triangle intersection tests against all faces of the mesh.

Vertices	Triangles	Complexity	Unshadowed (s)	Shadowed (s)
2868	5862		25.50	116.97

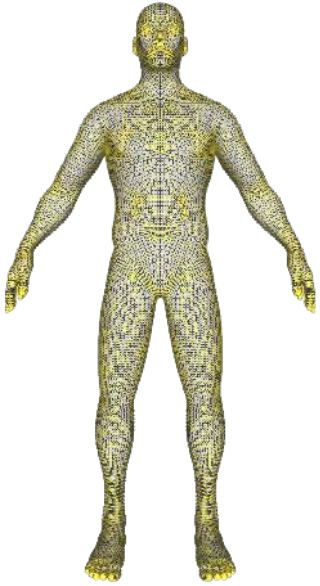
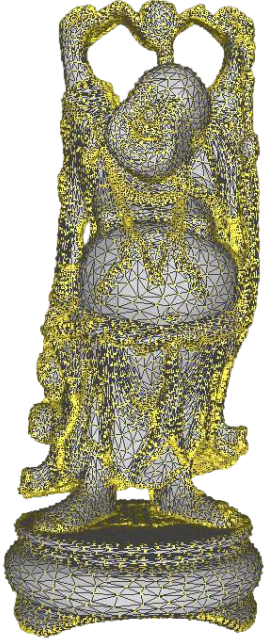
24461	48918		216.87	1408.67
49990	100000		445.91	2963.69
72027	144046		639.18	5009.21

Table 2: Precomputation times of the Python PRT implementation for meshes of varying vertex counts. Shadowed PRT incurs significantly higher cost due to ray-triangle visibility queries.

Figure 10 visualizes these results, highlighting the stark difference in scaling behavior between the two variants. The experiments were conducted on a workstation equipped with an AMD Ryzen 7 7700 eight-core processor operating at 3.80 GHz, 32 GB of system memory, and an NVIDIA GeForce RTX 4060 graphics card with 8 GB of dedicated VRAM. While unshadowed PRT already reaches several minutes for moderately complex meshes, shadowed PRT requires over an hour for 70K vertices, illustrating the impracticality of this approach for real-time or iterative workflows. The unshadowed and shadowed variants of PRT were implemented within our Python framework, representing original contributions developed specifically for this work.

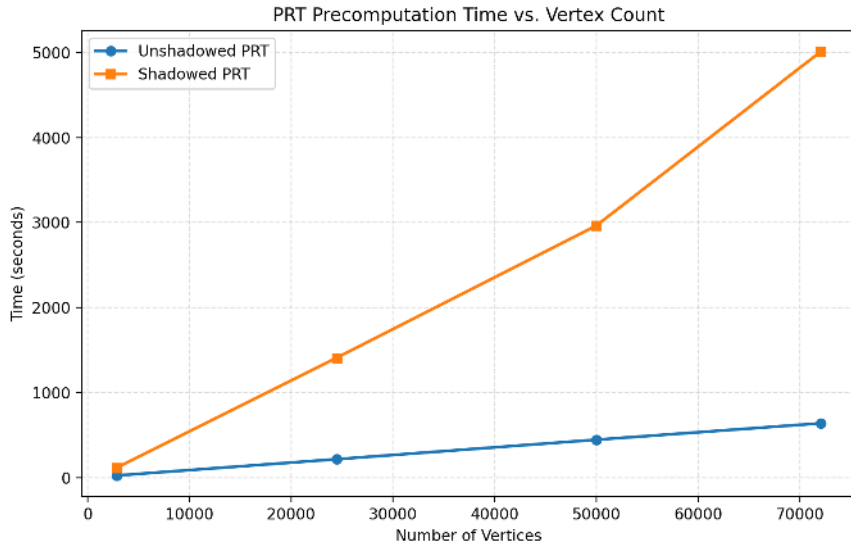


Figure 10: Scaling of unshadowed vs. shadowed PRT precomputation time with respect to vertex count. While both grow with complexity, shadowed PRT exhibits significantly steeper growth due to visibility tests.

While our implementation is written in Python and not optimized to the same extent as C++ counterparts (e.g., [75, 9]), the scaling behavior we observe with respect to vertex count is consistent with prior reports. This confirms that, although absolute runtimes are implementation-dependent, the overall computational complexity of PRT is correctly captured in our baseline. At the same time, the steep growth in precomputation times highlights a critical limitation: traditional PRT, even when parallelized, remains too expensive for real-time use. Reducing this cost is essential, since only then can PRT be extended to dynamic scenes and animations rather than being restricted to static, precomputed setups. This challenge directly motivates the neural approximations introduced in the following sections. Our Python-based implementation thus provides a flexible and extensible foundation, enabling the integration of GNNs into a real-time global illumination pipeline for dynamic environments.

Building upon this baseline, Neural-GASh was introduced as a neural reformulation of PRT capable of predicting per-vertex SH coefficients directly from CGA encodings. This allows the system to generalize across arbitrary mesh geometries—from simple objects to complex anatomical models—while reducing computation time by several orders of magnitude compared to traditional PRT. By encoding each vertex’s spatial and angular properties in CGA, Neural-GASh learns a richer mapping between surface structure and radiance transfer, improving prediction accuracy in concave and geometrically intricate regions.

In practice, the trained Neural-GASh models exhibit several desirable properties: they operate independently of mesh topology, support dynamic environmental lighting through SH rotation, and recompute per-vertex transfer coefficients in real time, allowing for animated or deformable geometry without pre-baked lighting. Although developed and trained entirely in Python, the models maintain full runtime compatibility with Unity, supporting deployment on both desktop and mobile XR platforms.

3.2 Problem Formulation

For each vertex v with normal n , the baseline PRT algorithm computes a lighting-independent SH transfer vector $\mathbf{t}^*(v) \in \mathbb{R}^9$. An HDR environment map is projected into SH form as $\mathbf{L} \in \mathbb{R}^{9 \times 3}$. The diffuse RGB irradiance at vertex v is then

$$\mathbf{C}(v) = \text{albedo}(v) \odot (\mathbf{t}(v)^\top \mathbf{L}).$$

Thus, the learning problem reduces to approximating the mapping $(v, n) \mapsto \mathbf{t}(v)$ so that the predicted coefficients $\mathbf{t}(v)$ closely reproduce $\mathbf{t}^*(v)$ while remaining computationally efficient for real-time use.

3.3 Geometry Encoding with CGA

Each vertex–normal pair (v, n) is represented as a 32-dimensional conformal multivector $\mathbf{m}(v, n) \in \mathbb{R}^{32}$. This dimensionality arises from the full geometric algebra $\mathbb{R}_{4,1}$, which has $2^5 = 32$ basis elements corresponding to all possible combinations of the five basis vectors (three spatial, one origin, and one infinity). Unlike standard Euclidean coordinates that represent only position and direction, the conformal model embeds points, lines, planes, and spheres within a unified algebraic space. This formulation allows geometric transformations—such as rotation, translation, scaling, and inversion—to be expressed as single algebraic operations (via versors), preserving incidence and metric relationships intrinsically. In practice, this means that the CGA encoding inherently captures both spatial and

angular relationships between vertices, providing a richer and more stable input representation than simple Euclidean concatenations. This results in smoother learning behavior for networks operating on 3D geometry, particularly in complex or concave regions where local coordinate systems vary sharply. The full derivation of the CGA encoding is provided in Chapter 4, Section 4.1.1, and additional background appears in Appendix C.

3.4 Neural Variants of PRT

Building on the previously described framework, the *Neural-GASh* system is instantiated through two neural formulations that approximate the PRT operator at different levels of geometric awareness. These models share the same learning objectives and data preparation pipeline but differ in how they represent spatial relationships across the mesh.

The first variant, detailed in Chapter 4, employs an MLP for efficient per-vertex prediction of SH coefficients, making it well suited for real-time engine deployment. The second, described in Chapter 5, extends this formulation using a GNN that propagates information over mesh connectivity, enabling the reconstruction of non-local radiance effects such as self-occlusion and soft shadowing.

3.4.1 Deployment Strategy

Two complementary runtime paths are supported. The first deploys the MLP model directly within Unity using the Barracuda inference engine, optimized for real-time performance. The second uses a lightweight Flask backend for GNN inference, where Unity transmits CGA features, receives SH coefficients asynchronously, and applies them for shading. Both pipelines share identical CGA encodings and SH environment data, ensuring consistency between offline and real-time evaluations.

3.4.2 Architectural Variants

On this foundation, two architectures are examined. Chapter 4 presents the per-vertex MLP model optimized for in-engine deployment, while Chapter 5 extends the formulation to graphs. In the latter, visibility is first estimated from CGA features and mesh connectivity, and the resulting cues inform the coefficient predictor—recovering complex shadowing and diffuse interreflections that per-vertex models cannot capture. Together, these variants demonstrate the transition from local regression to topology-aware neural light transport while remaining faithful to the physical formulation of diffuse PRT.

3.5 Summary

In summary, this methodology establishes the foundation for the Neural-GASh system by reformulating diffuse PRT into a learnable, geometry-aware process. The combination of CGA-based feature encoding and neural coefficient prediction enables physically consistent, real-time illumination adaptable to any mesh topology or deformation. The following chapters detail the MLP- and GNN-based implementations that realize this framework.

4 Neural-GASh (MLP-Based)

The Neural-GASh pipeline [22], designed for efficient radiance prediction with seamless integration into Unity, was recognized with the *ENGAGE 2025 Geometric Algebra Computing Award*, as documented in public announcement [82]. This award highlights both the novelty of the proposed approach and its ease of use within practical rendering pipelines.

The PRT algorithm demonstrates strong performance in the presented Python-based implementation; however, it presents certain limitations. The most significant challenges include the extended precomputation time, which can range from a few minutes to several hours or even days, depending on the complexity of the geometry and the computational capabilities of the hardware. Additionally, once the vertex coefficients of a model are computed, the resulting lighting remains static, meaning that the algorithm does not inherently support movement or animation of the geometry. This limitation poses a significant challenge that, to the best of our knowledge, has not yet been resolved.

To overcome these limitations, we propose a neural network-based approach that learns a function capable of predicting the PRT coefficients dynamically, allowing for real-time adaptability to geometry deformations and changing light conditions. By leveraging deep learning, we aim to approximate the radiance transfer function in a way that maintains both accuracy and real-time efficiency, enabling dynamic PRT without the need for costly recomputation. This approach bridges the gap between physically accurate precomputed lighting and the flexibility of real-time rendering, making PRT more suitable for interactive applications. We created two variants of PRT algorithm leveraging neural networks. The first one is Neural-GASh [22], a novel real-time shading pipeline for 3D meshes, that leverages a neural radiance field architecture to perform Image-Based Rendering (IBR), while utilizing a Multilayer Perceptron (MLP). The second approach of Neural-GASh will be described in section 5, where the MLP is replaced with Graph Neural Networks (GNNs).

However, it is important to recognize the inherent limitations of MLPs in this context. While effective as universal function approximators, MLPs treat vertices as independent inputs and are therefore unable to exploit the underlying mesh topology. This often leads to difficulties in capturing fine-grained self-occlusion or complex lighting interactions. The adoption of CGA representations mitigates some of these shortcomings by providing a richer, mathematically coherent input space, which stabilizes training and improves predictive accuracy. Nevertheless, further gains can be achieved by extending the framework beyond purely feed-forward architectures. In particular, GNNs naturally incorporate mesh con-

nectivity and neighborhood relationships, offering a promising path toward even more accurate and topology-aware radiance transfer prediction (Section 5).

Neural-GASH is a neural radiance field (see Figure 11), which replaces the rigid precomputation stage of traditional PRT with a flexible neural network capable of predicting radiance transfer in real time. Rather than relying on costly offline sampling and the generation of SH coefficients for every static or animated mesh, Neural-GASH learns a direct mapping from mesh features—specifically CGA-based representations of vertex positions and normals—to their corresponding SH lighting coefficients. In doing so, it eliminates the need for repeated precomputation whenever geometry changes, thereby offering a lightweight and scalable solution that is well suited for dynamic and deformable scenes. Although it does not fully replicate the photorealism and precision of traditional PRT, Neural-GASH significantly improves upon local illumination models such as Blinn–Phong, which, despite their efficiency, fail to capture the global interactions of light. Importantly, Neural-GASH supports real-time rendering of animated and rigged models, with CGA enabling efficient SH operations for dynamic lighting control. This makes the method particularly valuable for interactive graphics applications and performance-constrained environments.

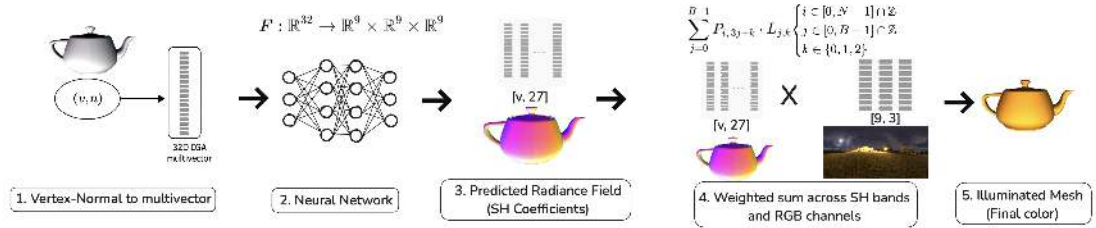


Figure 11: The neural radiance field architecture utilized in Neural-GASH.

Neural-GASH offers several important capabilities that extend its applicability beyond traditional PRT. First, it operates independently of the underlying geometry, whether simple or highly complex, as the neural network is trained to predict per-vertex SH coefficients in a fraction of the time required by conventional methods. The training process leverages CGA-based multivectors that encode vertex coordinates and normals, providing a richer geometric representation than classical vector-based inputs. This enables the model to learn lighting behavior across diverse spatial configurations with greater accuracy, leading to improved visual quality.

In terms of integration and deployment, Neural-GASH maintains full compatibility with the Unity game engine, despite being developed and trained in Python. This cross-platform design allows seamless incorporation into interactive applications and supports deployment on a broad range of platforms, including

resource-constrained mobile devices. By removing the costly precomputation step of PRT, the method significantly reduces the hardware requirements for real-time rendering. Furthermore, Neural-GASH supports dynamic environmental lighting by efficiently rotating the SH coefficients of radiance maps through CGA operations. These transformations are smooth, computationally inexpensive, and allow for real-time manipulation of complex illumination scenarios.

Finally, the method is designed to adapt to runtime geometry changes. It recomputes per-vertex coefficients efficiently when deformations or animations occur, enabling robust handling of continuously changing vertex positions and normals in dynamic scenes. Neural-GASH also achieves real-time performance in virtual reality environments, particularly when applied to lightweight meshes containing fewer than 1000 vertices, where smooth illumination updates are maintained even on performance-constrained VR hardware. Collectively, these features establish Neural-GASH as a flexible, efficient, and scalable extension of PRT, bridging the gap between physically based global illumination models and the performance demands of modern interactive graphics.

4.1 Euclidean vs CGA Input

In the initial stages of this work, the network was trained using raw Euclidean data: concatenated vertex positions and surface normals, forming a 6-dimensional input representation. While this encoding provided a simple description of the mesh geometry, the resulting predictions were often unstable and inaccurate, particularly in regions of complex curvature or self-occlusion. The limited feature space restricted the network’s ability to capture the intricacies of radiance transfer, leading to blurred or incorrect coefficient reconstructions.

To overcome these limitations, we adopted a Conformal Geometric Algebra (CGA) representation [34], where each vertex–normal pair is embedded as a 32-dimensional multivector. This switch provided the network with a significantly richer input space. Unlike simply concatenating additional Euclidean features, the CGA embedding ensures that the expanded representation remains mathematically coherent and encodes structured geometric relationships, while still being compatible with standard MLP architectures.

Empirical results confirmed the benefits of this approach: MLPs trained on 6D inputs frequently struggled to converge and produced perceptual artifacts, whereas networks trained on 32D CGA multivectors converged more reliably and yielded sharper, more accurate reconstructions [76, 51, 63]. By embedding the input into a higher-dimensional algebraic framework, the network gained access to a broader and more expressive representation, improving both stability and

predictive accuracy.

As illustrated in Figure 13, this shift enabled the method to handle complex or large-scale meshes more robustly. The adoption of CGA multivectors thus provided a principled way to enrich the input space, yielding consistently higher-quality results for Neural-GASH and supporting its integration into neural global illumination pipelines.

4.1.1 From Euclidean to CGA Representation

Each vertex–normal pair (v, n) , with $v \in \mathbb{R}^3$ and $n \in \mathbb{R}^3$, is mapped to a 32-dimensional conformal multivector that we use as network input. The construction proceeds as follows: (i) normal sanitization, (ii) assembly of a rigid motion, (iii) lifting this motion to a *motor* in CGA, and (iv) expansion in the 32-component basis of $\mathbb{R}_{4,1}$.

Normal sanitization. Given n , we ensure robustness and a consistent up-direction:

$$\hat{n} = \begin{cases} (0, 1, 0), & \text{if } \|n\| < 10^{-6} \text{ or } n \text{ is invalid,} \\ \frac{n}{\|n\|}, & \text{otherwise,} \end{cases} \quad \text{and set } \hat{n}_y \leftarrow |\hat{n}_y|.$$

Rigid motion. To align the canonical up vector $e_y = (0, 1, 0)^\top$ with the normalized surface normal $\hat{n}$, we compute the axis of rotation $\mathbf{u} = e_y \times \hat{n}$ and the rotation angle $\phi = \arccos(e_y \cdot \hat{n})$. Geometrically, $\mathbf{u}$ is the axis around which we must rotate, and ϕ is the angle between the two vectors. From these quantities, a 3×3 rotation matrix R is built using Rodrigues’ rotation formula [53], operated internally by the SciPy library [86]. This guarantees that the transformed up vector matches the surface normal, i.e. $Re_y = \hat{n}$.

Finally, the rotation is combined with the vertex position v into a 4×4 homogeneous transformation matrix

$$M = \begin{bmatrix} R & v \\ 0 & 1 \end{bmatrix}.$$

This matrix encodes a rigid-body motion consisting of the rotation R and the translation to the vertex position v .

CGA lifting: rotor and translator. In CGA, the 3D Euclidean space is embedded in a higher-dimensional space $\mathbb{R}^{4,1}$, a five-dimensional vector space with signature $(4, 1)$: four positive and one negative square norms. Its Clifford

algebra, denoted $\mathbb{R}_{4,1}$, has $2^5 = 32$ basis elements, which is why each multivector expands to 32 components. The basis is commonly written as $\{e_1, e_2, e_3, e_o, e_\infty\}$, where e_o and e_∞ are so-called null vectors.

The rotation is encoded by a *rotor*

$$R_{\text{CGA}} = \exp\left(-\frac{\phi}{2} \mathbf{u} I_3\right), \quad I_3 := e_1 e_2 e_3, \quad \mathbf{u} = u_x e_1 + u_y e_2 + u_z e_3,$$

where $\mathbf{u}$ and ϕ are the axis and angle extracted from R . The translation by $t = v_x e_1 + v_y e_2 + v_z e_3$ is encoded by the *translator*

$$T = \exp\left(-\frac{1}{2} t e_\infty\right).$$

The combined rigid motion is the *motor*

$$M_{\text{CGA}} = T R_{\text{CGA}}.$$

32D multivector expansion. Finally, M_{CGA} is expanded in the 32-dimensional blade basis of $\mathbb{R}_{4,1}$. Denoting by $\langle \cdot \rangle$ the coordinate along a given blade, we obtain

$$\mathbf{m} = (m_0, m_1, \dots, m_{31}) \in \mathbb{R}^{32},$$

where entries correspond to scalar, vector, bivector, trivector, 4-blade, and 5-blade coefficients in a fixed basis ordering. This 32D vector $\mathbf{m}$ is used as the network input.

4.2 Data formation

In the Neural-GASH framework, the neural network is trained to predict per-vertex SH coefficients for a given mesh. This stage constitutes the main computational bottleneck, as coefficient generation for complex geometries is time-intensive, precluding dynamic scene updates or geometry modifications during runtime.

Following the principles of traditional PRT [75], the vertex coefficients depend solely on geometric properties—specifically, vertex positions and surface normals—while remaining independent of the lighting environment. Consequently, the neural network inputs are constructed from vertex–normal pairs of the target mesh, whereas the lighting function (HDRI) is incorporated only during the rendering stage. The SH basis functions, being fixed for a chosen sampling and frequency configuration, are not learned but applied consistently across all meshes.

For this implementation, the SH order was set to $L = 2$, corresponding to the first three bands and $L^2 = 9$ coefficients. This configuration is widely rec-

ognized as sufficient for accurately representing diffuse irradiance under distant illumination. Ramamoorthi and Hanrahan [65] demonstrated that the Lambertian BRDF acts as a low-pass filter, well approximated by the first two SH orders. Similarly, Basri and Jacobs [4] showed that Lambertian reflectance functions lie close to a nine-dimensional linear subspace, confirming that low-frequency components dominate diffuse reflectance. Green [26] further notes that $l = 2$ (nine coefficients) is generally sufficient for diffuse lighting effects in real-time graphics.

Coefficient estimation is performed through stratified hemispherical sampling with $N = 5$ subdivisions per axis, producing $N^2 = 25$ sample directions. While no fixed standard exists for N , this setting provides a balanced trade-off between numerical stability and computational efficiency. As described in Monte Carlo analysis [61], the root-mean-square error of SH projection decreases as $O(1/\sqrt{N})$, implying diminishing returns for higher sample counts. In practice, this sampling density yielded stable results while maintaining real-time feasibility.

The network was implemented and trained using PyTorch [59], which represents both inputs and outputs as *tensors*—multi-dimensional arrays optimized for GPU-accelerated computation. Each vertex-normal pair (x, y, z, a, b, c) is encoded into a 32-dimensional CGA multivector, as detailed in Section 4.1.1, providing a rich and rotation-consistent representation of local geometry.

For each vertex, the network predicts nine SH coefficients per color channel [26], corresponding to the chosen order $L = 2$. These coefficients capture the per-vertex radiance transfer, encoding how each vertex responds to incident lighting under diffuse reflection. The total number of coefficients per vertex therefore follows:

$$N_{\text{coefficients}} = (l + 1)^2 \quad [26] \tag{4}$$

For example, using 3rd-order SH ($l = 3$) results in $(3 + 1)^2 = 16$ coefficients per vertex, while 2nd-order SH ($l = 2$) yields $(2 + 1)^2 = 9$ coefficients per vertex. These coefficients capture the angular variations of the radiance transfer function at each vertex.

We adopt the standard diffuse (achromatic) PRT formulation. Each vertex stores a 9-dimensional SH transfer vector for $L = 2$, $\mathbf{t}(x) \in \mathbb{R}^9$, which encodes geometry-dependent transport (visibility and clamped cosine). The environment illumination is represented per color channel as SH coefficients, yielding a 9×3 matrix $\mathbf{L} = [\mathbf{L}^R \mid \mathbf{L}^G \mid \mathbf{L}^B]$. The per-vertex RGB radiance is then computed by

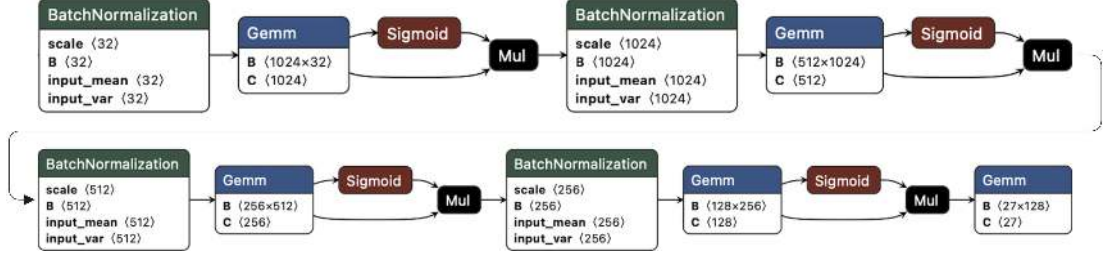


Figure 12: The neural network architecture, used in the neural radiance field of Neural-GASH.

reusing the same transfer vector across channels:

$$\mathbf{C}(x) = \text{albedo}(x) \odot \begin{bmatrix} \mathbf{t}(x)^\top \mathbf{L}^R \\ \mathbf{t}(x)^\top \mathbf{L}^G \\ \mathbf{t}(x)^\top \mathbf{L}^B \end{bmatrix} = \text{albedo}(x) \odot (\mathbf{t}(x)^\top \mathbf{L}).$$

Thus, the network predicts 9 transfer coefficients per vertex, while the lighting contributes 9×3 coefficients (9 per channel). This design is consistent with Sloan’s diffuse PRT and avoids color-dependent transport parameters per vertex.

4.3 Network architecture

The neural network employed in Neural-GASH is a MLP, chosen primarily because MLPs are the only neural network architecture fully supported by the Unity game engine, which constitutes a key target platform for our method. The architecture, illustrated in Figure 12, is composed of five fully connected layers with progressively decreasing dimensionality: 1024, 512, 256, 128, and finally 27 output neurons. Each hidden layer is followed by a SiLU (Sigmoid-weighted Linear Unit) activation function, selected due to its smooth non-linear characteristics and proven suitability for regression tasks. The network’s 27-dimensional output is subsequently reshaped to correspond to the number of vertices in the mesh, thereby enabling the model to generalize across arbitrary geometries. This design provides a balance between representational capacity and computational efficiency, allowing the network to be deployed in real-time rendering scenarios and on performance-constrained platforms.

4.3.1 Raw vertex-normal pairs vs CGA Multivectors

We opted to train our neural network using 32-dimensional multivector representations derived from CGA [34], as this approach yielded more stable and perceptually accurate results, particularly in challenging scenarios. Using a 32-dimensional multivector as input provides a richer and more expressive representation of local



(a) Neural-GASH: 32D CGA input

(b) Same architecture network: 6D euclidean input

Figure 13: Shading results of a cat mesh using (a) Neural-GASH, i.e., model is using 32 CGA multivectors as input, and (b) a network with identical architecture as Neural-GASH, using 6D euclidean coordinates as input (for position and normal of each vertex). In (a), the red light source illuminates only the edge of the cat’s head, consistent with its frontal position. In contrast, in (b), the entire back of the cat (including the tail) is unrealistically illuminated by the same frontal red light source. This represents a challenging edge case in large-scale 3D meshes, which is more accurately addressed by incorporating the 32D CGA representation in our neural network input.

geometry compared to a simple 6D vertex-normal pair. Multivectors can encode higher-order spatial relationships and directional context, allowing the network to better capture subtle variations in surface structure and lighting interactions, leading to improved prediction accuracy. This representation offers inherent advantages, including equivariance to rigid body transformations and the ability to capture nonlinear spatial relationships pertinent to radiative transport. As can be observed in Figure 13, networks trained on 6D euclidean-based inputs produced suboptimal results in cases involving large-scale or complex geometry, whereas the CGA-based formulation demonstrated superior visual performance and generalization in such conditions.

4.4 Network training and validation

Using the traditional PRT version that we implemented in Python, we processed a collection of 3D meshes with varying geometric complexity. Each mesh contained vertex positions and normals; if normals were missing, they were computed programmatically.

All experiments were conducted on a workstation equipped with:

- **Processor:** AMD Ryzen 9 5900X 12-core @ 3.70 GHz
- **Memory:** 32.00 GB Dual-Channel RAM @ 1330.6 MHz
- **Graphics Processing Unit (GPU):** NVIDIA RTX 3070 Ti (12GB VRAM)

This configuration provided sufficient computational resources to handle both the traditional PRT precomputation and the training of neural networks within reasonable timeframes. While not a high-end research cluster, this setup reflects commodity hardware accessible to developers, reinforcing the practicality of the proposed approach for real-world deployment.

For the training and validation dataset, we employed a collection of 3D meshes across general categories [38], trained for 200 epochs. In total, the dataset comprised 30 meshes for training and 8 meshes for validation. Although modest in count, this corpus was sufficient for a proof-of-concept for three reasons. First, the supervision in our setting is per-vertex: each mesh contributes thousands of training instances (one feature/target pair per vertex), so the effective sample size is the sum of all vertices across meshes rather than the mesh count alone. This yields on the order of 10^5 - 10^6 training examples in total, which is adequate for fitting a low-order SH transfer model. Second, the meshes span varied topology and curvature patterns (e.g., smooth/planar regions, sharp features, concavities), exposing the network to the geometric configurations that drive visibility and diffuse transfer. Third, our goal here is not to claim dataset-level state of the art, but to demonstrate feasibility: replacing PRT precomputation with learned predictors can recover transfer coefficients with low error and real-time feasibility. This diversity was selected to ensure exposure to a range of geometric and topological configurations. It should be noted that the networks were not fine-tuned on individual meshes. Instead, they were trained on the pooled dataset and subsequently evaluated on unseen meshes to assess generalization capability. This experimental design allowed us to test whether the proposed architecture can capture mesh-independent radiance transfer patterns, rather than memorizing geometry-specific details. Our results indicate that the learned models do indeed generalize across different mesh types, although some variation in accuracy was observed depending on surface complexity and mesh topology.

For each mesh, the ground-truth SH coefficients were computed using the traditional PRT method and exported to a text file for supervised training. Each file contained $27 \times N$ floating-point values, where N is the number of vertices. These 27 values were in fact the 9 coefficients reshaped across the 3 RGB channels. The i -th row corresponds to the i -th vertex and stores the 27 values (9 coefficients, replicated across the 3 color channels), denoted t_0 to t_{26} , representing the vertices' radiance response to incident lighting. These coefficients provide a compact, rotation-aware encoding of light transport and remain consistent across different meshes, provided a fixed set of lighting samples and SH bands.

Once the SH coefficients are predicted for each vertex, they are combined with the corresponding lighting coefficients derived from the HDRI environment map

to compute the diffuse illumination. This is achieved by evaluating a weighted sum across SH bands and RGB channels, which, while conceptually analogous to a dot product, explicitly accounts for the structured organization of SH coefficients and their alignment across color channels. These lighting coefficients are essential, as they determine the diffuse contribution of the scene’s illumination and thereby govern its overall realism.

To enable this computation, each HDRI environment map is preprocessed using a custom Python-based PRT implementation, which projects the incoming radiance of the environment onto the SH basis and stores the resulting coefficients in separate files. This preprocessing step ensures seamless runtime integration within Unity. The projection function computes the SH coefficients [65] by projecting the radiance function $L(\omega)$ onto the SH basis functions. Formally, given a set of N uniformly distributed sample directions $\{\omega_i\}_{i=1}^N$, the SH coefficient $\mathbf{c}_l \in \mathbb{R}^3$ associated with basis function $Y_l(\omega)$, for $l = 0, \dots, B^2 - 1$ where B denotes the number of SH bands, can be approximated as

$$\mathbf{c}_l \approx \frac{4\pi}{N} \sum_{i=1}^N L(\omega_i) Y_l(\omega_i), \quad (5)$$

where $L(\omega_i)$ is the RGB radiance sampled in direction ω_i . The normalization factor $\frac{4\pi}{N}$ ensures accurate integration over the unit sphere under uniform sampling. Once the training procedure is complete, a .pth file containing the network weights, is produced. In order to be able to use this later in Unity, we converted it to Open Neural Network Exchange (ONNX) [2, 68] type, which is the standard neural network type compatible with Unity Barracuda [78], which is Unity’s lightweight, cross-platform neural network inference library designed to run pre-trained models directly within the Unity engine.

4.5 Unity Integration

Figure 14 presents the rendering pipeline of the Neural-GASH system integrated into the Unity environment. The core functionality is encapsulated within the NGASH component, a C# script attached to an empty Unity GameObject. This component exposes two primary fields: the model asset, which contains the trained ONNX model, and the mesh field, which specifies the target mesh to be illuminated. The mesh is required to use an NGASH material, implemented as a lightweight HLSL vertex–fragment shader. The shader’s purpose is to render the raw per-vertex colors computed by the NGASH system.

Once the necessary inputs are configured, the runtime inference and processing stage is executed. The NGASH component iterates over all vertex–normal pairs

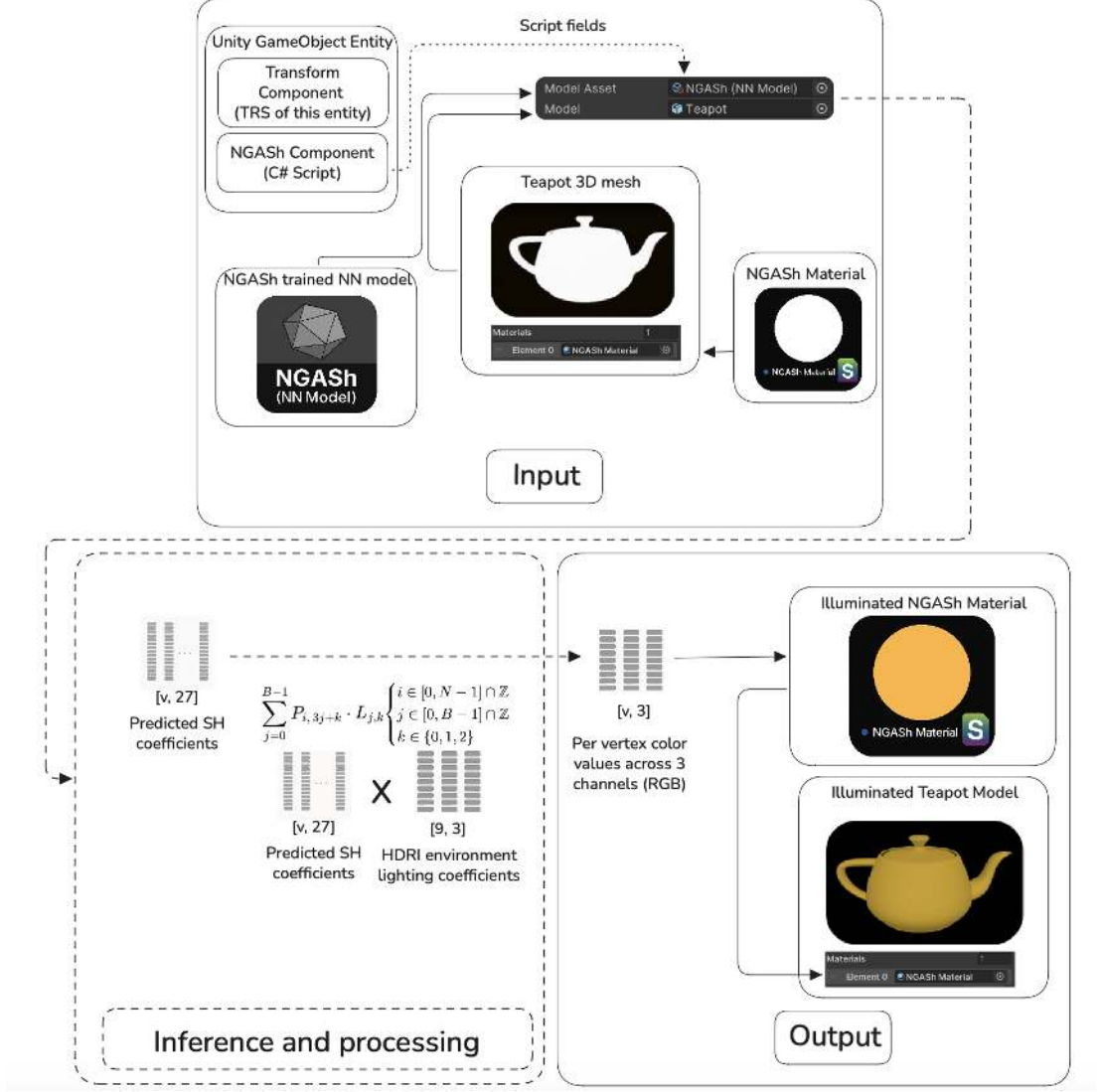


Figure 14: The main structure of the Neural-GASH pipeline in Unity for the teapot model.

of the mesh, encoding each pair as a 32-dimensional CGA multivector. These multivectors are passed to the neural network, which predicts 27 values per vertex (corresponding to 9 SH coefficients replicated across the 3 RGB channels). The predicted coefficients are then combined with the precomputed SH lighting coefficients of the HDRI environment map through per-channel dot-product operations, yielding the final diffuse color at each vertex. The resulting color values are stored in a float array and transmitted to the material via the fragment shader, producing the final rendered output in real time.

4.6 Results

The following table presents the precomputation times for various 3D meshes using both the traditional PRT algorithm and Neural-GASH. The performance

Mesh Name	Mesh Complexity	Traditional PRT Time	Neural-GASh Time (Unity)
Teapot	672 vertices	100.12 s	0.03 s
Pawn	4892 vertices	868.42 s	0.12 s
Humanoid	5340 vertices	892.28 s	0.20 s
Wolf	5559 vertices	962.27 s	0.15 s
Statue	49671 vertices	10957.75 s	1.31 s

Table 3: Precomputation times for various meshes using Traditional PRT and Neural-GASh. The bold result indicates real-time performance for meshes with fewer than 1000 vertices.

evaluation was conducted on a system with the following hardware specifications:

- **Processor:** AMD Ryzen 5 (Raven Ridge, 14nm Technology)
- **Memory:** 8.00 GB Dual-Channel RAM @ 266 MHz
- **Graphics Processing Unit (GPU):** NVIDIA GeForce GTX 1060 (6GB VRAM)

This benchmark provides insights into the computational efficiency of both methods under identical hardware conditions.

Since the traditional PRT algorithm was implemented in Python, we utilized the `time.time()` function as documented in the official Python Standard Library. This function accurately measures code execution time, making it suitable for benchmarking purposes.

For Neural-GASh, execution time was measured within the Unity game engine. Similar to traditional PRT, the recorded time corresponds to the duration required for the neural network to compute the vertex coefficients for the given geometry. To achieve high-precision timing, we employed the `Stopwatch` class from the .NET framework. As a high-resolution timer, `Stopwatch` is specifically designed for benchmarking and performance analysis, providing precise measurements of elapsed time and making it well-suited for profiling code execution.

The traditional PRT measured time refers to the computational time required to determine the vertex coefficients for a given model. This computation time is directly influenced by the complexity of the model, as illustrated in the accompanying table. Specifically, as the number of vertices increases, the time required for coefficient computation proportionally increases.

This version of PRT incorporates self-shadowing, with the most computationally expensive operation being the visibility check for each triangle in a given direction relative to the others. This operation is particularly costly because it

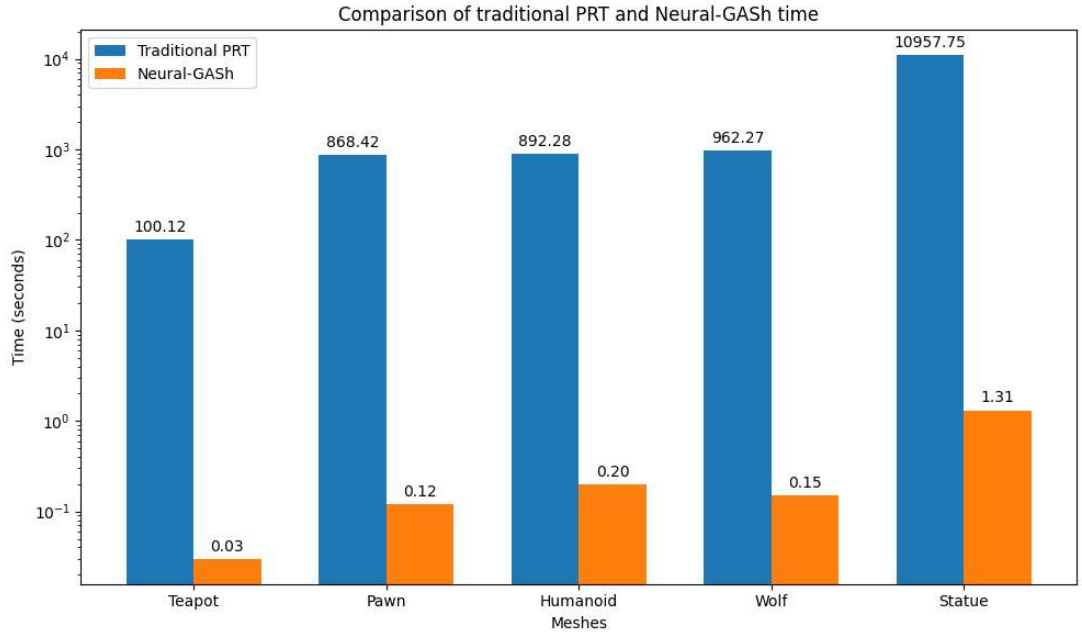


Figure 15: Traditional PRT and Neural-GASH time comparison.

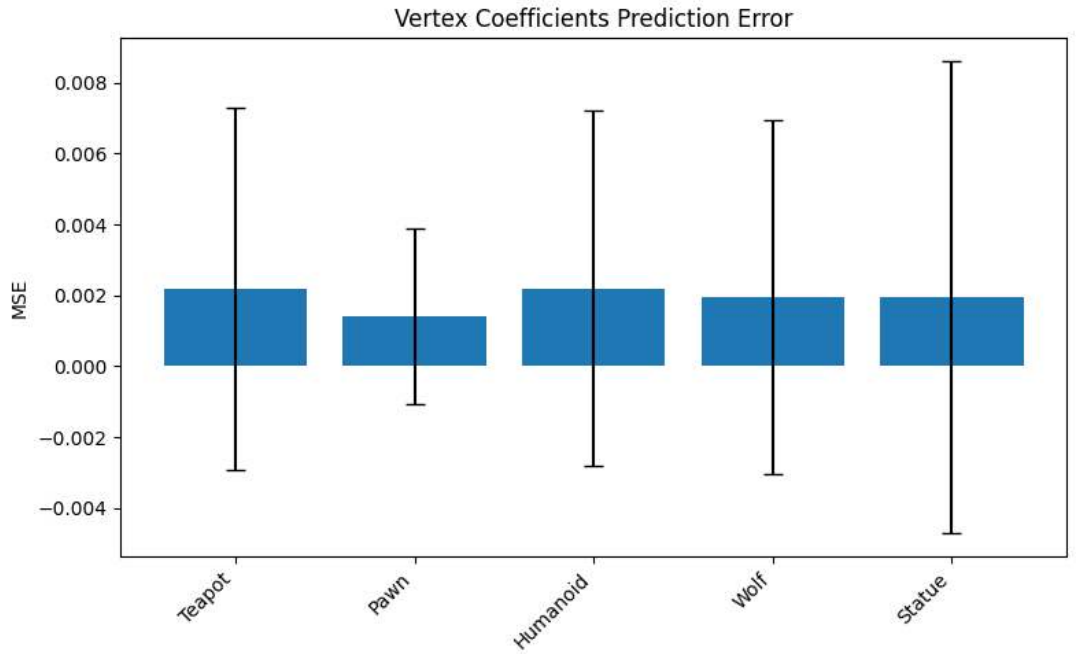


Figure 16: MSE and deviation of the predicted vertex coefficients for the five 3D meshes.

involves iterative loops whose execution time scales with the number of vertices in the model.

The Neural-GASh measured time, on the other hand, represents the time required for a neural network to infer the vertex coefficients of a model based on input data, such as vertex positions, normals, and other geometric attributes. Unlike the traditional PRT approach, which directly computes these coefficients, the Neural-GASh method leverages pre-trained deep learning models to accelerate this process. The results are visualized in Figure 15.

Figure 16 illustrates the mean squared error (MSE) in predicted vertex PRT coefficients for five different 3D models—Teapot, Pawn, Wolf, Humanoid and Statue—along with the standard deviation of errors across all vertices for each model. Each bar represents the average MSE for a given model, while the black error bars indicate the variability (± 1 standard deviation) of the vertex-wise coefficient prediction errors. Notably, the Statue model exhibits the largest standard deviation, indicating that the prediction of PRT coefficients for this model poses a greater challenge for the neural network, likely due to more complex surface features or lighting interactions that lead to higher and more variable errors. In contrast, the Pawn model shows lower average errors and less variability, suggesting more consistent and accurate coefficient predictions. These results demonstrate that the model’s performance is more sensitive to the complexity of the radiance transfer characteristics rather than the sheer number of vertices.

To further demonstrate the robustness and general applicability of the proposed framework, we evaluated Neural-GASh on textured 3D assets rendered in Unity. Since the method predicts achromatic diffuse SH coefficients, illumination and surface color are treated independently: the predicted radiance field represents purely geometric light transport, while the final appearance is obtained by multiplying the irradiance with the per-vertex albedo sampled from the texture. This decoupling of lighting and material ensures that the framework generalizes seamlessly to textured models without retraining or modification. The results in Table 5 illustrate that Neural-GASh reproduces the shading and global illumination effects of traditional PRT even on complex textured surfaces, confirming that the neural coefficients maintain physical consistency under realistic material conditions.

Mesh	Traditional PRT	Neural-GASh
Teapot		
Pawn		
Humanoid		
Wolf		
Statue		

Table 4: Results comparison of five different meshes under traditional PRT and Neural-GASh.

Mesh	Traditional PRT	Neural-GASh
Barrel		
Statue		
Brain		

Table 5: Visual comparison of textured meshes rendered with traditional PRT (left) and Neural-GASh (right). The results show that Neural-GASh reproduces the same global illumination, and diffuse shading characteristics as classical PRT, while operating in real time. The near-identical appearance between the two columns demonstrates that the neural model preserves lighting fidelity even on textured, complex geometries.

4.7 Gaussian Splatting & Neural-GASh

The most advanced neural rendering technique currently available is 3D Gaussian Splatting (3DGS). This method produces highly realistic results by explicitly representing scene geometry through a large set of 3D Gaussians [43]. Despite its impressive quality, 3DGS suffers from a critical limitation: it is fundamentally restricted to static scenes. Consequently, it does not support deformations, animations, or dynamic lighting changes, which limits its applicability in interactive or real-time scenarios.

An example of this limitation is illustrated in Figure 17, which depicts the well-known garden scene from the Mip-NeRF 360 dataset [3] reconstructed using the 3DGS method. While the reconstruction achieves high fidelity, the representation is static and does not permit dynamic scene manipulation.



Figure 17: The garden scene from the Mip-NeRF 360 project [3]. On the left is the 3D Gaussian Splatting reconstruction, generated from a dataset of 185 images. On the right is the reconstruction using a 3D point cloud, generated with the SuGaR method [29], with texture extracted from the Gaussians.

In contrast, the Neural-GASH framework operates directly on explicit 3D geometry in the form of vertices and normal vectors. To enable its application to the same garden scene, we reconstructed the dataset using the SuGaR technique [29], which produces a 3D point cloud representation. This representation provides the vertex and normal data required by Neural-GASH, thereby bridging the gap between static Gaussian splat representations and geometry-driven rendering.

By leveraging this geometric input, Neural-GASH overcomes one of the key limitations of 3DGS: its restriction to static environments. Our system enables support for dynamic geometry and real-time lighting control through the integration of CGA, which facilitates efficient rotation of SH light coefficients. This capability is demonstrated in Figure 18 and Figure 19, where scene lighting is interactively altered by rotating the radiance map in the skybox. Importantly, these updates are applied without restarting the rendering process, with each lighting update completed in under one second on mid-range evaluation hardware.

A direct comparison with 3DGS further highlights the practical advantages of Neural-GASH. For the garden scene, Neural-GASH required approximately 4 seconds to complete the initial lighting computation, while subsequent lighting updates in an already running scene could be performed in just 0.3–0.6 seconds. In contrast, 3DGS relies on training from image data to reconstruct the scene representation, a process that typically ranges from several minutes to multiple hours depending on the dataset size and desired quality [43]. Moreover, even minor modifications to the scene (e.g., geometry changes or lighting updates) necessitate retraining, incurring substantial additional cost. This distinction underscores Neural-GASH’s suitability for interactive and dynamic applications, where responsiveness and adaptability are critical, whereas 3DGS is more aligned with offline reconstruction tasks or static scene rendering.

To comply with tensor size limitations imposed by the Unity Barracuda infer-



Figure 18: Neural-GASH results for the Mip-NeRF 360 scene [3], reconstructed with the SuGaR method [29], with each instance involving the rotation of light and light coefficients using CGA logic.

ence engine, the original reconstructed geometry was simplified to approximately 80,000 vertices. This mesh was subsequently divided into two sub-meshes of approximately 40,000 vertices each, processed in parallel by two instances of Neural-GASH. The reported timing reflects the average duration required to update the SH lighting coefficients across both sub-meshes. For comparison, computing the same per-vertex coefficients using the conventional PRT pipeline required 16,923.55 seconds, underscoring the dramatic performance advantage of the proposed system.



Figure 19: Lighting results on the reconstructed scene from Mip-NeRF 360 [3], reconstructed with the SuGaR method [29], utilizing different radiance maps. The time in seconds needed to switch from one lightmap to another, was between 0.3-0.6 seconds. The lighting changes are performed during program runtime.

4.8 Neural-GASh vs State-of-the-art

Table 6 presents a comparative evaluation of state-of-the-art PRT-based lighting techniques alongside the proposed Neural-GASh framework. As a primary baseline, we include the traditional PRT method [75], against which more recent neural approaches are benchmarked. These include NeuralPRT [64], DeepPRT [48], and 3D Gaussian Splatting [43], the latter representing one of the most widely adopted real-time rendering methods in contemporary computer graphics. The comparison addresses several critical aspects: the nature of input and output data, support for animation and deformation, the ability to update textures dynamically at runtime, support for real-time lighting rotation, and compatibility with modern game engines for integration into interactive applications.

	Traditional PRT [75]	NeuralPRT [64]	DeepPRT [48]	3D Gaussian Splatting [43]	Neural- GASH
Input	3D Mesh	Pre- Rendered image of mesh	3D Mesh	COLMAP Images	3D Mesh
Output	Colors (float array)	2D Images (Per Pixel Radiance)	3D Mesh	Gaussians	Colors (float array)
Animation Support	No	No	Yes	No	Yes
Runtime Texture Update Support	Under Conditions	No	Not Clear	No	Yes
Lighting Rotation Support	Under Conditions	No	No	No	Yes
Game- Engine Compati- ble	No	No	No	No	Yes

Table 6: Comparison of PRT methods and their capabilities with Neural-GASH.

In terms of input representation, the methods exhibit clear differences. Traditional PRT operates directly on the fundamental 3D mesh data, specifically vertex positions and normals, and thus provides a straightforward pipeline. NeuralPRT, on the other hand, relies on pre-rendered images of the mesh, applying PRT-based lighting directly in image space. DeepPRT accepts raw 3D mesh data but extracts a harmonic map from the mesh, which is subsequently encoded as a texture to drive its predictions. By contrast, 3D Gaussian Splatting reconstructs geometry from COLMAP [72, 71] image datasets, representing the scene as a collection of Gaussian distributions. Neural-GASH remains conceptually closest to the original PRT formulation, operating directly on vertices and normals without requiring intermediate image-based or texture-based preprocessing, thereby maintaining a simple and efficient pipeline.

The nature of the output also varies substantially. Traditional PRT produces a color array associated with the mesh, encoding per-vertex lighting information. NeuralPRT, in contrast, generates 2D images that represent the mesh under PRT-based illumination. DeepPRT outputs coefficient images, which are subsequently mapped onto the mesh to reconstruct the final appearance. 3D Gaussian Splat-

ting generates a set of 3D Gaussians that collectively encode the reconstructed scene along with its lighting properties. Neural-GASH follows the PRT paradigm most directly, producing a per-vertex color array that encapsulates lighting predictions generated by the model itself.

Support for animation and deformation is another key differentiating factor. Among the compared methods, only DeepPRT explicitly addresses dynamic geometry by predicting updated coefficients under mesh deformation. In contrast, traditional PRT assumes static geometry, NeuralPRT operates entirely on image data, and Gaussian Splatting reconstructs scenes from 2D images into static 3D representations, none of which permit geometric flexibility. Neural-GASH, by design, extends the classical PRT framework to dynamic and deformable scenes by predicting per-vertex coefficients in real time, thereby overcoming this restriction.

The comparison further highlights differences in support for dynamic texture updates. Traditional PRT may provide partial support depending on the implementation and shader configuration; however, such functionality is neither guaranteed nor standardized. NeuralPRT, being restricted to pre-rendered images, does not support runtime texture modifications. DeepPRT theoretically could support this feature but does not explicitly demonstrate it. Gaussian Splatting, as a scene reconstruction method, does not accommodate dynamic texture swapping since any modification would necessitate a complete reprocessing of the input images. Neural-GASH, in contrast, incorporates a lightweight HLSL vertex-fragment shader that supports runtime texture mapping. Consequently, any changes to the assigned texture are immediately reflected in the rendered output, enabling seamless and fully interactive texture swapping.

Lighting rotation represents another critical criterion for global illumination methods, particularly for simulating natural light sources such as the sun, which change orientation over time. None of the state-of-the-art approaches reviewed provide native support for dynamic lighting rotation at runtime. Building on prior work in CGA-based lighting transformations [56], Neural-GASH integrates CGA to enable efficient rotation of SH lighting coefficients. This feature makes Neural-GASH the only method in the comparison to offer real-time lighting rotation within a PRT framework.

Finally, Neural-GASH introduces a practical contribution through direct compatibility with modern game engines. Specifically, it is the first PRT-based lighting framework fully integrated into Unity, facilitating deployment across a wide range of interactive applications and platforms. This compatibility bridges the gap between advanced global illumination methods and real-time rendering requirements, extending PRT-based lighting into practical, interactive environments such as video games, simulations, and virtual reality.

4.9 Limitations

During evaluation, several limitations of the MLP-based Neural-GASh framework were identified, primarily related to scalability and geometric expressiveness. The current implementation operates on a single 3D mesh at a time; supporting multiple geometries requires instantiating separate modules for each, which introduces additional overhead. While multiple meshes can be merged into a single geometry prior to processing, this approach is impractical for animated or deformable assets.

Residual inaccuracies were also observed in models with large spatial extents or complex concavities. Although the use of CGA multivectors as input features significantly improved predictive stability compared to conventional vector representations, the MLP architecture lacks explicit awareness of mesh connectivity. As a result, it struggles to infer self-occlusion and ambient shadowing in regions with deep cavities or intricate topology.

A further limitation arises from the architectural constraints of the Unity Barracuda inference engine, which currently supports only static MLP networks and imposes strict tensor size limits. This restricts inference to meshes containing roughly 100,000 vertices or fewer, preventing deployment on larger assets. The standalone Python implementation of Neural-GASh does not suffer from these restrictions, confirming that the limitation originates from Barracuda rather than the model itself.

Looking forward, several potential solutions could address these issues. One direction involves the adoption of alternative inference backends within Unity, such as the upcoming Inference Engine [77], which may include future support for GNNs. Another, more flexible approach is to integrate native inference libraries (e.g., the PyTorch C++ API) as Unity plugins, enabling direct deployment of graph-based architectures. Techniques such as model quantization and pruning may further reduce memory overhead while maintaining accuracy, improving compatibility with Barracuda’s constraints.

In summary, while the MLP-based Neural-GASh effectively demonstrates the feasibility of using CGA encodings for real-time radiance transfer prediction, its inability to model topological relationships, handle self-occlusion, and support large meshes within Unity highlights the need for a more expressive framework. These limitations directly motivated the development of the GNN-based extension presented in Chapter 5, which integrates mesh connectivity into the learning process. By propagating illumination across neighboring vertices, this topology-aware formulation overcomes the cavity and occlusion limitations of the MLP model and removes dependence on Barracuda’s tensor constraints, thereby ex-

tending the applicability of Neural-GASh to complex and large-scale 3D environments.

5 Neural-GASh (GNN-Based)

Although Neural-GASh produced promising results, successfully illuminating objects in accordance with the principles of the PRT algorithm, it exhibited several limitations, as previously discussed. The most critical of these was the absence of self-shadowing (or self-occlusion). While objects were correctly lit based on the HDRI environment map, they lacked shadows cast by their own geometry. For example, in the case of a humanoid model, if the arm was raised close to the chest, no corresponding shadow would be rendered, as illustrated in Figure 20. Similar deficiencies were observed in other models containing complex cavities or concave structures.

In the MLP-based formulation of Neural-GASh, the network output was structured as 27 values per vertex, corresponding to the 9 SH basis functions replicated across the three RGB channels. While this formulation was consistent with a channel-wise representation of radiance transfer, it introduced redundancy, since in our achromatic diffuse setting the SH basis is identical across channels. To streamline the representation, the final GNN-based Neural-GASh model was designed to predict only the 9 scalar SH coefficients per vertex. This revised formulation reduces the dimensionality of the learning task, removes redundancy, and better reflects the physical structure of PRT, where light transport is channel-independent and material color is applied as a separate modulation step.

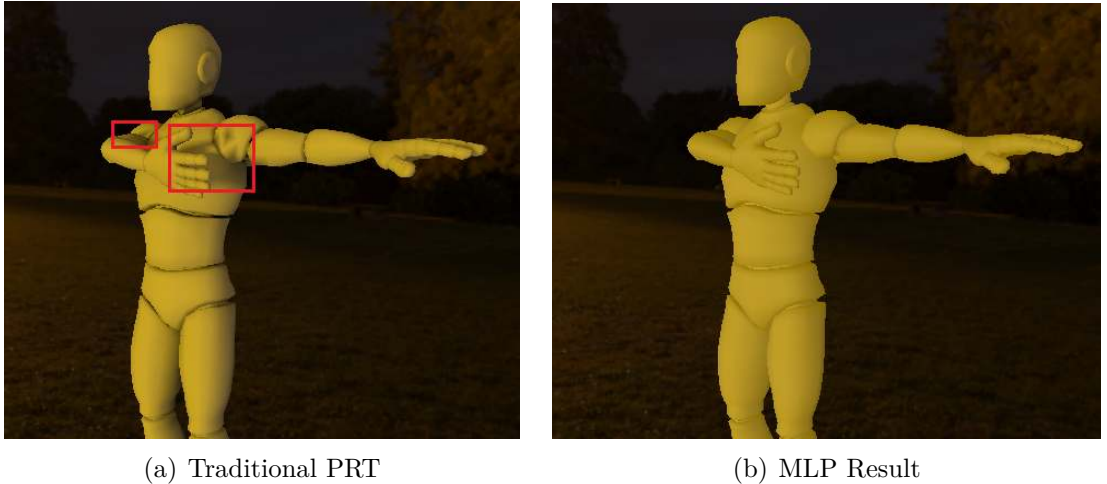


Figure 20: Comparison between (a) the traditional PRT baseline and (b) the MLP-based Neural-GASh result. The self-shadow beneath the arm, highlighted in red in (a), is missing in (b), revealing the MLP model’s limitation in capturing occlusion-dependent light transport.

Addressing this issue was essential, as self-shadowing effects are naturally captured in the results of traditional PRT and are vital for achieving visual realism. The overarching objective of this work was to design a neural radiance field

that replicates the behavior of traditional PRT while eliminating its computationally expensive precomputation step, thereby enabling efficient execution for both static and dynamic geometry. To resolve the self-shadowing limitation, we systematically investigated four alternative strategies to enhance Neural-GASH. These included: (i) retaining the original MLP architecture while expanding the training dataset to improve generalization, (ii) employing a single GNN-based architecture to leverage mesh connectivity, (iii) adopting a teacher–student paradigm in which a larger GNN supervises a lighter student network, and (iv) introducing a dual GNN-based architecture designed to jointly predict visibility and radiance transfer coefficients.

5.1 MLP-based with richer dataset

As discussed earlier, the initial objective was to retain the MLP architecture of Neural-GASH, as deployment within the Unity game engine was a primary requirement. Unity’s Barracuda interface, however, imposes restrictions that prevent the use of architectures beyond MLPs. Consequently, our first strategy was to expand and diversify the Neural-GASH training dataset to include more complex geometries, particularly those exhibiting self-shadowing and cavities. To achieve this, we incorporated rigged humanoid models, saved in multiple poses using automated scripts in Blender, alongside randomized geometries with cavities generated through custom Python scripts. In addition, we leveraged the Objaverse dataset [17, 16], ultimately expanding the training corpus to approximately 1,500 distinct models.

Despite this significantly enriched dataset, training with the same MLP architecture failed to resolve the absence of self-shadowing. Alternative adjustments—such as increasing network depth, modifying hyperparameters, and experimenting with activation functions—yielded no improvement. These results indicated that the MLP architecture had reached the limits of its representational capacity. It became evident that addressing self-shadowing and complex light transport phenomena required a transition to more advanced deep learning architectures. For this reason, the focus shifted from conventional MLPs toward Graph Neural Networks (GNNs), which are inherently better suited for processing structured geometric data.

5.2 GNN-based

To achieve higher accuracy and overcome the limitations of the MLP-based Neural-GASH, we transitioned to a GNN architecture. GNNs are inherently better suited for processing structured geometric data, as they can exploit mesh

connectivity to model spatial coherence, self-occlusion, and light transport interactions between neighboring vertices.

Before converging on the final design, we systematically evaluated four architectural variants to identify the optimal configuration for real-time deployment:

- **GNN (32D)**: A baseline GNN trained solely on CGA-derived input features (vertex-normal multivectors). While this model improved smoothness and inter-vertex consistency compared to the MLP baseline, it failed to predict accurate self-shadowing, as shown in Figure 13. Shadows often appeared misaligned or inconsistent in concave regions.
- **GNN (32D + 25D Visibility)**: This variant augmented the input features with 25-dimensional binary visibility vectors per vertex, computed from preprocessed ray-sampling directions. The model achieved higher shadowing accuracy but required costly precomputation of visibility terms, limiting its practicality for dynamic or deformable meshes.
- **Teacher-Student Architecture**: A distillation approach in which a visibility-aware teacher GNN supervised a lightweight student network trained on CGA features alone. Despite reduced inference cost, the student failed to recover visibility cues with sufficient precision, resulting in shadow artifacts similar to the 32D-only configuration.
- **Two-Stage GNN**: The adopted architecture decouples geometric encoding and illumination prediction into two sequential stages. The first stage infers per-vertex visibility features directly from mesh connectivity, while the second stage predicts SH transfer coefficients conditioned on these inferred features. This structure proved most effective, combining accuracy and runtime flexibility without explicit visibility precomputation.

5.2.1 GNN (32D)

The 32-dimensional CGA multivectors remain central to all configurations, encoding vertex positions and normals in a unified algebraic form. However, this configuration did not yield satisfactory results: while shadows began to emerge in the rendered output, their placement was often inconsistent or incorrect. This observation nevertheless provided a strong indication that the GNN architecture was well-suited to the problem, as it demonstrated the capacity to capture shadowing effects absent from the MLP-based approach.

We designed this GNN specifically to predict SH radiance coefficients from geometric features of 3D meshes, in the same way as with the MLP. The architecture is based on a Graph Attention Network v2 (GATv2) backbone, enhanced

with residual connections, normalization layers, and dropout regularization to ensure both stability and expressive capacity. This design was chosen to overcome the limitations of the earlier MLP-based model, particularly its inability to accurately capture self-shadowing effects and complex ambient occlusion (AO) in geometrically intricate meshes. When employing the GNN with only the 32-dimensional CGA multivectors as input features, the model was able to reproduce shadow regions on the objects; however, their placement was often inaccurate and spatially misaligned, as illustrated in Figure 21.

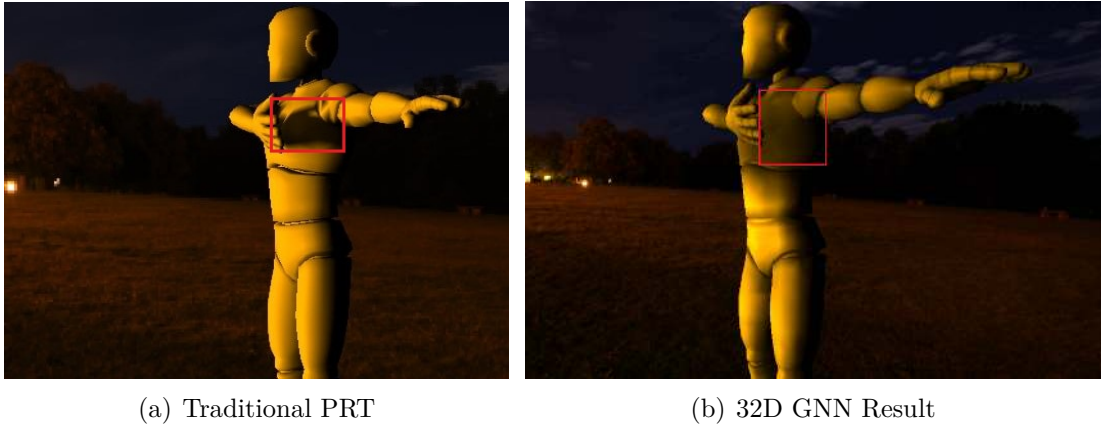


Figure 21: Comparison between (a) the traditional PRT baseline and (b) the GNN-based Neural-GASH output using only CGA-derived input features. The self-shadow region, highlighted in red, appears misaligned in (b), indicating that CGA features alone are insufficient for accurate shadow placement without incorporating explicit topological information.

5.2.2 GNN (32D + 25D Visibility)

To improve visual quality, it became clear that the network required richer information about the mesh topology. Revisiting the traditional PRT algorithm, we noted that for each vertex–normal pair, the method computes a set of binary visibility samples, indicating whether the vertex is visible from a given direction. In our specific configuration, we compute 25 visibility values per vertex, corresponding to 25 sampled directions per band. Inspired by this, we augmented the GNN input with these visibility features, effectively increasing the per-vertex feature vector from 32 to 57 dimensions. To facilitate this, we re-ran the traditional PRT algorithm across the entire dataset in order to extract and store the visibility values for each mesh, which were then incorporated into the training procedure.

At the input stage, each vertex of the mesh is represented by a 57-dimensional feature vector. These features encapsulate local geometric descriptors, including positional and normal information expressed in CGA form, which provides a richer representation of geometric relationships compared to traditional Euclidean

coordinates. The network begins with a fully connected linear projection layer, mapping these 57 input features to a latent space of 96 dimensions. This step serves both to normalize the feature scale and to prepare the data for subsequent message-passing operations within the GNN layers.

The core of the architecture consists of five stacked GATv2 convolutional layers, each configured with two attention heads. In contrast to traditional GCNs, which aggregate neighborhood information uniformly, the GATv2 layers employ an attention mechanism that learns to assign varying importance to different neighboring vertices. This is particularly advantageous in the context of mesh-based lighting, as the contribution of neighboring geometry to radiance transfer can vary significantly depending on spatial configuration and occlusion. The use of GATv2 therefore allows the network to learn fine-grained dependencies and interactions among vertices.

To mitigate common challenges associated with deep GNNs—most notably the issue of over-smoothing—the architecture integrates several stabilization mechanisms. As shown in Figure 25, each convolutional layer is followed by a Layer Normalization operation, which standardizes activations and ensures stable gradients during training. Additionally, a residual connection is employed wherever feasible, allowing the model to propagate information across layers while preserving identity mappings. This design facilitates deeper architectures without significant performance degradation. To further improve generalization, each layer includes a dropout module with a rate of 0.2, introducing stochastic regularization and reducing overfitting risks.

Following the sequence of GATv2 layers, the final latent representations of the vertices are projected to the target output dimension through a fully connected linear layer. The network predicts 9 SH coefficients per vertex, corresponding to the basis functions required for low-order SH lighting. These coefficients encode the local radiance transfer properties of the geometry and are subsequently combined with the environment lighting coefficients to compute the per-vertex illumination. The result for the same model as shown in Figure 21 can now be seen in Figure 22 below.

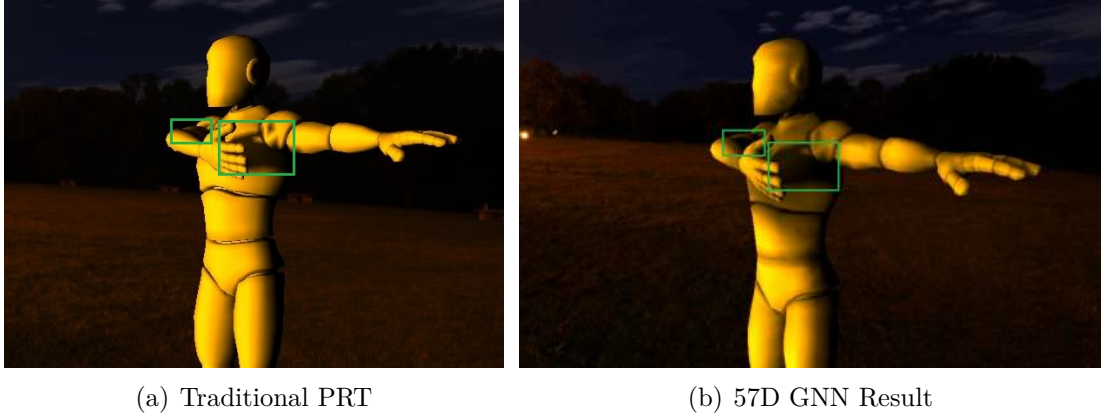


Figure 22: Comparison between (a) the traditional PRT baseline and (b) the GNN-based Neural-GASH output augmented with visibility samples. The self-shadow region, highlighted in green, is more accurately reproduced in (b), demonstrating that incorporating visibility information enhances topological awareness and corrects the misalignment observed in Figure 21.

The primary limitation of the proposed architecture lies in its reliance on visibility samples, which are not available during inference. This constitutes a significant drawback, as these samples cannot be computed at runtime without reintroducing the costly precomputation step that forms the principal bottleneck of the traditional PRT algorithm. Consequently, it became imperative to devise an alternative solution that would eliminate this dependency while preserving the benefits gained from incorporating visibility information into the network.

5.2.3 Teacher–Student Architecture

To address the aforementioned limitation, we explored a teacher–student paradigm, using the already trained GNN as the teacher. The idea was to distill its knowledge into a lighter student network, trained only on the basic CGA multivectors. In principle, this approach would enable deployment in real-time settings by approximating the teacher’s predictions without requiring explicit visibility features.

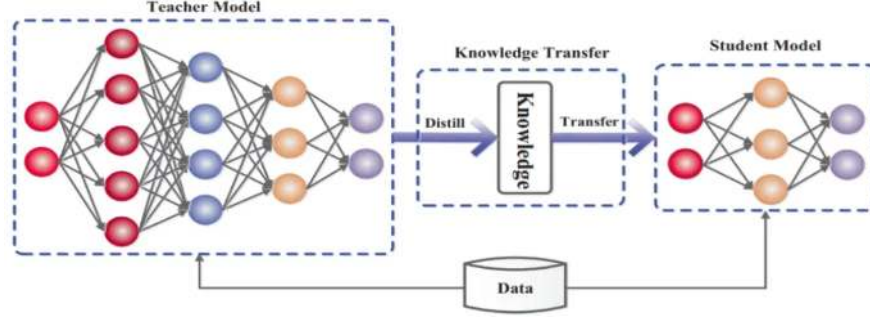


Figure 23: Illustration of the Teacher–Student architecture applied to Neural-GASH. The Teacher model (left), a larger GNN trained with full input including visibility samples, supervises a Student model (right) restricted to the CGA multivector input. Adapted from [69].

We investigated two student variants: an MLP (compatible with Unity’s Baracuda interface) and a lightweight GNN. In practice, both student variants failed to replicate the teacher. Shadows were absent or misaligned, global illumination was inconsistent, and in many cases the resulting mesh colors were incorrect. Because the outputs were visually implausible, we did not pursue quantitative evaluation further; error metrics would not have been meaningful in the absence of even basic shadow fidelity.

Why distillation failed. Although attractive in principle [33, 25], the teacher–student approach proved ineffective in this context. Four technical reasons explain this outcome:

1. **Information mismatch:** The teacher consumed 57 input features (32 CGA + 25 visibility), whereas the students had access only to 32. Visibility is not a deterministic function of CGA inputs, so the student was asked to infer information it could not possibly reconstruct.
2. **Inductive-bias gap:** The teacher was a GNN leveraging mesh connectivity, while the MLP student operated independently per vertex. Even when replaced with a lightweight GNN, the absence of explicit visibility still left the student blind to occlusion cues.
3. **Loss mismatch:** Distillation targeted coefficient regression with uniform MSE, ignoring the angular role of SH terms. High-frequency coefficients (which encode directional occlusion detail) were systematically underfit, consistent with the spectral bias of neural networks [63].
4. **Capacity/optimization limits:** An MLP small enough for deployment lacked the capacity to absorb the teacher’s function class, while regression-

style distillation is sensitive to hyperparameters and provided little robustness.

Why the MLP student in particular failed. The MLP treats vertices independently and has no mechanism to aggregate neighborhood context. Since the teacher’s predictions depend on local topology (e.g., cavities and concavities that drive self-shadowing), the MLP could not approximate the occlusion-dependent mapping. This explains why shadows were either absent or completely misplaced in practice.

In summary, the teacher–student architecture was abandoned after these failures. The outcome confirmed that accurate shadowing requires either explicit visibility features or graph-based inductive biases, which are absent from lightweight MLPs.

5.2.4 Two-Stage GNN

After the unsuccessful attempts with the teacher–student paradigm, it became necessary to explore an alternative strategy for addressing the fundamental limitation posed by the unavailability of visibility samples during inference. The central idea was to shift the computationally expensive task of generating visibility samples from the traditional PRT pipeline to a learned prediction model. Since the CGA multivectors were already available for each vertex–normal pair, we hypothesized that a dedicated GNN could be trained to map these multivectors directly to approximate visibility values. In this way, the prediction of visibility information would be performed efficiently at inference time, at a fraction of the computational cost required by the explicit sampling process. To realize this approach, we designed and trained a second GNN whose input consisted of the 32-dimensional CGA multivectors, while its output provided the 25 predicted visibility values required for the radiance transfer computation.

The GNN we created for this purpose is a graph-based model that predicts per-vertex visibility descriptors directly from mesh features and connectivity. Incoming node features (CGA multivectors) are first projected into a compact latent space by a fully connected layer, after which the network applies a deep stack of attention-based message-passing blocks built on Transformer-style graph convolutions. In each block, multi-head self-attention computes content-adaptive weights over a vertex neighborhood so that information from geometrically salient neighbors (e.g., along concavities or occluding structures) is emphasized. The block then applies layer normalization and a rectified nonlinearity, followed by light dropout for regularization. To sustain depth without over-smoothing, each block includes an additive residual connection that preserves the identity mapping when

dimensions match, improving gradient flow and stability.

After the sequence of message-passing layers, as shown in Figure 26 a shallow prediction head (dropout followed by a linear projection) maps the refined per-vertex embeddings to the desired visibility outputs (e.g., directional visibility scores or logits). In our configuration, the model uses a moderate latent width, a depth of three attention blocks, and a small number of attention heads, striking a balance between expressiveness and efficiency suitable for real-time or near-real-time pipelines. By operating on the mesh graph and learning attention over neighbors, the network captures topological cues essential for estimating self-occlusion and serves as an effective provider of visibility information for downstream radiance-coefficient prediction.

Finally, Figure 24 presents the final version of the updated GNN-based Neural-GASH framework, which extends its formulation to align with the principles of neural radiance fields.

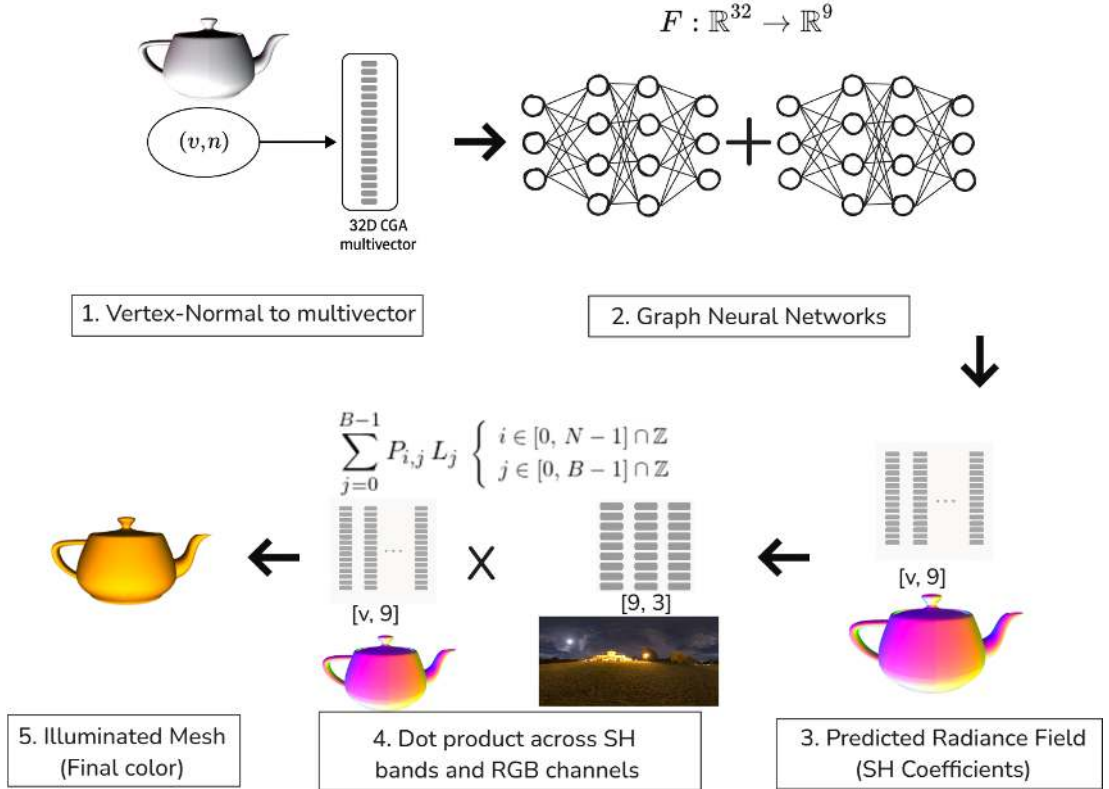


Figure 24: Conceptual illustration of the neural radiance field formulation adopted in the GNN-based Neural-GASH.

5.2.5 Design trade-offs: single vs. two-stage GNN

What we tried and why we moved to two stages. We first trained a *single, end-to-end GNN* that maps the 32-D CGA multivector per vertex directly to the nine SH coefficients. While this model converged and reproduced

low-frequency illumination well, it consistently underperformed on self-occlusion and cast-shadow structure. In hindsight, this is expected: visibility is a high-frequency, highly non-linear signal that is only weakly implied by local CGA features. Absent an explicit occlusion cue, the end-to-end model tended to smooth or misplace shadows. Introducing *explicit visibility* as an input (32+25 features) markedly improved shadow fidelity when the visibility values were ground truth.

Two-stage formulation and its consequences. To avoid runtime ray sampling, we replaced ground-truth visibility with a learned *visibility-prediction GNN* (stage 1), whose outputs feed the *coefficient-prediction GNN* (stage 2). This removes the precomputation bottleneck at inference but introduces two costs: (i) *sequential latency*: two forward passes must execute; and (ii) *error propagation*: mistakes in predicted visibility can bias the downstream SH coefficients (cascading error). In practice, we mitigate latency by batching both graphs on the GPU and reusing the same mesh connectivity; nevertheless, the total time is the sum of both passes. We mitigate error propagation by training with visibility-aware losses and regularizers; however, when the first stage deviates on hard concavities, the second inherits this bias and shadow boundaries may blur or shift.

Why the single end-to-end GNN was not retained. A natural question is whether a single GNN that predicts SH coefficients directly (with no explicit visibility) would be simpler and more robust. In our experiments, this architecture (with 32-D inputs) produced shadows but with inferior boundary placement and contrast relative to the two-stage approach. The gap stems from *information mismatch*: the teacher/coefficient target implicitly depends on visibility, which is not a deterministic function of CGA features alone. Providing an explicit visibility estimate—even a learned one—serves as a strong inductive cue for occlusion, improving spatial localization of shadows.

Summary. Our results indicate: (a) a single end-to-end GNN with only 32-D CGA inputs is *simpler* but underfits occlusion; (b) a two-stage pipeline *improves shadow fidelity* but adds sequential latency and can propagate visibility errors; and (c) a fused, multi-task GNN is a promising compromise that keeps one-pass latency while retaining an explicit occlusion pathway during training. In the present work we report the two-stage variant for clarity and modularity, and leave the fused multi-task alternative as a practical optimization for deployment.

5.3 Networks training and validation

Both networks operate on mesh-derived graphs, where each vertex is represented by features based on CGA encodings of vertex–normal pairs. Training each network required a carefully designed preprocessing pipeline for graph construction, feature normalization, and data splitting, as well as distinct loss functions and optimization strategies suited to their respective prediction targets. The following sections describe in detail the training procedures of the SH coefficients-predicting GNN and the Visibility GNN.

5.3.1 Coefficients GNN

The basic workflow of the coefficients GNN is illustrated in Figure 25, where the processing of vertex features follows a structured sequence of operations designed to progressively refine the per-vertex representations before predicting SH coefficients. Each vertex is initially represented by a 57-dimensional feature vector, which is first projected into a 96-dimensional latent space through a linear transformation followed by a ReLU activation. Subsequently, the network applies a series of five graph attention blocks, each consisting of a GATv2 convolution, layer normalization, ReLU activation, dropout, and a residual skip connection. It is within the GATv2 convolution layers that the mesh connectivity, is explicitly used: the edge information specifies which vertices exchange information during message passing, thereby enabling each vertex to aggregate features from its immediate neighbors. In contrast, the subsequent normalization, non-linearity, dropout, and residual operations act independently on each vertex without reference to the graph structure. After passing through these blocks, the refined vertex features are mapped through a final linear layer to produce nine SH coefficients per vertex. This workflow ensures that local geometric relationships encoded in the mesh topology directly influence the predicted coefficients, while non-graph operations stabilize and regularize the learning process.

We train the graph network to regress per-vertex SH radiance coefficients from mesh-based features and visibility. For each mesh in the corpus, vertex positions and normals are converted into CGA multivectors and normalized to unit length. Moreover, the 25 visibility features per vertex are included, yielding a 57 sized input. Positions are recentred (zero mean) and isotropically scaled to a bounded range to stabilize neighborhood construction. A radius-graph is then built on the normalized positions so that edges connect spatially proximate vertices; the radius is chosen adaptively from the average inter-vertex spacing, ensuring comparable neighborhood sizes across meshes. The resulting graphs (features, edges, targets) are cached to disk to avoid repeated preprocessing. Targets are the low-

order SH coefficients per vertex computed offline by the reference pipeline; inputs are the per-vertex CGA features (and, in variants, any auxiliary visibility descriptors where applicable). Finally, the included ground truth SH coefficients in the dataset are 9 per vertex as previously discussed.

The dataset is randomly partitioned into training and validation splits (80/20). Mini-batches consist of one graph per step (full-mesh batches), which preserves mesh-level structure and avoids fragmentation of neighborhoods. The model is optimized with a modern adaptive optimizer using weight decay. The loss is the mean-squared error between predicted and target SH coefficients, computed per vertex and averaged over the graph. To improve numerical stability and throughput, mixed-precision training is employed with automatic loss scaling, gradients are clipped to a fixed norm to prevent exploding updates, and invalid values (NaNs/Infs) are detected and filtered at both data-loading and loss-computation stages. A learning-rate scheduler monitors the validation loss and reduces the step size when progress plateaus, which helps refine convergence in later epochs.

Validation is run after each epoch on held-out meshes to track generalization. In addition to the global mean-squared error, we report an auxiliary “shadow-region” error: vertices with low reference irradiance (approximated by clamped SH brightness) form a mask, and the MSE is computed on this subset to specifically assess self-occlusion fidelity. Training proceeds for a fixed number of epochs or until convergence as indicated by the validation metrics, after which the model weights are serialized for deployment. This regimen consistently yields stable convergence across meshes of varying complexity while keeping the procedure compatible with real-time constraints at inference.

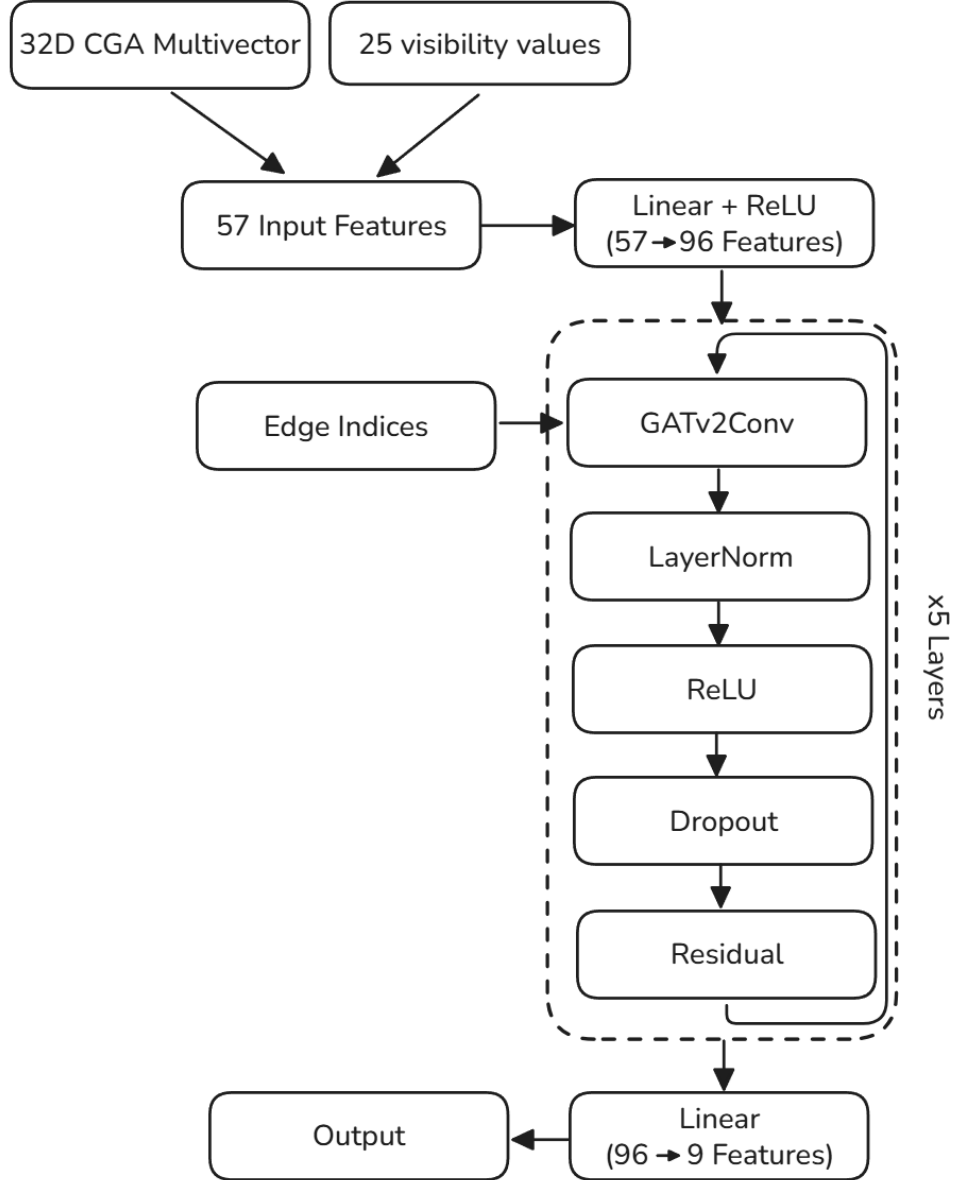


Figure 25: A flowchart of the coefficients GNN workflow.

5.3.2 Visibility GNN

The objective of this network, as discussed before, is to learn to approximate binary visibility samples for each vertex directly from the CGA representation of the mesh. Each vertex is described by a 32-dimensional motor-based feature vector derived from its position and normal. These features are globally normalized (per dimension) using dataset-wide mean and variance to reduce bias across meshes of different scales and improve stability. The target values are 25 binary visibility indicators per vertex, obtained from offline ray-casting.

Graphs are constructed by connecting vertices within a radius determined adaptively from the average pairwise distances in the normalized mesh. This ensures consistent neighborhood sizes across different models. Each mesh is pro-

cessed once and cached as a graph containing normalized vertex features, spatial connectivity, and the corresponding visibility labels.

The architecture of the visibility GNN is illustrated in Figure 26. The network receives as input a 32-dimensional CGA multivector representation of each vertex-normal pair, which encodes its geometric and spatial relationships. These features are first processed by a linear layer that projects them to a 64-dimensional latent space, followed by a ReLU activation. The core of the architecture consists of three stacked layers, each comprising a TransformerConv operator, Layer Normalization, ReLU activation, Dropout, and a residual connection. The edge indices, which encode the mesh connectivity, are explicitly provided only to the TransformerConv modules, enabling them to aggregate information from neighboring vertices in a topology-aware manner. The subsequent normalization and non-linear transformations refine the aggregated features, while residual connections ensure stability during training and facilitate deeper propagation of features. After the stacked layers, a dropout layer with $p = 0.2$ is applied to improve generalization, followed by a fully connected linear layer that maps the 64 latent features to 25 output values. These outputs correspond to the per-vertex visibility samples, which are later used as part of the Neural-GASh pipeline for SH coefficient prediction.

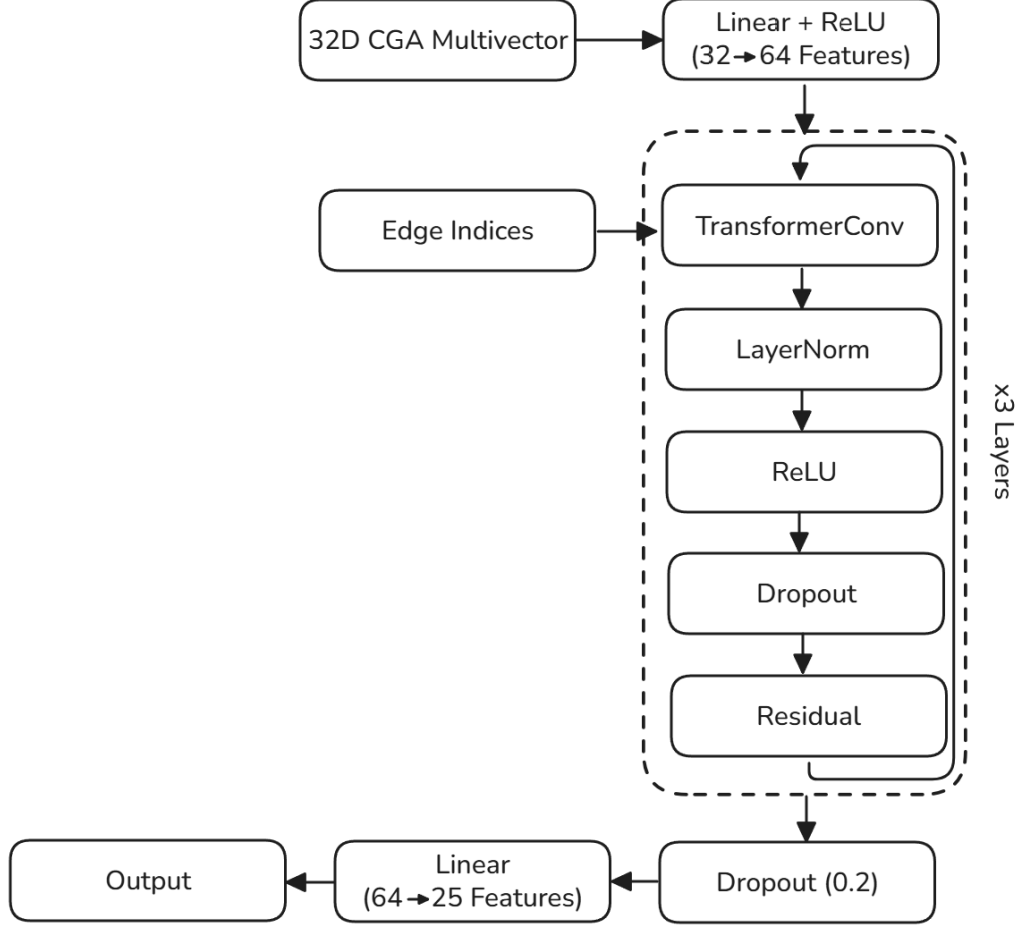


Figure 26: A flowchart of the visibility GNN workflow.

The dataset is split into training and validation (80/20) subsets at the mesh level. During training, graphs are batched and passed through the Visibility GNN, which produces per-vertex logits for the 25 visibility samples. The outputs are supervised with a class-balanced binary cross-entropy loss: for each visibility direction, positive and negative samples are reweighted according to their prevalence across the dataset. This prevents the model from biasing toward the majority “visible” class.

Optimization is performed with AdamW, using weight decay for regularization. A cosine annealing learning-rate schedule with warmup is employed, which gradually ramps up the learning rate and then decays it smoothly, improving convergence stability. To account for the highly imbalanced binary nature of the task, evaluation metrics include not only the average loss but also accuracy and the macro-averaged F1 score [83], which provides a more balanced measure across classes.

The best model checkpoint is selected based on validation F1 score. Early stopping is applied if no improvement is observed for a fixed patience interval, preventing overfitting and unnecessary computation. This procedure enables the

Visibility GNN to act as a surrogate for expensive visibility computations at inference time, providing fast and reasonably accurate approximations of ray-based sampling.

For more information regarding the Visibility or Coefficients GNN training, please refer to Appendix B.2.

5.4 Unity Integration

At this stage, the research had reached a significant milestone. Two complementary GNNs had been developed: the first was capable of predicting the SH coefficients required by the traditional PRT algorithm, while the second provided the corresponding visibility samples needed as input by the first. Together, these models constituted a functional pipeline that successfully reproduced the components of PRT without relying on costly precomputation.

The remaining challenge was to investigate the feasibility of deploying this pipeline within the Unity game engine, with the ultimate objective of enabling its use in real-time applications. Unlike the earlier MLP-based models, which could be directly integrated through Unity’s Barracuda interface, the GNN-based approach entered the domain of deep learning architectures. As a result, the direct execution of such models within Unity was non-trivial. A straightforward solution would have been to export the trained models in ONNX format and deploy them through Barracuda. However, since Barracuda supports only relatively simple architectures such as MLPs, several essential operations required by the GNNs (e.g., attention mechanisms such as GATv2) were not supported. Implementing these operations manually within Unity would have been prohibitively complex, error-prone, and time-consuming. Consequently, an alternative integration strategy was required.

5.4.1 Flask Server

The most practical and reliable solution was the deployment of a Flask server [27]. The role of this server is to initialize and host the two deep learning networks, while maintaining an open communication channel with the Unity environment. Upon receiving a request from Unity, the server performs inference through the networks, obtains the corresponding results, and transmits them back to Unity. This approach provides a stable and efficient pipeline for the evolved deep learning version of Neural-GASh, as illustrated in Figure 27. For further details on the code operating on the Flask server, refer to Appendix B.3.

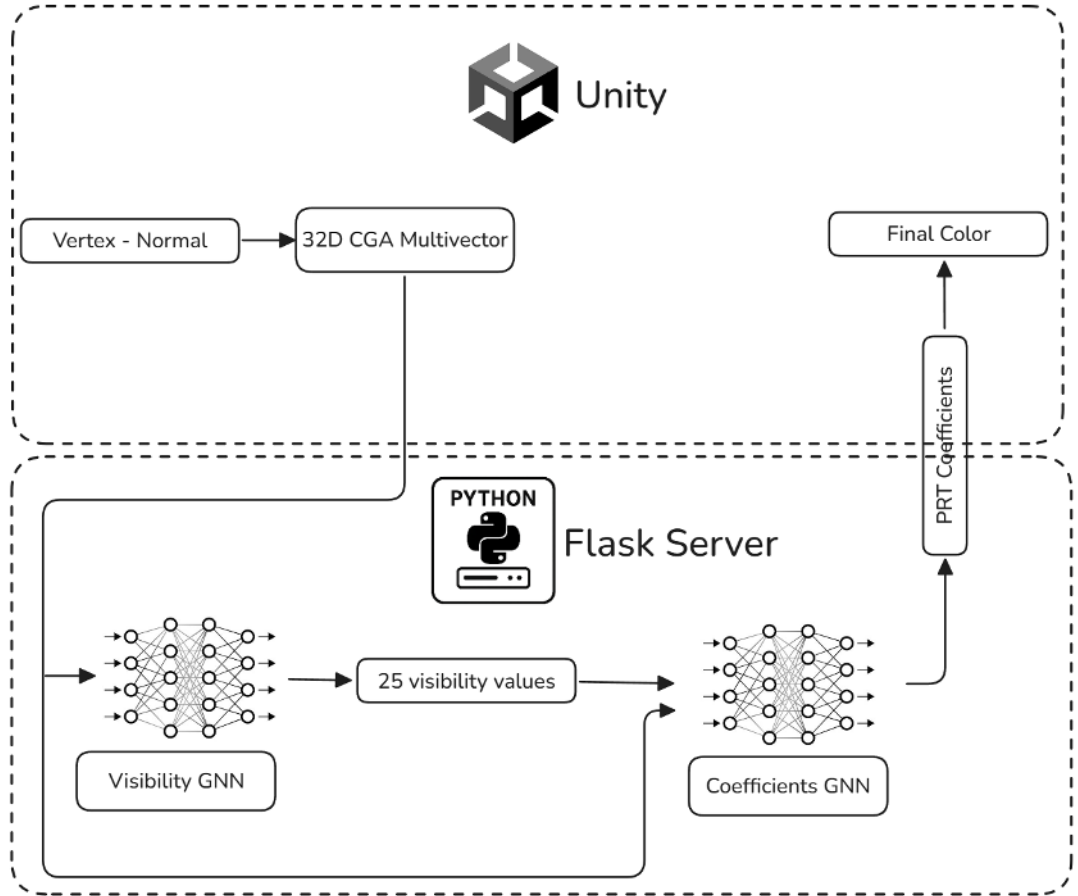


Figure 27: Abstract pipeline of the GNN-based Neural-GASH and its communication with the Flask server.

To enhance realism and self-shadowing in the neural predictions, we integrated an AO [8] post-processing stage that operates directly on the predicted SH coefficients. The method constructs a local neighborhood around each mesh vertex by connecting it with nearby vertices within a radius proportional to the bounding box diagonal, ensuring that only spatially relevant neighbors are considered. For every such pair, the direction vector between the source vertex and its neighbor is projected onto the source normal, allowing us to determine whether the neighbor lies above or below the tangent plane of the source. At each vertex, neighbors whose projections yield negative values are considered to contribute occlusion. The occlusion factor is then computed as the proportion of these occluding neighbors relative to the total set of neighbors. This factor, which represents the degree to which ambient light is locally blocked, is used to attenuate the predicted coefficients, with a mild damping term applied to avoid over-darkening. As a result, vertices in cavities or shadowed regions receive reduced radiance contributions, thereby reintroducing the subtle self-occlusion effects absent from the original neural model. The procedure is lightweight, geometry-driven, and suitable for real-time execution, while significantly improving the visual fidelity of

the rendered output.

5.4.2 Unity Pipeline

As illustrated in Figure 27, the Unity client begins by iterating over the vertex–normal pairs of the selected mesh and transforming them into 32-dimensional CGA multivectors, using the GA-Unity framework [41]. These multivectors are then transmitted to the Flask server via a dedicated Unity C# script, `FlaskServerComm`. This component, automatically attached to the mesh if not already present, manages the communication between Unity and the server and ensures the transmission of the required data for processing.

Upon receiving a request, the Flask server initiates the inference pipeline. First, the CGA multivectors are processed by the Visibility GNN, which predicts the 25 visibility samples per vertex. The same multivectors, augmented with the predicted visibility samples, are then passed to the Coefficients GNN, which produces the final SH coefficients for each vertex–normal pair. Once computation is complete, the server returns the SH coefficients to Unity. Unity then resumes the pipeline as in the original Neural-GASH system, computing the final vertex colors by applying a dot product between the predicted SH coefficients and the light coefficients extracted from the HDRI environment map. Since the Unity application must maintain real-time performance, it does not stall while waiting for results; instead, it continues execution and asynchronously checks for the arrival of the coefficients, applying them immediately upon receipt. This asynchronous design may introduce a minor delay, which will be analyzed in a subsequent section.

As illustrated in Figure 28, the proposed pipeline is organized into three principal stages: (i) Unity-side processing, (ii) server-side processing via Flask, and (iii) GPU-based processing within a compute shader. In this case, a compute shader was employed to further accelerate the procedure, as the conversion of vertex-normal pairs into CGA multivectors is already computationally intensive. Each stage contributes to the overall functionality and efficiency of the system.

The pipeline comprises six sequential workflows, illustrated in Figure 28 and labeled with the numbers 1–6. Specifically:

1. Vertex–normal pairs are encoded in the CGA representation.
2. The resulting multivectors are transmitted to the Flask server for neural inference.
3. The predicted SH coefficients are returned to Unity.
4. These coefficients are forwarded to the Neural-GASH component.

5. The GPU compute shader evaluates per-vertex color values.
6. The final colors are applied to the material for real-time rendering.

On the Unity side, two main *GameObjects* are employed. The first hosts the core logic script, the *NGASh Component*, which is attached alongside the mandatory *Transform Component*. The *NGASh Component* exposes two configurable fields: one specifying the file path of the compute shader and another referencing the target 3D mesh object to be illuminated. The second *GameObject* contains the mesh to be rendered and includes the *Transform Component*, a *Mesh Filter Component* responsible for providing vertex and normal data, and the *FlaskServerComm Component*, which handles communication between Unity and the Flask server. As with the MLP-based version of Neural-GASh, the mesh is rendered with a simple *NGASh Material*, implemented as a lightweight vertex-fragment shader pair.

The pipeline executes as follows, according to the sequence highlighted in Figure 27. In the first step, the *NGASh Component* preprocesses the vertex-normal pairs and constructs the corresponding CGA multivectors, which are forwarded to the *FlaskServerComm Component*. In the second stage, these multivectors are transmitted to the Flask server for further processing. On the server side, the CGA multivectors are first processed by the *Visibility GNN*, which computes 25 visibility samples per vertex. These visibility features, together with the original CGA multivectors, are then passed to the *Coefficients GNN*, which predicts the SH coefficients. Subsequently, an AO step refines these predictions, yielding AO-enhanced SH coefficients. During the third stage, these enhanced coefficients are returned to Unity via the *FlaskServerComm Component*. In the fourth stage, the *FlaskServerComm Component* forwards the coefficients to the *NGASh Component*, which, in the fifth stage, dispatches them to the GPU for compute shader execution. The compute shader evaluates the final per-vertex colors under the given environment lighting. Finally, in the sixth stage, the computed colors are written back to the *NGASh Material*, producing the illuminated result directly on the mesh in the Unity scene. The pipeline code is available in Appendix B.4.

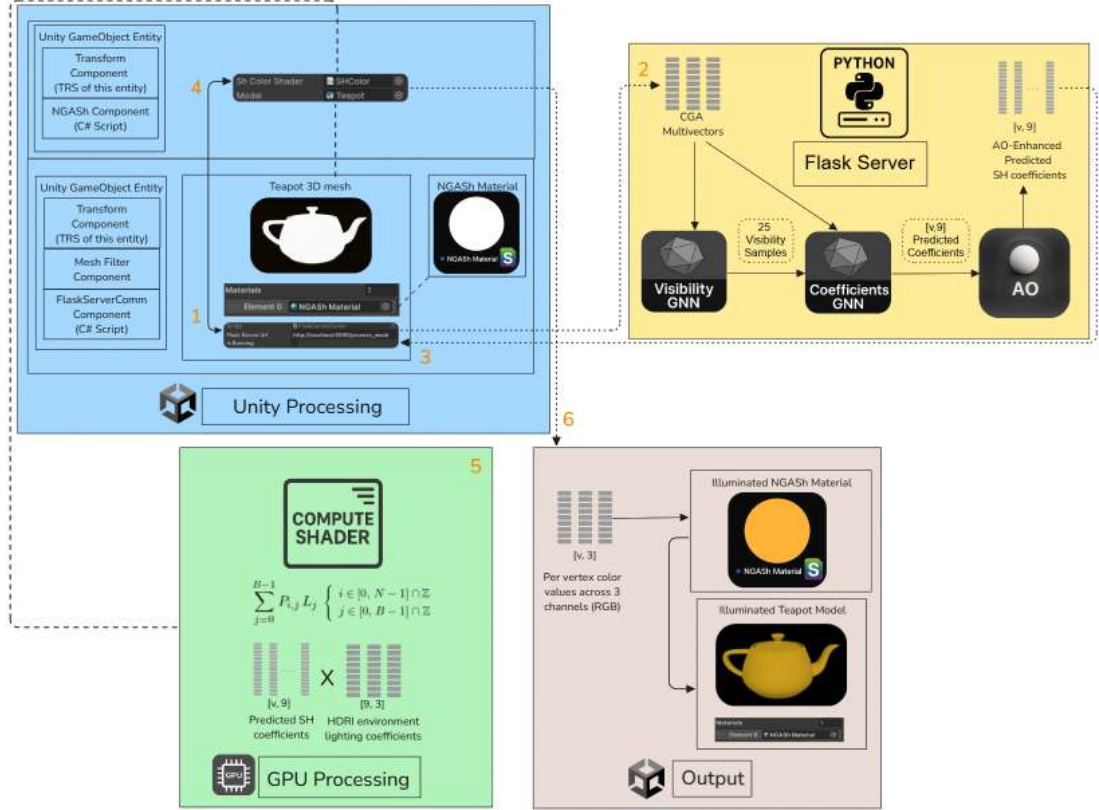


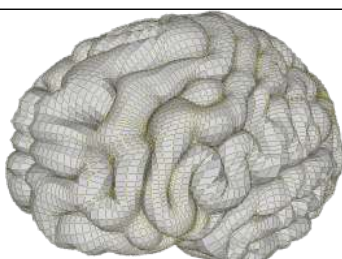
Figure 28: Detailed pipeline of GNN-based Neural-GASH.

5.5 Results

All experiments were conducted on a workstation equipped with an AMD Ryzen 7 7700 eight-core processor operating at 3.80 GHz, 32 GB of system memory, and an NVIDIA GeForce RTX 4060 graphics card with 8 GB of dedicated VRAM. This configuration reflects a modern consumer-grade system and was selected to ensure that the reported performance results are both reproducible and representative of realistic hardware conditions available to typical end users.

5.5.1 Qualitative Analysis

For the evaluation of our method, we employed a diverse set of 3D meshes with varying levels of geometric complexity. The dataset includes the Coolseal (9673 vertices), Bipolar (3808 vertices), Focus (8534 vertices), Brain (21966 vertices), Heart (29661 vertices), and Human Anatomy (33995 vertices) models, as shown in Table 7. This range of mesh complexities allows us to assess the scalability and robustness of the proposed approach across both relatively simple and highly detailed geometries.

Mesh Name	Mesh Wireframe	Vertex Count	Traditional PRT Time (s)	GNN-based Neural-GASH Time (s)
Coolseal		9673	417.54	0.14
Bipolar		3808	151.90	0.05
Focus		8534	369.27	0.09
Brain		21966	1051.54	0.29

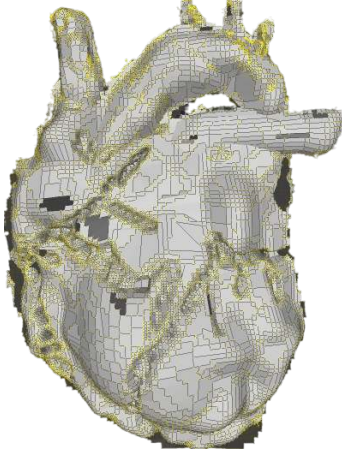
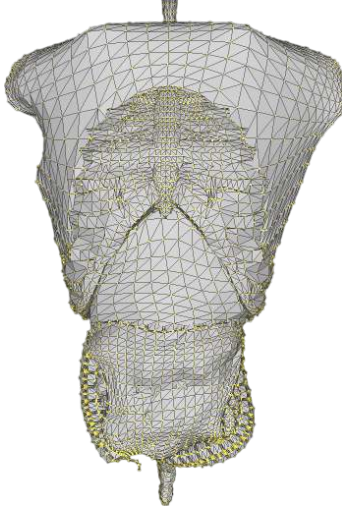
Mesh Name	Mesh Wireframe	Vertex Count	Traditional PRT Time (s)	GNN-based Neural-GASH Time (s)
Heart		29661	1619.16	2.69
Human Anatomy		33995	1908.58	4.13

Table 7: Precomputation times for various meshes using Traditional PRT and GNN-based Neural-GASH.

For these results we focus on medical content and equipment never seen by our pipeline during training or validation. Tables 8–10 provide a qualitative evaluation of the proposed GNN-based Neural-GASH method compared to traditional PRT and the MLP baseline under three distinct illumination conditions: dark-lit, bright-lit, and normal-lit environments. These comparisons allow us to assess the perceptual fidelity of the predicted results in scenarios with different lighting intensities, which strongly influence the visibility of shading detail and illumination transfer.

In the dark-lit environment (Table 8), the shading contributions are subtle

and illumination differences are more difficult to perceive. Figure 29 contains the HDRI map that we used for the meshes in this table. Across all six meshes, GNN-based Neural-GASH produces renderings that are visually indistinguishable from the traditional PRT results, whereas the MLP baseline exhibits slightly less accurate shading in certain regions. The difference maps highlight that the majority of discrepancies occur along object boundaries and in fine geometric detail, such as folds in the heart or brain surface. Importantly, these differences remain minor in magnitude, which explains why they are not perceptually noticeable in the rendered images.

The bright-lit environment (Table 9) presents a more challenging scenario, as high-intensity illumination amplifies discrepancies between methods. Figure 30 contains the HDRI map that we used for the meshes in this table. Even under this extreme condition, GNN-based Neural-GASH reproduces the global illumination effects with high fidelity across all meshes. The difference maps show stronger error signals in bright regions, particularly in concave areas where occlusion effects dominate; however, these localized deviations do not alter the overall appearance. Compared to the MLP baseline, GNN-based Neural-GASH better preserves contrast and fine shading variations, which demonstrates the advantage of the GNN architecture in modeling complex light transport.

Finally, the normal-lit environment (Table 10) provides a balanced test case representative of practical usage. Figure 31 contains the HDRI map that we used for the meshes in this table. Once again, the results of GNN-based Neural-GASH are nearly indistinguishable from the traditional PRT solution. Across both anatomical and surgical instrument meshes, shading gradients, occlusion effects, and overall light distribution are faithfully captured. The difference maps reveal that remaining errors are localized and relatively uniform in scale, further confirming the stability of the method.

Taken together, these results demonstrate that GNN-based Neural-GASH maintains robust visual results across a wide range of illumination conditions. Unlike the MLP baseline, which is unable to predict self-shadowing and self-occlusion elements, the GNN-based model consistently generalizes to unseen meshes and lighting setups.

5.5.2 Quantitative Analysis

Figure 32 highlights the substantial efficiency gains achieved by the proposed GNN-based Neural-GASH method compared to the traditional PRT pipeline. Across all six test meshes, Neural-GASH consistently outperforms the baseline by several orders of magnitude, reducing computation times from hundreds or even thousands of seconds to well under five seconds. For instance, the Brain mesh,



Figure 29: The HDRI map that was used to provide the results in Table 8. The HDRI is sourced from *Poly Haven* [62].

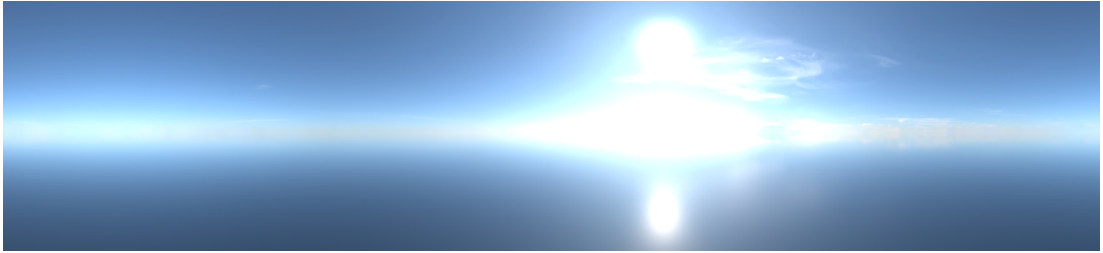


Figure 30: The HDRI map that was used to provide the results in Table 9. The HDRI is sourced from *Poly Haven* [62].



Figure 31: The HDRI map that was used to provide the results in Table 10. The HDRI is sourced from *Poly Haven* [62].

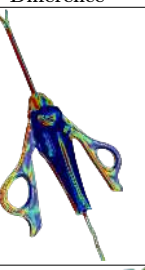
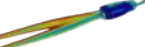
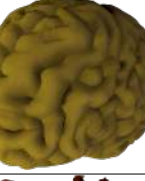
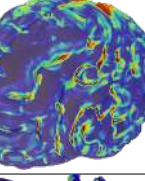
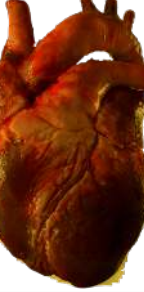
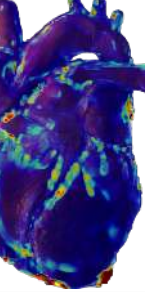
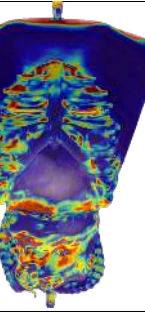
Traditional PRT	Neural-GASH (MLP)	Neural-GASH (GNN)	AO-Enhanced	Difference
				
				
				
				
				
				

Table 8: Results comparison of six different meshes under a dark lit environment.

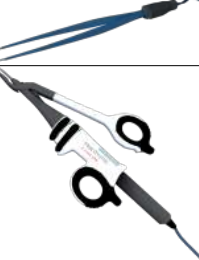
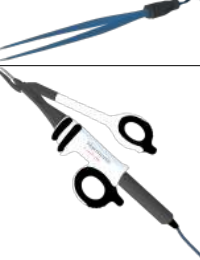
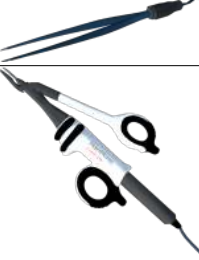
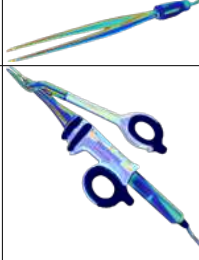
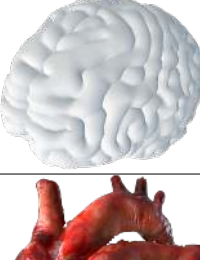
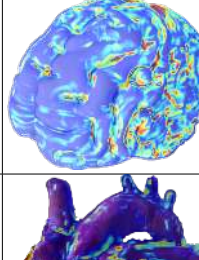
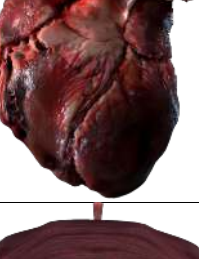
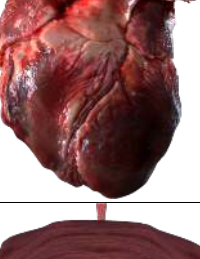
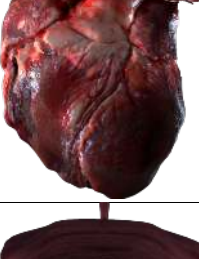
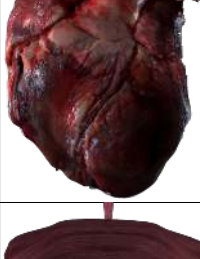
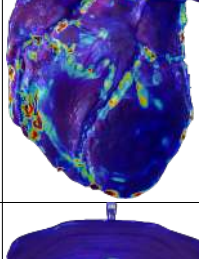
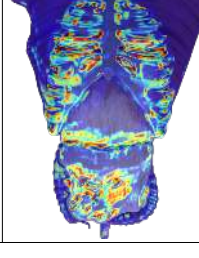
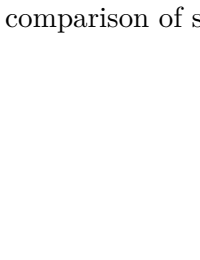
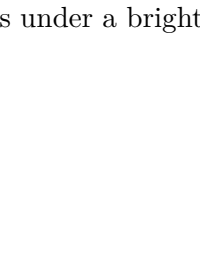
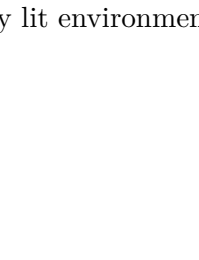
Traditional PRT	Neural-GASH (MLP)	Neural-GASH (GNN)	AO-Enhanced	Difference
				
				
				
				
				
				

Table 9: Results comparison of six different meshes under a brightly lit environment.

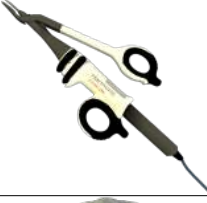
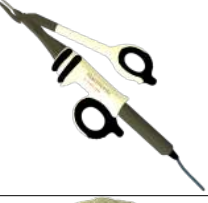
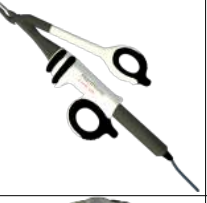
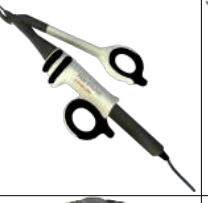
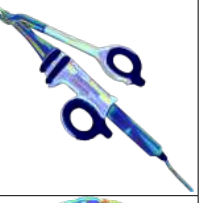
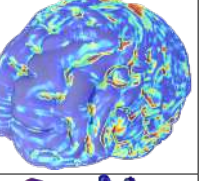
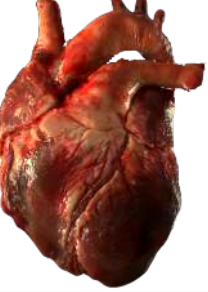
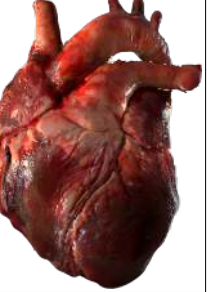
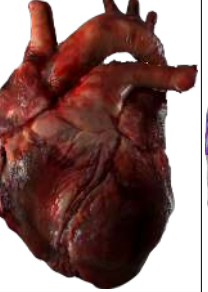
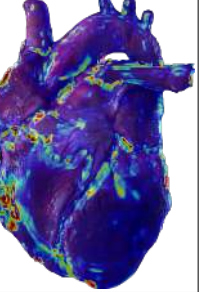
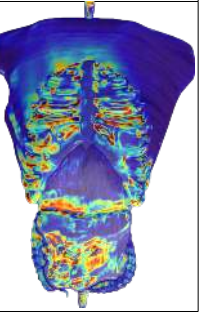
Traditional PRT	Neural-GASH (MLP)	Neural-GASH (GNN)	AO-Enhanced	Difference
				
				
				
				
				
				

Table 10: Results comparison of six different meshes under a normal lit environment.

which required over 1050 seconds with the traditional approach, was processed in only 0.29 seconds with GNN-based Neural-GASH, representing a speedup factor of more than 3000. Similarly, the most complex mesh in the study, Human Anatomy, required approximately 1908 seconds with conventional PRT but only 4.13 seconds using our approach. This performance gain is particularly noteworthy, since the experimental protocol required that all test meshes be strictly excluded from the training and validation sets, thereby demonstrating strong generalization ability and robustness to unseen geometry. Overall, the results confirm that Neural-GASH not only provides substantial speedup but also scales effectively with mesh complexity, making it a practical and viable alternative for interactive or real-time global illumination applications.

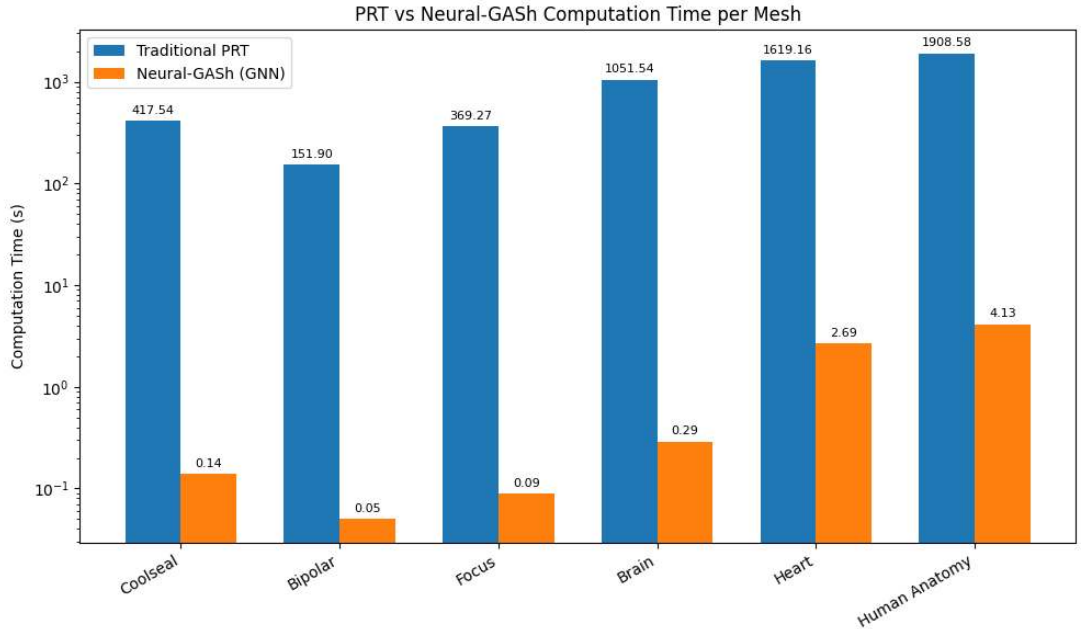


Figure 32: Time comparison in seconds (s) of the traditional PRT algorithm and the GNN-based version of Neural-GASH.

Figures 33 and 34 provide a detailed analysis of the accuracy and stability of the GNN-based Neural-GASH method by comparing its predicted SH coefficients against those obtained with the traditional PRT pipeline. The mean squared error (MSE) results in Figure 33 demonstrate that the method achieves consistently low errors across all nine coefficient indices, with values typically below 0.01 and only a few peaks reaching approximately 0.017. Such small magnitudes confirm that the predicted coefficients closely match the ground truth even for meshes that were not part of the training or validation sets. Notably, the errors are not uniformly distributed across the SH coefficients: in the coefficient enumeration used in our experiments, indices 3 and 7—i.e., the $m = 0$ (zonal) terms of the $l = 1$ and $l = 2$ bands—tend to exhibit higher deviations across several

meshes. This behavior is consistent with SH theory for diffuse illumination: zonal components carry most of the low-frequency, azimuthally symmetric energy of the Lambertian kernel, whereas non-zonal ($m \neq 0$) components contribute less to the overall irradiance and are easier to approximate locally [65, 73]. Interestingly, this pattern can also be attributed to the locality bias of GNNs: since the GNN learns primarily through message passing among neighboring vertices, it effectively captures directional and neighborhood-level correlations but remains less sensitive to global illumination variations that influence the zonal terms. Consequently, the higher errors observed in these coefficients likely reflect the global nature of the irradiance field rather than a deficiency in the network’s learning capacity. Nevertheless, even in these more difficult cases, the error remains modest, suggesting that the GNN-based Neural-GASH is able to capture the dominant illumination structure with high fidelity.

The standard deviation (STD) analysis in Figure 34 complements the MSE evaluation by measuring the spread of errors across all vertices. The results show that the majority of coefficients have standard deviations below 0.1, with the lowest values approaching 0.02. This indicates that the error distribution is tightly clustered and that the model’s predictions are not only accurate on average but also consistent across the surface of each mesh. The higher variability observed in the same coefficients where MSE peaks occur (indices 3 and 7) further supports their identification as the zonal ($m = 0$) SH terms of the $l = 1$ and $l = 2$ bands. These terms encode the dominant, low-frequency irradiance that integrates lighting over the entire hemisphere. As a result, even small local prediction deviations can accumulate into larger global discrepancies, explaining the higher standard deviation observed for these coefficients [65]. Importantly, the general trends are consistent across all six test meshes, ranging from highly complex anatomical structures to mechanical surgical instruments. This consistency provides strong evidence of the robustness and generalization ability of the GNN-based Neural-GASH: despite differences in geometry, topology, and material appearance, the method produces stable and reliable results.

Taken together, Figures 33 and 34 highlight both the strengths and the limitations of the proposed approach. GNN-based Neural-GASH clearly demonstrates the ability to reproduce PRT coefficients with low error and low variability, which directly translates into visually faithful renderings, as confirmed by the qualitative comparisons. At the same time, the presence of localized peaks in the error distribution draws attention to the inherent difficulty of capturing high-frequency lighting detail with a compact neural representation. These insights not only validate the method’s effectiveness but also provide a clear direction for future improvements, such as extending the representation to capture higher-frequency

illumination or incorporating perceptual loss functions that align coefficient accuracy more directly with visual quality.

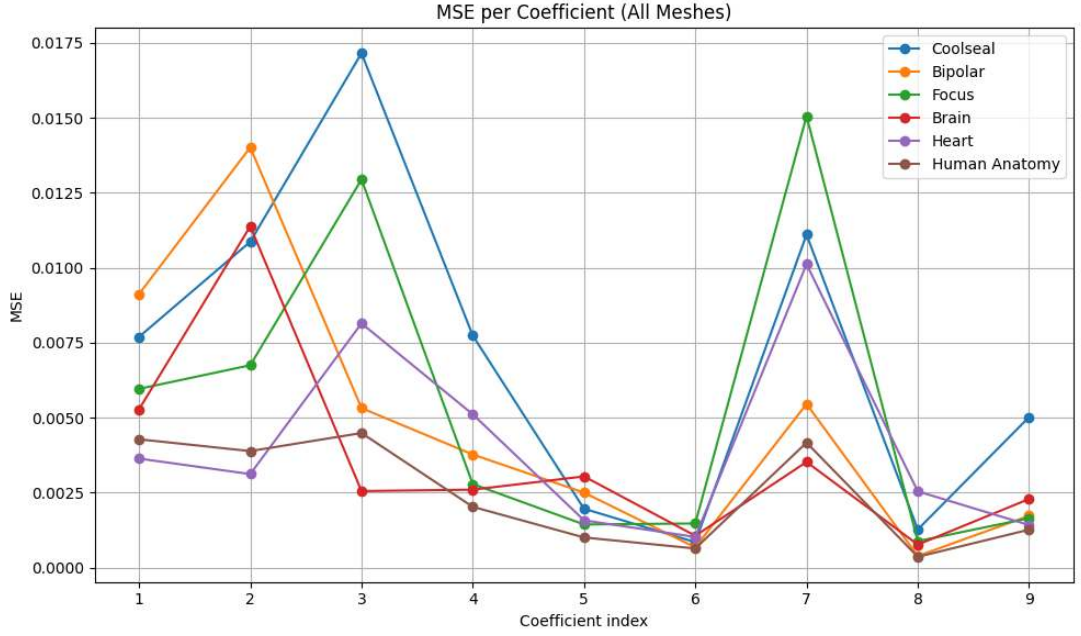


Figure 33: Mean Squared Error (MSE) of coefficients between the traditional PRT algorithm and GNN-based version of Neural-GASH, across the 9 coefficients indices.

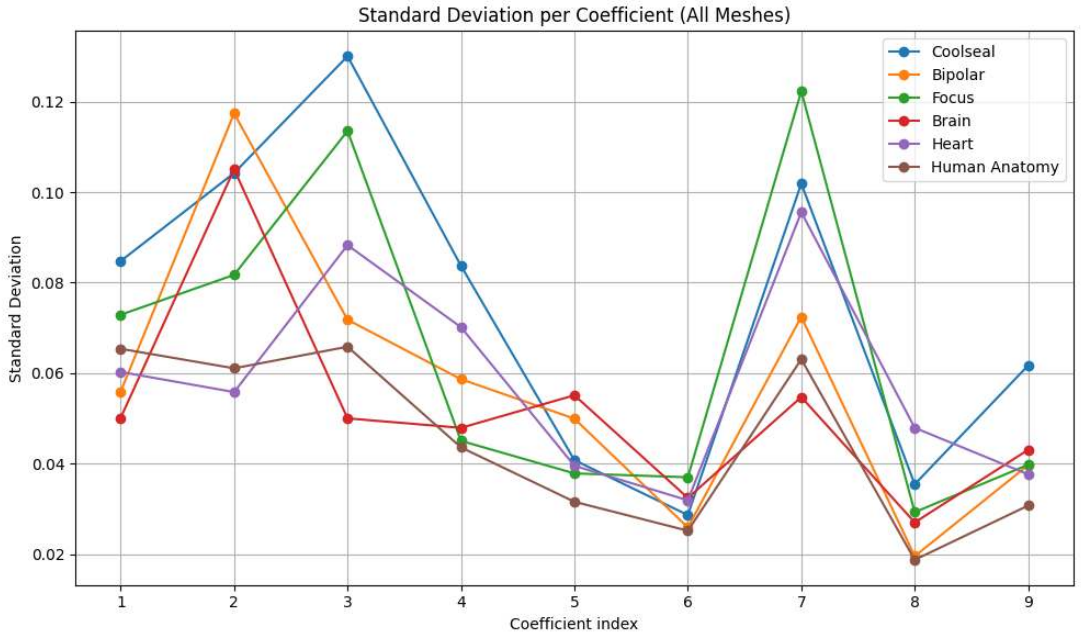


Figure 34: Standard Deviation (STD) of coefficients between the traditional PRT algorithm and GNN-based version of Neural-GASH, across the 9 coefficients indices.

Figure 35 illustrates the prediction accuracy of the proposed visibility network across six representative 3D assets of varying geometric complexity. The

models exhibit diverse behaviors depending on their structural continuity and spatial topology. As shown, smoother and more coherent geometries such as the Brain and Human Anatomy achieve the highest prediction accuracies (0.72 and 0.82, respectively), demonstrating the ability of the network to learn stable visibility patterns when surface connectivity is consistent and self-occlusion effects are locally smooth.

In contrast, the Bipolar, Coolseal, and Heart models exhibit comparatively lower accuracy scores, which can be attributed to their pronounced geometric discontinuities and internal cavities. The Bipolar instrument, resembling a surgical grasper with elongated jaws and narrow interlocking gaps, introduces abrupt topological transitions and sharp self-occlusions. These structural features create rapidly fluctuating visibility patterns across adjacent vertices, making local radiance transfer inherently more complex to approximate. Similarly, the Coolseal and Heart models contain deep cavities and open boundaries, including discontinuities, which amplify visibility variation. The Focus geometry also presents localized concavities, though to a lesser extent, resulting in moderate prediction accuracy. Importantly, these variations do not reflect instability in the network but rather the intrinsic complexity of the underlying visibility field. Despite these challenges, the model consistently preserves coherent illumination structure across all assets, demonstrating its generalization capability even under severe topological irregularities.

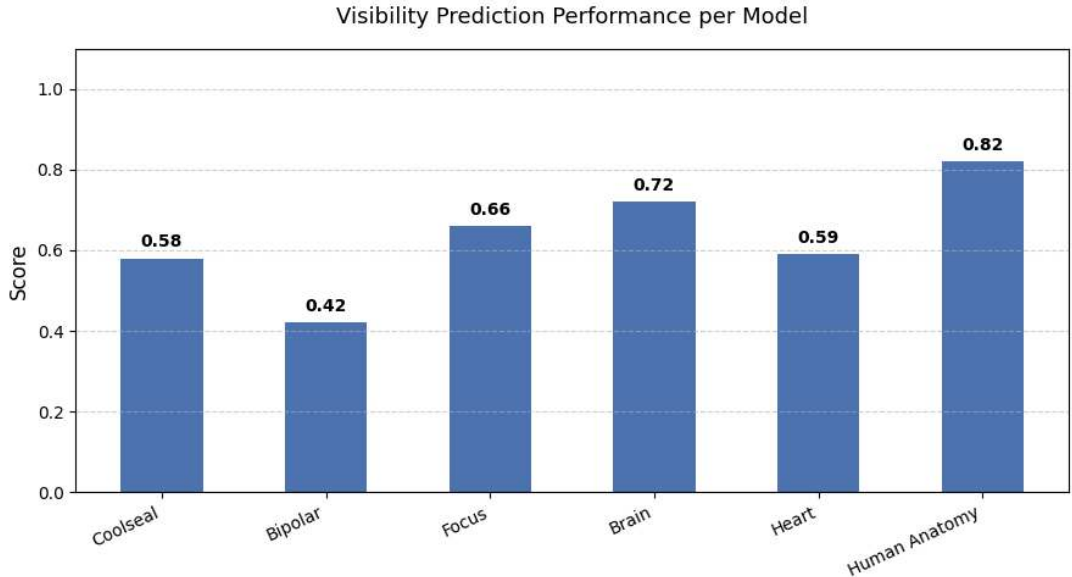


Figure 35: Prediction accuracy of the proposed visibility network across different 3D models.

5.5.3 Flask Server Latency Analysis

A critical aspect of the Neural-GASH integration pipeline concerns the latency introduced by the Flask server, which mediates communication between Unity and the PyTorch-based inference backend. Since each rendered frame requires a round-trip request–response cycle, it is important to quantify the per-stage processing time, evaluate how this scales with mesh complexity, and assess whether the overall latency remains compatible with interactive rendering budgets.

Figure 36 reports the breakdown of end-to-end request latency into nine stages:

1. **Data Reception from Unity** – raw binary transmission.
2. **Deserialization and Data Separation** – extraction of vertex positions, normals, and CGA features.
3. **Visibility Network Input Preparation** – normalization of multivectors prior to GNN inference.
4. **Graph Construction** – neighborhood building (via k -nearest neighbors).
5. **Visibility GNN Inference** – prediction of per-vertex visibility descriptors.
6. **Coefficient Network Input Preparation** – feature concatenation and normalization.
7. **Coefficient GNN Inference** – prediction of SH coefficients.
8. **Ambient Occlusion Pass** – optional lightweight per-vertex adjustment using local geometry.
9. **Serialization and Response Preparation** – conversion of coefficients to binary and packaging for return.

Across the seven tested meshes, the measured latencies ranged from tens of milliseconds for small and moderately complex assets (e.g., Humanoid, Bipolar) up to several seconds for the largest anatomy models (e.g., Heart, Human Anatomy). The principal bottlenecks were identified in the two GNN inference stages (5 and 7) as well as the graph generation (stage 4), as expected, with the AO pass (stage 8) contributing significant cost for large meshes due to the quadratic complexity of naïve neighborhood checks. Importantly, the AO step is optional and can be omitted entirely in scenarios where fine-scale detail is not critical.

For applications where strict real-time responsiveness is required, a fallback option exists in the MLP-based Neural-GASH variant. This version trades some shading fidelity for dramatically reduced latency, as it bypasses the GNN-heavy server pipeline altogether and runs directly inside Unity via the inference engine.

Mesh	Vertices	Min (ms)	Max (ms)	Total (ms)
Humanoid Animated	5340	0.11	65.75	120.11
Coolseal	9673	0.10	59.51	146.47
Bipolar	3808	0.05	34.89	68.10
Focus	8534	0.15	52.15	121.85
Brain	21966	0.11	111.03	271.30
Heart	29661	0.10	1582.75	2730.27
Human Anatomy	33995	0.13	2459.41	4886.75

Table 11: End-to-end round-trip latency measured on the Flask server for various mesh models. All values are reported in milliseconds and represent the average of ten inference requests per mesh. The total latency includes data transmission, neural inference, and ambient occlusion processing.

It is also worth noting that in static scenes the end-to-end server inference cost is a one-time expense, since coefficients are computed once and reused. The animated humanoid case was included here specifically to test continuous per-frame communication, where the server must respond to every frame. To further mitigate overhead in practice, Unity-side request throttling is employed: if the per-frame motion or rotation is below a perceptual threshold (e.g., 0.2 world units), no new request is sent, since the lighting change would be imperceptible. This policy significantly reduces unnecessary server queries during subtle or slow movements.

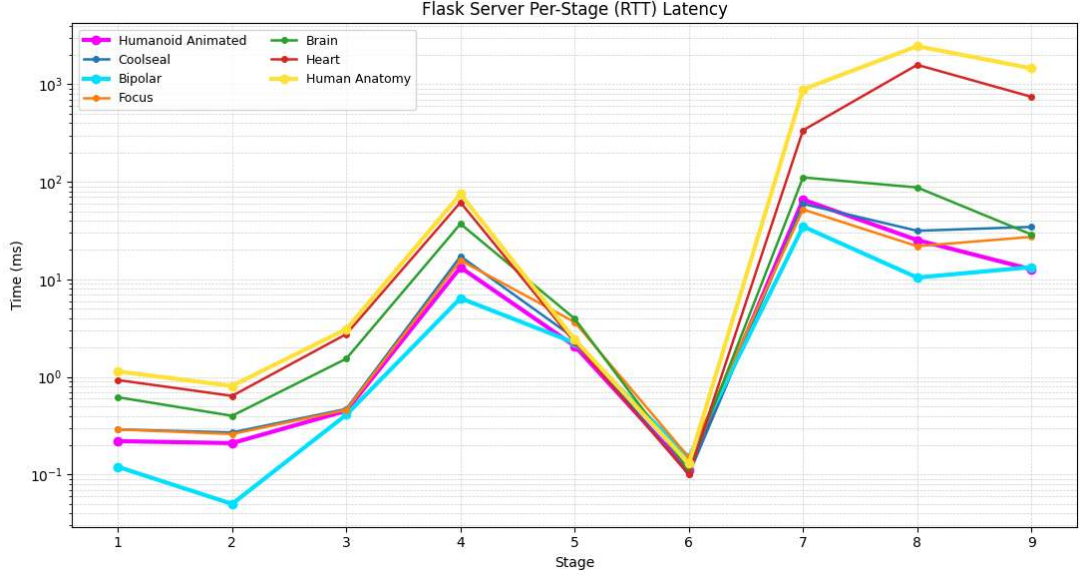


Figure 36: Per-stage latency breakdown of the Flask server for seven different meshes. Each line corresponds to a mesh, with time (in milliseconds, log scale) plotted across the nine pipeline stages. The AO stage (8) is optional, and performance-sensitive applications can omit it.

In our setup Unity and the Flask server run on the same workstation, so wire latency reduces to local IPC/loopback cost. Stage 1 (*Data reception*) is a direct proxy for this transfer and is consistently sub-millisecond to a few milliseconds across all meshes (e.g., ~ 0.1 – 1.1 ms; Table 11). The return path is bundled into Stage 9 (*Serialization & response prep*); the dominant cost there is server-side packing (copy/format) rather than transit time. In short, with a local server, network/IPC time is negligible compared to graph construction and GNN inference; the measured round-trip is therefore compute-bound.

For interactive applications, a key benchmark is the frame time budget: at frame rates of 30–60 FPS, the maximum allowable per-frame latency is approximately 16–33 ms. Our measurements indicate that for small to mid-sized meshes (≤ 50 K vertices), the round-trip time typically falls within the range of 20–100 ms, which is acceptable for near-real-time rendering and animation, though not strictly within the 60 FPS budget. For large medical models exceeding 500K vertices, the latency increases by more than an order of magnitude, with end-to-end times in the hundreds or thousands of milliseconds. In the case of the Human Anatomy mesh in Table 11, the elevated processing time on the Flask server is not primarily attributed to its 34K vertices, but rather to its composition of nine distinct submeshes treated as a single entity, which increases the complexity of the AO computation.

Two scaling trends emerge clearly:

- **Linear scaling** of GNN inference time with vertex count, consistent with

the message-passing architecture.

- **Superlinear scaling** of the AO pass, whose cost grows significantly with mesh size.

While the prototype implementation demonstrates feasibility, several optimizations are needed to meet strict real-time thresholds. These include reducing Python–Unity communication overhead, migrating inference to Unity’s native inference engine (thus avoiding network round-trips), and replacing the AO kernel with GPU-accelerated or approximated alternatives. With such refinements, we expect latency for mid-sized meshes to be consistently maintained below 50–100 ms, ensuring compatibility with interactive frame rates in practical applications.

5.5.4 Animated Geometry

In addition to supporting static geometry, the GNN-based Neural-GASh framework is also capable of handling animated geometry as illustrated in Figure 37, a feature that is not feasible with traditional PRT. This capability is achieved through the *NGAShModel component* in Unity, which continuously monitors whether the geometry has been updated. When an update is detected, the system computes the corresponding CGA multivectors from the transformed vertex–normal pairs and sends a query to the Flask server to obtain the updated SH coefficients. The server then returns the new coefficients, enabling real-time updates that accurately reflect the animated geometry. As illustrated in 13, Neural-GASh also introduces certain performance optimizations: for example, during rapid animations, intermediate poses may be skipped. This strategy improves efficiency without perceptible loss in visual fidelity, as rapid changes are unlikely to be noticed by the user.

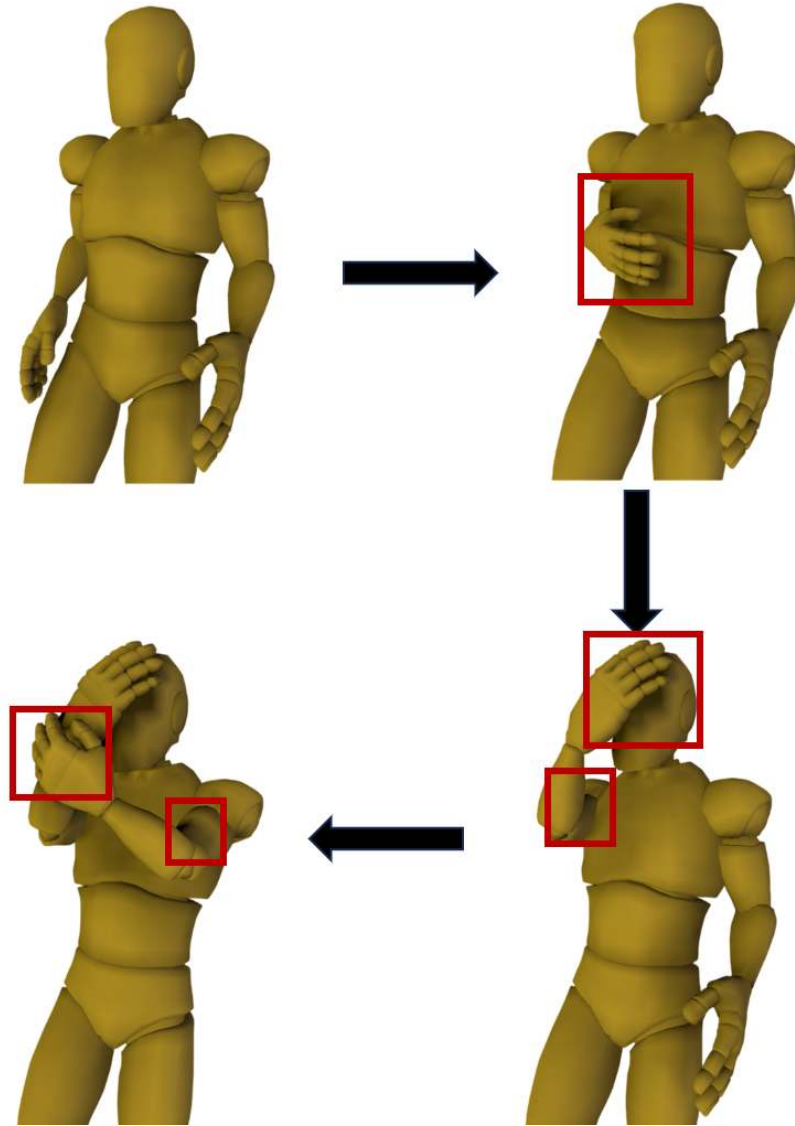


Figure 37: Example of the GNN-based Neural-GASH applied on animated geometry. The animation progresses from left to right and the shadows are highlighted within the red boxes. The HDRI map in Figure 29 is utilized to light the object.

5.5.5 Multiple Objects & Scenes

The GNN-based Neural-GASH framework was designed with multi-model support in mind, following the principles established by its MLP-based predecessor. The self-shadowing mechanism operates locally on each individual model, ensuring that geometric details are consistently preserved. Multiple instances of models can coexist within the same scene, each equipped with its corresponding components as outlined previously, including a dedicated *NGASHModel* component. On the server side, the Flask backend is implemented to handle all incoming requests in parallel, thereby supporting interactive performance even in complex scenarios. Furthermore, as illustrated in Figure 38, the system is capable of operating on scenes composed of multiple objects, provided these objects are merged into a sin-

gle mesh file. In such cases, the shadowing effects naturally extend across object boundaries, allowing the shadows of one object to influence those of another.



(a)



(b)

Figure 38: Comparison of the surgery scene rendered with the GNN-based Neural-GASH (a) and Unity’s built-in global illumination (b). Under the simplest Unity GI configuration, Neural-GASH achieves superior results in both visual quality and performance. In particular, the objects under the table are more accurately lit in (a) as well as the cloth, as its shadow is more visible. Neural-GASH reproduces self-shadowing effects on the doctor, which are absent in the Unity rendering.

As illustrated in Figure 38, in order to provide a baseline comparison, we also reproduced the scene using Unity’s built-in global illumination system. For fairness, we employed the simplest possible setup: a single area light covering the scene, with its color manually adjusted to resemble the dominant illumination extracted from the light coefficients used by the GNN-based Neural-GASH. The meshes in the scene were flagged as static and the global illumination bake was executed, requiring approximately 42.8 seconds to complete.

However, Unity’s baked GI has well-known limitations when applied to animated content. In particular, skinned mesh renderers (such as the doctor model) are treated as dynamic geometry and therefore do not receive baked lighting. As a result, the doctor appeared unrealistically lit in the baked solution. To partially alleviate this, we added a light probe group so that the skinned mesh could receive at least approximate illumination. In contrast, Neural-GASH operates uniformly on both static and skinned meshes, computing per-vertex shading with self-shadowing effects, and required only 3.54 seconds to process the same scene.

Visually, the difference between the two methods is notable. GNN-based Neural-GASH produces stronger and more coherent shadowing, with consistent illumination across all geometry, including animated meshes. The Unity bake, by contrast, failed to capture self-shadowing on the doctor and relied on probes for approximate lighting. While both methods reproduce the overall light distribution, GNN-based Neural-GASH provides a more realistic result at a fraction of the computation time.

5.5.6 Scalability to Large Meshes

While the quantitative evaluation used meshes up to $\sim 34\text{K}$ vertices, many production assets are substantially larger. Here we discuss how the proposed pipeline scales and what engineering choices enable inference on large meshes.

Asymptotic costs. Let V be the vertex count, k the average neighborhood size, H the hidden width, and L the number of graph layers/blocks. With a fixed k , the time for a message-passing GNN forward pass scales approximately linearly:

$$T_{\text{GNN}} = \mathcal{O}(L \cdot (V + E) \cdot H) = \mathcal{O}(L \cdot V \cdot k \cdot H),$$

since $E \approx kV$. This complexity is consistent with standard analyses of GCN/-GAT layers, where each edge contributes a constant-size message [45, 85]. In our server, additional costs arise from graph construction and the optional AO pass. kNN graph building on the CPU/GPU scales roughly as $\mathcal{O}(V \log V)$ (tree-based) or $\mathcal{O}(Vk)$ with approximate methods [19]. The AO pass, if implemented with dense neighborhood checks, can grow super-linearly for large V ; omitting or replacing it (e.g., fixed-radius sparsity or tiled GPU kernels) restores near-linear behavior.

Memory footprint. Memory scales as $\mathcal{O}(V \cdot F + E \cdot H)$, where F is feature dimensionality (32 for CGA, 57 when concatenating predicted visibility). For large meshes, the edge tensor dominates; keeping k modest ($k \in [8, 16]$), using

half precision (FP16), and reusing a single shared edge index for both stages helps stay within commodity GPU budgets. This memory scaling behavior is consistent with empirical characterizations from large-scale graph learning systems [21].

Practical envelope and scalability considerations. In practice, the Neural-GASh framework maintains real-time performance for assets up to approximately $V \leq 50,000$ vertices, with round-trip inference (including visibility prediction) completing in the order of tens of milliseconds. For larger meshes, several strategies can be applied to sustain interactivity. One option is to partition the geometry into smaller, spatially coherent submeshes and process each in parallel. This approach preserves throughput but may introduce minor discontinuities in shading or self-shadowing across submesh boundaries. Alternatively, when self-occlusion is not a priority, the AO stage can be omitted, reducing graph density and computational overhead.

For performance-critical applications such as virtual or extended reality, the MLP-based Neural-GASh variant provides a practical alternative. While it lacks topology-aware features and therefore cannot capture detailed self-shadowing, it delivers significantly lower latency and memory usage, making it suitable for lightweight, real-time, or head-mounted rendering scenarios.

Static vs. animated content. For static or slowly changing scenes, coefficients can be computed once and reused, amortizing cost. For animated meshes, request throttling and pose-change thresholds on the client curb server load.

Summary. Under fixed k , both visibility and coefficient GNN inference scale near-linearly in V . The primary scaling hazards are graph construction and the optional AO post-pass; eliminating or approximating these, combined with tiling and one-pass multi-task architectures, extends the method to substantially larger production assets while preserving interactive performance targets where needed.

6 Discussion & Future Work

The results of this work demonstrate that the proposed Neural-GASh framework provides a viable and efficient alternative to traditional PRT. By replacing the heavy precomputation stage with a learned model, we achieved significant reductions in runtime, making real-time rendering feasible even in dynamic scenes. The adoption of CGA multivectors as the input representation proved particularly effective: while early experiments with simple Euclidean vertex-normal pairs often led to unstable training and artifacts, the 32-dimensional multivector encoding offered a richer and more stable input space. This allowed both MLP and GNN architectures to converge more reliably and to yield sharper reconstructions of radiance transfer coefficients.

A key insight emerged from the comparison between MLP- and GNN-based formulations. While both benefited from the CGA embedding, the GNNs were uniquely capable of exploiting mesh connectivity and neighborhood information. This property enabled them to capture self-occlusion effects and complex shadowing patterns that remained elusive in purely feed-forward architectures. Nevertheless, errors were observed to concentrate in specific SH coefficients, notably those associated with higher-frequency angular content, which had a disproportionate impact on visual quality. The teacher-student paradigm further highlighted the limitations of MLPs for this task, while experiments with dedicated visibility-prediction networks confirmed that topology-aware models can serve as effective providers of visibility cues. At the same time, this direction introduced additional architectural complexity that must be carefully managed in practical deployments.

It is important to emphasize that the current implementation is based exclusively on the *diffuse component* of the traditional PRT formulation. As a result, the method is achromatic and does not account for interreflections or specular transport. These phenomena, while critical for photorealistic rendering, introduce additional complexity in both precomputation and learning, and were therefore excluded from the present study. Extending Neural-GASh to support specular reflections, glossy materials, and interreflections constitutes a natural and necessary direction for future work.

Another important observation relates to the behavior of the SH coefficients predicted by the neural networks. Although the overall accuracy was high, errors were not uniformly distributed across all coefficients. In particular, indices 2, 3, and 7 consistently exhibited higher deviations across several meshes, as illustrated in Figures 33 and 34. This pattern can be explained by the role of SH in approximating lighting. For a given band index L , the number of basis

functions is $B = L^2$. In our implementation, we used three bands ($L = 2$), yielding nine coefficients in total. The lower-order coefficients (such as index 0 and 1) primarily encode the low-frequency, smooth components of lighting, whereas higher-order coefficients (e.g., indices 2, 3, and 7) capture more directional and oscillatory components that vary rapidly over the sphere. These higher-frequency terms are inherently more sensitive to small perturbations in geometry, sampling noise, or approximation errors introduced by the neural network. As a result, both traditional PRT and neural surrogates face greater challenges in predicting them accurately [91]. Nevertheless, even in these more difficult cases, the errors remained modest, and the GNN-based Neural-GASh was able to reproduce the dominant illumination structure with high fidelity, indicating that the essential radiance transfer signal is robustly captured.

6.1 Limitations

Despite its strengths, several limitations remain. First, while visibility samples are not required at inference, they are still necessary during training, and their extraction constitutes a bottleneck when generating large-scale datasets. Second, although the approach generalized well to the tested medical meshes, broader applicability across domains such as natural scenes, game assets, or animated characters has yet to be demonstrated. The scalability of the method is another concern: although real-time performance was achieved for meshes of approximately up to 35,000 vertices, it is unclear how inference pipelines will behave for models containing millions of vertices. Furthermore, the current work was restricted to three SH bands (nine coefficients). While sufficient for diffuse global illumination, higher-order expansions would be needed to represent sharper lighting effects, at the cost of greater computational demand. Finally, the Python-based prototype provided flexibility for experimentation, but it remains less optimized than a production-level C++ or CUDA implementation.

Additional limitations relate to the architectural and system-level design. In the two-stage GNN pipeline, prediction errors from the visibility network can propagate to the coefficient network, occasionally resulting in blurred or misplaced shadow boundaries. Moreover, while Unity–Flask communication was found to be acceptable on a local workstation (tens of milliseconds for mid-sized meshes), the quantitative latency budget leaves little margin for high-FPS applications and could worsen in remote or distributed setups. Memory usage is another constraint: although the current evaluation covered meshes up to $\sim 34K$ vertices, scaling to models with millions of vertices would stress GPU RAM due to the quadratic growth of edge tensors. Careful engineering (e.g., tiling, spar-

sification, compression) will be necessary to make the method practical at that scale.

6.2 Future Work

Several avenues emerge for future research. The most immediate priority is the expansion of the training dataset, both in mesh diversity and lighting conditions. Larger and more heterogeneous data would strengthen generalization and mitigate overfitting, moving the method closer to robust deployment across varied applications. A second major direction is the removal of the Unity–Flask communication bottleneck. Deploying GNN inference directly inside Unity—through native runtimes or dedicated plugins—would eliminate round-trip latency and make the pipeline viable for production-level interactive rendering. A third critical step is to extend Neural-GASh beyond diffuse transfer by incorporating specular and interreflection components. Training on richer transport phenomena would enable the framework to reproduce glossy, reflective, and more photorealistic effects.

Other promising directions include developing sample- and band-independent architectures, as well as exploring transformer-based GNNs or implicit neural representations. These models are particularly appealing because they can capture long-range dependencies and non-local visibility cues more effectively than standard message-passing GNNs, thereby reducing the required depth and alleviating issues of over-smoothing and scalability [10]. On the systems side, further optimizations such as C++/Vulkan backends, pruning, and quantization could substantially improve runtime efficiency. Finally, packaging the approach into developer-friendly APIs for Unity or Unreal would facilitate adoption in practical game and visualization pipelines.

6.3 Conclusion

PRT has long been valued for its ability to capture global illumination effects with high fidelity, but its reliance on expensive offline sampling and coefficient precomputation makes it impractical for dynamic or interactive scenes. This thesis addressed that gap by introducing *Neural-GASh*, a framework that reformulates PRT within a neural rendering paradigm. By encoding vertex–normal pairs as CGA multivectors and training modern neural architectures, we demonstrated that radiance transfer can be learned and inferred in real time without traditional precomputation.

The key contributions of this work are threefold. First, we established that CGA-based representations improve stability and expressiveness compared to Euclidean inputs, enabling neural models to learn light transport more effec-

tively. Second, we showed that GNNs, leveraging mesh connectivity, capture self-shadowing and occlusion effects that MLPs fail to reproduce. Third, we integrated the resulting models into Unity via a server-based pipeline, demonstrating practical deployment in interactive environments with dynamic and animated geometry. Together, these contributions transform PRT from a static precomputation technique into a neural pipeline compatible with real-time applications.

Several challenges remain: reliance on visibility samples during training, limited scalability to million-vertex meshes, the restriction to diffuse transfer, and Python–Unity communication overhead. These limitations, however, point directly toward future work in dataset expansion, optimized GPU/C++ implementations, and extensions to specular and higher-order lighting.

In conclusion, this thesis establishes that combining geometric algebra with GNN-based neural models provides a viable path toward real-time radiance transfer. By overcoming the fundamental precomputation bottleneck of traditional PRT, Neural-GASh lays groundwork for the next generation of neural rendering pipelines, enabling interactive global illumination in applications ranging from gaming to medical visualization and virtual reality.

Bibliography

- [1] AKGWSB. CasualPRT, 2022. Accessed: 2025-03-03. URL: <https://github.com/AKGWSB/CasualPRT>.
- [2] Junjie Bai, Fang Lu, Ke Zhang, et al. ONNX: Open Neural Network Exchange. <https://github.com/onnx/onnx>, 2019.
- [3] Jonathan T. Barron, Ben Mildenhall, Dor Verbin, Pratul P. Srinivasan, and Peter Hedman. Mip-NeRF 360: Unbounded Anti-Aliased Neural Radiance Fields. In *2022 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 5460–5469, 2022. doi:10.1109/CVPR52688.2022.00539.
- [4] R. Basri and D.W. Jacobs. Lambertian reflectance and linear subspaces. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 25(2):218–233, 2003. doi:10.1109/TPAMI.2003.1177153.
- [5] Laurent Belcour, Thomas Deliot, Wilhem Barbier, and Cyril Soler. A Data-Driven Paradigm for Precomputed Radiance Transfer. *Proc. ACM Comput. Graph. Interact. Tech.*, 5(3), July 2022. doi:10.1145/3543864.
- [6] Stéphanie Bertrand, Martha Vassiliadi, Paul Zikas, Efstratios Geronikolakis, and George Papagiannakis. From Readership to User-ship: Communicating Heritage Digitally Through Presence, Embodiment and Aesthetic Experience. *Frontiers in Communication*, Volume 6 - 2021, 2021. URL: <https://www.frontiersin.org/journals/communication/articles/10.3389/fcomm.2021.676446>, doi:10.3389/fcomm.2021.676446.
- [7] James F. Blinn. Models of light reflection for computer synthesized pictures. *SIGGRAPH Comput. Graph.*, 11(2):192–198, July 1977. doi:10.1145/965141.563893.
- [8] Michael Bunnell. *Chapter 14 Dynamic Ambient Occlusion and Indirect Lighting*, volume 2, pages 223–233. 01 2005.
- [9] c he. PRT-SH, 2018. GitHub repository, Accessed: 2025-03-03. URL: <https://github.com/c-he/PRT-SH>.

- [10] Dexiong Chen, Leslie O’Bray, and Karsten Borgwardt. Structure-aware transformer for graph representation learning. In *International conference on machine learning*, pages 3469–3489. PMLR, 2022.
- [11] Guikun Chen and Wenguan Wang. A Survey on 3D Gaussian Splatting. *CoRR*, abs/2401.03890, 2024. URL: <https://doi.org/10.48550/arXiv.2401.03890>.
- [12] David Cline and Parris Egbert. A practical introduction to metropolis light transport. *Brigham Young University*, 2005.
- [13] Robert L. Cook, Thomas Porter, and Loren Carpenter. Distributed ray tracing. *SIGGRAPH Comput. Graph.*, 18(3):137–145, January 1984. doi:10.1145/964965.808590.
- [14] Cyril Crassin, Fabrice Neyret, Miguel Sainz, Simon Green, and Elmar Eisemann. Interactive indirect illumination using voxel-based cone tracing: an insight. In *ACM SIGGRAPH 2011 Talks*, SIGGRAPH ’11, New York, NY, USA, 2011. Association for Computing Machinery. doi:10.1145/2037826.2037853.
- [15] Cyrille Damez, Kirill Dmitriev, and Karol Myszkowski. Global Illumination for Interactive Applications and High-Quality Animations. In *Eurographics 2002 - STARs*. Eurographics Association, 2002. doi:/10.2312/egst.20021050.
- [16] Matt Deitke, Ruoshi Liu, Matthew Wallingford, Huong Ngo, Oscar Michel, Aditya Kusupati, Alan Fan, Christian Laforte, Vikram Voleti, Samir Yitzhak Gadre, Eli VanderBilt, Aniruddha Kembhavi, Carl Vondrick, Georgia Gkioxari, Kiana Ehsani, Ludwig Schmidt, and Ali Farhadi. Objaverse-XL: a universe of 10M+ 3D objects. In *Proceedings of the 37th International Conference on Neural Information Processing Systems*, NIPS ’23, Red Hook, NY, USA, 2023. Curran Associates Inc.
- [17] Matt Deitke, Dustin Schwenk, Jordi Salvador, Luca Weihs, Oscar Michel, Eli VanderBilt, Ludwig Schmidt, Kiana Ehsani, Aniruddha Kembhavi, and Ali Farhadi. Objaverse: A Universe of Annotated 3D Objects. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 13142–13153, 2023.
- [18] Sirikonda Dhawal, Aakash Kt, and P. J. Narayanan. Transfer Textures for Fast Precomputed Radiance Transfer. In Basile Sauvage and Jasminka Hasic-

- Telalovic, editors, *Eurographics 2022 - Posters*. The Eurographics Association, 2022. doi:10.2312/egp.20221012.
- [19] Wei Dong, Charikar Moses, and Kai Li. Efficient k-nearest neighbor graph construction for generic similarity measures. In *Proceedings of the 20th International Conference on World Wide Web, WWW '11*, page 577–586, New York, NY, USA, 2011. Association for Computing Machinery. doi:10.1145/1963405.1963487.
- [20] Geronikolakis Efstratios, Tsioumas Michael, Bertrand Stephanie, Loupas Athanasios, Zikas Paul, and Papagiannakis George. New Cross/Augmented Reality Experiences for the Virtual Museums of the Future. In Marinos Ioannides, Eleanor Fink, Raffaella Brumana, Petros Patias, Anastasios Doulamis, João Martins, and Manolis Wallace, editors, *Digital Heritage. Progress in Cultural Heritage: Documentation, Preservation, and Protection*, pages 518–527, Cham, 2018. Springer International Publishing.
- [21] Matthias Fey and Jan Eric Lenssen. Fast Graph Representation Learning with Pytorch Geometric. *ArXiv*, abs/1903.02428, 2019. URL: <https://api.semanticscholar.org/CorpusID:70349949>.
- [22] Efstratios Geronikolakis, Manos Kamarianakis, Antonis Protopsaltis, and George Papagiannakis. Neural-GASh: A CGA-based neural radiance prediction pipeline for real-time shading. In *Computer Graphics International 2025 (CGI 2025) – ENGAGE 2025 Workshop*, Lecture Notes in Computer Science. Springer, 2025. (Best GA Computing Paper Award).
- [23] Efstratios Geronikolakis and George Papagiannakis. An XR rapid prototyping framework for interoperability across the reality spectrum. *arXiv preprint arXiv:2101.01771*, 2021.
- [24] Efstratios Geronikolakis, Paul Zikas, Steve Kateros, Nick Lydatakis, Stelios Georgiou, Mike Kentros, and George Papagiannakis. *A True AR Authoring Tool for Interactive Virtual Museums*, pages 225–242. Springer International Publishing, Cham, 2020. doi:10.1007/978-3-030-37191-3_12.
- [25] Jianping Gou, Baosheng Yu, Stephen J. Maybank, and Dacheng Tao. Knowledge Distillation: A Survey. *Int. J. Comput. Vision*, 129(6):1789–1819, June 2021. doi:10.1007/s11263-021-01453-z.
- [26] Robin Green. Spherical harmonic lighting: The gritty details. In *Archives of the game developers conference*, volume 56, page 4, 2003.

- [27] Miguel Grinberg. *Flask web development*. " O'Reilly Media, Inc.", 2018.
- [28] Christina Gsaxner, Shohei Mori, Dieter Schmalstieg, Jan Egger, Gerhard Paar, Werner Bailer, and Denis Kalkofen. DeepDR: Deep Structure-Aware RGB-D Inpainting for Diminished Reality . In *2024 International Conference on 3D Vision (3DV)*, pages 750–760, Los Alamitos, CA, USA, March 2024. IEEE Computer Society. URL: <https://doi.ieeecomputersociety.org/10.1109/3DV62453.2024.00037>, doi:10.1109/3DV62453.2024.00037.
- [29] Antoine Guédon and Vincent Lepetit. SuGaR: Surface-Aligned Gaussian Splatting for Efficient 3D Mesh Reconstruction and High-Quality Mesh Rendering. In *2024 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 5354–5363, 2024. doi:10.1109/CVPR52733.2024.00512.
- [30] Xuanyu He, Wei Zhang, Haifeng Zhang, Lin Ma, and Yibin Li. Reversible data hiding for high dynamic range images using edge information. *Multimedia Tools Appl.*, 78(20):29137–29160, October 2019. doi:10.1007/s11042-018-6589-x.
- [31] Dietmar Hildenbrand. Geometric computing in computer graphics using conformal geometric algebra. *Computers & Graphics*, 29(5):795–803, 2005.
- [32] Dietmar Hildenbrand. *Foundations of Geometric Algebra Computing*. Springer Publishing Company, Incorporated, 2012.
- [33] Geoffrey E. Hinton, Oriol Vinyals, and Jeffrey Dean. Distilling the Knowledge in a Neural Network. *ArXiv*, abs/1503.02531, 2015. URL: <https://api.semanticscholar.org/CorpusID:7200347>.
- [34] Eckhard Hitzer. Inner Product of Two Oriented Points in Conformal Geometric Algebra in Detail. In David William Honorio Araujo Da Silva, Dietmar Hildenbrand, and Eckhard Hitzer, editors, *Advanced Computational Applications of Geometric Algebra*, pages 77–96, Cham, 2024. Springer Nature Switzerland.
- [35] Eckhard Hitzer, Manos Kamarianakis, George Papagiannakis, and Petr Vašík. Survey of new applications of geometric algebra. *Mathematical Methods in the Applied Sciences*, 47(14):11368–11384, 2024.
- [36] Eckhard Hitzer, George Papagiannakis, and Petr Vasik. Empowering Novel Geometric Algebra for Graphics and Engineering. In *7th International Workshop on Empowering Novel Geometric Algebra for Graphics and Engineer-*

- ing (ENGAGE 2022), Virtual Event, September 12, 2022, Lecture Notes in Computer Science. Springer, 2023. doi:10.1007/978-3-031-30923-6_6.
- [37] David S. Immel, Michael F. Cohen, and Donald P. Greenberg. A radiosity method for non-diffuse environments. *SIGGRAPH Comput. Graph.*, 20(4):133–142, August 1986. doi:10.1145/15886.15901.
- [38] Alec Jacobson. Common 3D Test Models (GitHub repository). <https://github.com/alecjacobson/common-3d-test-models>, 2022. Accessed: 2025-01-18.
- [39] Henrik Wann Jensen. Global illumination using photon maps. In *Proceedings of the Eurographics Workshop on Rendering Techniques '96*, page 21–30, Berlin, Heidelberg, 1996. Springer-Verlag.
- [40] James T. Kajiya. The rendering equation. *SIGGRAPH Comput. Graph.*, 20(4):143–150, August 1986. doi:10.1145/15886.15902.
- [41] Manos Kamarianakis, Nick Lydatakis, and George Papagiannakis. GA-Unity: A Production-Ready Unity Package for Seamless Integration. In *Advances in Computer Graphics: 41st Computer Graphics International Conference, CGI 2024, Geneva, Switzerland, July 1–5, 2024, Proceedings, Part III*, page 349–363, Berlin, Heidelberg, 2025. Springer-Verlag. doi:10.1007/978-3-031-82024-3_27.
- [42] Manos Kamarianakis and George Papagiannakis. An All-in-One Geometric Algorithm for Cutting, Tearing, and Drilling Deformable Models. *Advances in Applied Clifford Algebras*, 31(3):58, 2021. doi:10.1007/s00006-021-01151-6.
- [43] Bernhard Kerbl, Georgios Kopanas, Thomas Leimkuehler, and George Drettakis. 3D Gaussian Splatting for Real-Time Radiance Field Rendering. *ACM Trans. Graph.*, 42(4), July 2023. doi:10.1145/3592433.
- [44] John Kessenich, Graham Sellers, and Dave Shreiner. *OpenGL Programming Guide: The official guide to learning OpenGL, version 4.5 with SPIR-V*. Addison-Wesley Professional, 2016.
- [45] Thomas N. Kipf and Max Welling. Semi-Supervised Classification with Graph Convolutional Networks. In *International Conference on Learning Representations*, 2017. URL: <https://openreview.net/forum?id=SJU4ayYgl>.

- [46] Eric Lafortune and Yves Willems. Rendering participating media with bi-directional path tracing. In *Eurographics Workshop on Rendering Techniques*, pages 91–100. Springer, 1996.
- [47] Eric Lafortune and Yves Willems. Bi-Directional Path Tracing. *Proceedings of Third International Conference on Computational Graphics and Visualization Techniques (Compugraphics)*, 93, 01 1998.
- [48] Yue Li, Pablo Wiedemann, and Kenny Mitchell. Deep Precomputed Radiance Transfer for Deformable Objects. *Proc. ACM Comput. Graph. Interact. Tech.*, 2(1), June 2019. doi:10.1145/3320284.
- [49] Zhihao Liang, Qi Zhang, Wenbo Hu, Lei Zhu, Ying Feng, and Kui Jia. Analytic-Splatting: Anti-Aliased 3D Gaussian Splatting via Analytic Integration. In Aleš Leonardis, Elisa Ricci, Stefan Roth, Olga Russakovsky, Torsten Sattler, and Gül Varol, editors, *Computer Vision – ECCV 2024*, pages 281–297, Cham, 2025. Springer Nature Switzerland.
- [50] Zander Majercik, Thomas Mueller, Alexander Keller, Derek Nowrouzezahrai, and Morgan McGuire. Dynamic Diffuse Global Illumination Resampling. In *ACM SIGGRAPH 2021 Talks*, SIGGRAPH ’21, New York, NY, USA, 2021. Association for Computing Machinery. doi:10.1145/3450623.3464635.
- [51] Ben Mildenhall, Pratul P. Srinivasan, Matthew Tancik, Jonathan T. Barron, Ravi Ramamoorthi, and Ren Ng. NeRF: representing scenes as neural radiance fields for view synthesis. *Commun. ACM*, 65(1):99–106, December 2021. doi:10.1145/3503250.
- [52] Tomas Möller and Ben Trumbore. Fast, minimum storage ray/triangle intersection. In *ACM SIGGRAPH 2005 Courses*, SIGGRAPH ’05, page 7–es, New York, NY, USA, 2005. Association for Computing Machinery. doi:10.1145/1198555.1198746.
- [53] Richard M. Murray, S. Shankar Sastry, and Li Zexiang. *A Mathematical Introduction to Robotic Manipulation*. CRC Press, Inc., USA, 1st edition, 1994.
- [54] Margarita Papaefthymiou, Dietmar Hildenbrand, and George Papagianakis. A Conformal Geometric Algebra Code Generator Comparison for Virtual Character Simulation in Mixed Reality. *Advances in Applied Clifford Algebras*, 27(3):2051–2066, 2017. doi:10.1007/s00006-016-0689-3.

- [55] Margarita Papaefthymiou, Marios Evangelos Kanakis, Efstratios Geronikolakis, Argyrios Nochos, Paul Zikas, and George Papagiannakis. *Rapid Reconstruction and Simulation of Real Characters in Mixed Reality Environments*, pages 267–276. Springer International Publishing, Cham, 2018. doi:10.1007/978-3-319-75826-8_22.
- [56] Margarita Papaefthymiou and George Papagiannakis. Real-time rendering under distant illumination with Conformal Geometric Algebra. *Mathematical Methods in the Applied Sciences*, 41(11):4131–4147, 2018.
- [57] George Papagiannakis, Efstratios Geronikolakis, Maria Pateraki, Víctor Manuel Bendicho, Michael Tsioumas, Stella Sylaiou, Fotis Liarokapis, Athina Grammatikopoulou, Kosmas Dimitropoulos, Grammalidis Nikos, Nikolaos Partarakis, George Margetis, Giannis Drossis, Martha Vassiliadi, Alan Chalmers, Constantine Stephanidis, and Nadia Thalmann. *Mixed Reality Gamified Presence and Storytelling for Virtual Museums*. 02 2018. doi:10.1007/978-3-319-08234-9_249-2.
- [58] George Papagiannakis, Paul Zikas, Nick Lydatakis, Steve Kateros, Mike Kentros, Efstratios Geronikolakis, Manos Kamarianakis, Ioanna Kartsonaki, and Giannis Evangelou. MAGES 3.0: Tying the knot of medical VR. In *ACM SIGGRAPH 2020 Immersive Pavilion*, SIGGRAPH ’20, New York, NY, USA, 2020. Association for Computing Machinery. doi:10.1145/3388536.3407888.
- [59] Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, Alban Desmaison, Andreas Köpf, Edward Yang, Zach DeVito, Martin Raison, Alykhan Tejani, Sasank Chilamkurthy, Benoit Steiner, Lu Fang, Junjie Bai, and Soumith Chintala. *PyTorch: an imperative style, high-performance deep learning library*. Curran Associates Inc., Red Hook, NY, USA, 2019.
- [60] John Petropoulos, Manos Kamarianakis, Antonis Protopsaltis, and George Papagiannakis. pyGANDALF - An open-source Geometric, ANimation, Directed, Algorithmic, Learning Framework for Computer Graphics. In *SIGGRAPH Asia 2024 Educator’s Forum*, SA ’24, New York, NY, USA, 2024. Association for Computing Machinery. doi:10.1145/3680533.3697057.
- [61] Matt Pharr, Wenzel Jakob, and Greg Humphreys. *Physically Based Rendering: From Theory to Implementation*. Morgan Kaufmann Publishers Inc., San Francisco, CA, USA, 3rd edition, 2016.

- [62] Poly Haven. Poly Haven HDRIs. Accessed: 2024-12-10. URL: <https://polyhaven.com/hdris>.
- [63] Nasim Rahaman, Aristide Baratin, Devansh Arpit, Felix Dräxler, Min Lin, Fred A. Hamprecht, Yoshua Bengio, and Aaron C. Courville. On the Spectral Bias of Neural Networks. In *International Conference on Machine Learning*, 2018. URL: <https://api.semanticscholar.org/CorpusID:53012119>.
- [64] Gilles Rainer, Adrien Bousseau, Tobias Ritschel, and George Drettakis. Neural Precomputed Radiance Transfer. *Computer Graphics Forum (Proceedings of the Eurographics conference)*, 41(2), April 2022. URL: <http://www-sop.inria.fr/reves/Basilic/2022/RBRD22>.
- [65] Ravi Ramamoorthi and Pat Hanrahan. An efficient representation for irradiance environment maps. In *Proceedings of the 28th Annual Conference on Computer Graphics and Interactive Techniques*, SIGGRAPH '01, page 497–500, New York, NY, USA, 2001. Association for Computing Machinery. doi:10.1145/383259.383317.
- [66] Sara Rojas, Jesus Zarzar, Juan C. Pérez, Artsiom Sanakoyeu, Ali Thabet, Albert Pumarola, and Bernard Ghanem. Re-ReND: Real-time Rendering of NeRFs across Devices. In *2023 IEEE/CVF International Conference on Computer Vision (ICCV)*, pages 3609–3618, 2023. doi:10.1109/ICCV51070.2023.00336.
- [67] David Ruhe, Jayesh K. Gupta, Steven De Keninck, Max Welling, and Johannes Brandstetter. Geometric clifford algebra networks. In *Proceedings of the 40th International Conference on Machine Learning*, ICML'23. JMLR.org, 2023.
- [68] ONNX Runtime Developers. ONNX Runtime. <https://onnxruntime.ai/>, 2025. Accessed: 2025-03-07, Version: 1.18.0.
- [69] Amit Saha. Everything You Need to Know About Knowledge Distillation (aka Teacher–Student model). <https://shorturl.at/INuvC>, 2020. Accessed: 2025-04-24.
- [70] Andreas Schneider, Sandro Schönborn, Bernhard Egger, Lavrenti Frobeen, and Thomas Vetter. Efficient Global Illumination for Morphable Models. In *2017 IEEE International Conference on Computer Vision (ICCV)*, pages 3885–3893, 2017. doi:10.1109/ICCV.2017.417.

- [71] Johannes L. Schönberger, Enliang Zheng, Jan-Michael Frahm, and Marc Pollefeys. Pixelwise View Selection for Unstructured Multi-View Stereo. In Bastian Leibe, Jiri Matas, Nicu Sebe, and Max Welling, editors, *Computer Vision – ECCV 2016*, pages 501–518, Cham, 2016. Springer International Publishing.
- [72] Johannes L. Schönberger and Jan-Michael Frahm. Structure-from-Motion Revisited. In *2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 4104–4113, 2016. doi:10.1109/CVPR.2016.445.
- [73] Peter-Pike Sloan. Stupid Spherical Harmonics (SH) Tricks. *Game Developers Conference*, 01 2008.
- [74] Peter-Pike Sloan, Jesse Hall, John Hart, and John Snyder. Clustered principal components for precomputed radiance transfer. *ACM Trans. Graph.*, 22(3):382–391, July 2003. doi:10.1145/882262.882281.
- [75] Peter-Pike Sloan, Jan Kautz, and John Snyder. Precomputed radiance transfer for real-time rendering in dynamic, low-frequency lighting environments. *ACM Trans. Graph.*, 21(3):527–536, July 2002. doi:10.1145/566654.566612.
- [76] Matthew Tancik, Pratul P. Srinivasan, Ben Mildenhall, Sara Fridovich-Keil, Nithin Raghavan, Utkarsh Singhal, Ravi Ramamoorthi, Jonathan T. Barron, and Ren Ng. Fourier features let networks learn high frequency functions in low dimensional domains. In *Proceedings of the 34th International Conference on Neural Information Processing Systems, NIPS ’20*, Red Hook, NY, USA, 2020. Curran Associates Inc.
- [77] Unity Technologies. Inference engine (formerly Sentis). <https://docs.unity3d.com/Packages/com.unity.ai.inference@latest>, 2025. Accessed: 2025-02-05.
- [78] Unity Technologies. Unity barracuda: Neural network inference library for unity. <https://docs.unity3d.com/Packages/com.unity.barracuda@latest>, 2025. Accessed: 2025-01-21.
- [79] Daniel Thul, Vagia Tsiminaki, Lubor Ladický, and Marc Pollefeys. Pre-computed Radiance Transfer for Reflectance and Lighting Estimation. In *2020 International Conference on 3D Vision (3DV)*, pages 1147–1156, 2020. doi:10.1109/3DV50981.2020.00125.

- [80] Xuechang Tu, Lukas Radl, Michael Steiner, Markus Steinberger, Bernhard Kerbl, and Fernando de la Torre. VRSplat: Fast and Robust Gaussian Splatting for Virtual Reality. *Proc. ACM Comput. Graph. Interact. Tech.*, 8(1), May 2025. doi:10.1145/3728311.
- [81] Unity Technologies. Unity, 2025. Accessed: 2025-03-03. URL: <https://unity.com/>.
- [82] GA Net Updates. ENGAGE at CGI 2025 GA Computing Award (1st place): Efstratios Geronikolakis. <https://gaupdate.wordpress.com/2025/07/23/engage-at-cgi-2025-ga-comp-award-1st-place-efstratios-geronikolakis/>, 2025. Public blog announcement of the certificate.
- [83] C.J. Van Rijsbergen. *Information Retrieval*. Butterworths, 1979. URL: <https://books.google.gr/books?id=t-pTAAAAAAAJ>.
- [84] Eric Veach and Leonidas J. Guibas. Metropolis light transport. In *Proceedings of the 24th Annual Conference on Computer Graphics and Interactive Techniques*, SIGGRAPH '97, page 65–76, USA, 1997. ACM Press/Addison-Wesley Publishing Co. doi:10.1145/258734.258775.
- [85] Petar Veličković, Guillem Cucurull, Arantxa Casanova, Adriana Romero, Pietro Liò, and Yoshua Bengio. Graph Attention Networks. In *International Conference on Learning Representations*, 2018. URL: <https://openreview.net/forum?id=rJXmpikCZ>.
- [86] Pauli Virtanen, Ralf Gommers, Travis E. Oliphant, Matt Haberland, Tyler Reddy, David Cournapeau, Evgeni Burovski, Pearu Peterson, Warren Weckesser, Jonathan Bright, Stéfan J. van der Walt, Matthew Brett, Joshua Wilson, K. Jarrod Millman, Nikolay Mayorov, Andrew R. J. Nelson, Eric Jones, Robert Kern, Eric Larson, C. J. Carey, İlhan Polat, Yu Feng, Eric W. Moore, Jake VanderPlas, Denis Laxalde, Josef Perktold, Robert Cimrman, Ian Henriksen, E. A. Quintero, Charles R. Harris, Anne M. Archibald, Antônio H. Ribeiro, Fabian Pedregosa, Paul van Mulbregt, Aditya Vijaykumar, Alessandro Pietro Bardelli, Alex Rothberg, Andreas Hilboll, Andreas Klöckner, Anthony Scopatz, Antony Lee, Ariel Rokem, C. Nathan Woods, Chad Fulton, Charles Masson, Christian Häggström, Clark Fitzgerald, David A. Nicholson, David R. Hagen, Dmitrii V. Pasechnik, Emanuele Olivetti, Eric Martin, Eric Wieser, Fabrice Silva, Felix Lenders, Florian Wilhelm, G. Young, Gavin A. Price, Gert-Ludwig Ingold, Gregory E. Allen, Gregory R. Lee, Hervé Audren, Irvin Probst, Jörg P. Dietrich, Jacob Silterra, James T. Webber, Janko Slavič, Joel Nothman, Johannes Buchner, Jo-

- hannes Kulick, Johannes L. Schönberger, José Vinícius de Miranda Cardoso, Joscha Reimer, Joseph Harrington, Juan Luis Cano Rodríguez, Juan Nunez-Iglesias, Justin Kuczynski, Kevin Tritz, Martin Thoma, Matthew Newville, Matthias Kümmerer, Maximilian Bolingbroke, Michael Tartre, Mikhail Pak, Nathaniel J. Smith, Nikolai Nowaczyk, Nikolay Shebanov, Oleksandr Pavlyk, Per A. Brodtkorb, Perry Lee, Robert T. McGibbon, Roman Feldbauer, Sam Lewis, Sam Tygier, Scott Sievert, Sebastiano Vigna, Stefan Peterson, Surhud More, Tadeusz Pudlik, Takuya Oshima, Thomas J. Pingel, Thomas P. Robitaille, Thomas Spura, Thouis R. Jones, Tim Cera, Tim Leslie, Tiziano Zito, Tom Krauss, Utkarsh Upadhyay, Yaroslav O. Halchenko, Yoshiki Vázquez-Baeza, and SciPy 1.0 Contributors. SciPy 1.0: Fundamental Algorithms for Scientific Computing in Python. *Nature Methods*, 17(3):261–272, 2020. doi:10.1038/s41592-019-0686-2.
- [87] Linning Xu, Vasu Agrawal, William Laney, Tony Garcia, Aayush Bansal, Changil Kim, Samuel Rota Bulò, Lorenzo Porzi, Peter Kotschieder, Aljaž Božič, Dahua Lin, Michael Zollhöfer, and Christian Richardt. VR-NeRF: High-Fidelity Virtualized Walkable Spaces. In *SIGGRAPH Asia 2023 Conference Papers*, SA '23, New York, NY, USA, 2023. Association for Computing Machinery. doi:10.1145/3610548.3618139.
- [88] Mingyuan Yao, Yukang Huo, Yang Ran, Qingbin Tian, Ruifeng Wang, and Haihua Wang. Neural Radiance Field-based Visual Rendering: A Comprehensive Review, 2024. URL: <https://arxiv.org/abs/2404.00714>, arXiv:2404.00714.
- [89] Vickie Ye, Ruilong Li, Justin Kerr, Matias Turkulainen, Brent Yi, Zhuoyang Pan, Otto Seiskari, Jianbo Ye, Jeffrey Hu, Matthew Tancik, et al. GSplat: An open-source library for Gaussian Splatting. *Journal of Machine Learning Research*, 26(34):1–17, 2025.
- [90] Alex Yu, Ruilong Li, Matthew Tancik, Hao Li, Ren Ng, and Angjoo Kanazawa. PlenOctrees for Real-time Rendering of Neural Radiance Fields. In *2021 IEEE/CVF International Conference on Computer Vision (ICCV)*, pages 5732–5741, 2021. doi:10.1109/ICCV48922.2021.00570.
- [91] Annan Yu, Yunan Yang, and Alex Townsend. Tuning Frequency Bias in Neural Network Training with Nonuniform Data. In *International Conference on Learning Representations*, 2023. URL: <https://api.semanticscholar.org/CorpusID:252531626>.

- [92] Libo Zhang, Yuxuan Han, Wenbin Lin, Jingwang Ling, and Feng Xu. PRT-Gaussian: Efficient Relighting Using 3D Gaussians with Precomputed Radiance Transfer. In *2024 Asia Pacific Signal and Information Processing Association Annual Summit and Conference (APSIPA ASC)*, pages 1–6. IEEE, 2024.
- [93] Paul Zikas, Manos Kamarianakis, Ioanna Kartsonaki, Nick Lydatakis, Steve Kateros, Mike Kentros, Efstratios Geronikolakis, Giannis Evangelou, Achilles Apostolou, Paolo Alejandro Alejandro Catilo, and George Papagiannakis. Covid-19 - vr strikes back: innovative medical vr training. In *ACM SIGGRAPH 2021 Immersive Pavilion*, SIGGRAPH '21, New York, NY, USA, 2021. Association for Computing Machinery. doi:10.1145/3450615.3464546.
- [94] Paul Zikas, Steve Kateros, Nick Lydatakis, Mike Kentros, Efstratios Geronikolakis, Manos Kamarianakis, Giannis Evangelou, Ioanna Kartsonaki, Achilles Apostolou, Tanja Birrenbach, et al. Virtual reality medical training for covid-19 swab testing and proper handling of personal protective equipment: Development and usability. *Frontiers in virtual reality*, 2:740197, 2022.
- [95] Paul Zikas, Antonis Protopsaltis, Nick Lydatakis, Mike Kentros, Stratos Geronikolakis, Steve Kateros, Manos Kamarianakis, Giannis Evangelou, Achilleas Filippidis, Eleni Grigoriou, Dimitris Angelis, Michail Tamiolakis, Michael Dodis, George Kokiadis, John Petropoulos, Maria Pateraki, and George Papagiannakis. MAGES 4.0: Accelerating the World’s Transition to VR Training and Democratizing the Authoring of the Medical Metaverse. *IEEE Comput. Graph. Appl.*, 43(2):43–56, March 2023. doi:10.1109/MCG.2023.3242686.

A Appendix

A.1 Pythonic PRT basic algorithm implementation

The PRT algorithm is predominantly implemented in C++, with the majority of its references and existing implementations found in this language. There has also been an attempt to integrate PRT directly within the Unity [81] game engine [1]; however, no further information regarding this effort is available. To facilitate the integration of deep learning methods into the algorithm, a Python implementation was required.

To the best of our knowledge, no prior implementation of the PRT algorithm in Python has been publicly documented or made available. A comprehensive search was conducted across multiple sources, including academic literature databases (IEEE Xplore, ACM Digital Library, arXiv), open-source code repositories (GitHub, GitLab, Bitbucket), and relevant technical forums. Despite this extensive investigation, no fully functional Python-based PRT implementation was found. Existing implementations are predominantly written in C++ and integrated with real-time graphics engines such as DirectX or Unity. Given the absence of a Python version, this work presents the first documented implementation of PRT in Python, enabling easier integration with deep learning frameworks and extending the accessibility of PRT research. Consequently, we identified a reliable version of the PRT algorithm, written in C++ [9] and based in the original implementation of Sloan et. al [75], successfully executed it, and proceeded with porting it to Python. Our final Python codebase is available in Appendix B.1.

A.2 Platform

To implement and extend the PRT algorithm, an OpenGL-based graphics framework was required to support real-time rendering, shader integration, and visualization of light transport results. OpenGL, as a widely used cross-platform graphics API, provides the low-level functionality necessary for rendering interactive 2D and 3D graphics [44]. Among the available Python-compatible frameworks, *pyGANDALF* [60] was selected as the development platform due to its modular design, extensibility, and compatibility with modern compute backends.

The *pyGANDALF* framework combines Python’s flexibility with an Entity Component System (ECS) architecture, offering both OpenGL and WebGPU backends for rendering and computation. This hybrid design ensures broad platform compatibility while providing a foundation for parallelizable tasks such as visibility sampling and SH coefficient projection. Its modular organization separates core functionalities—such as rendering, scene management, and mathe-

mational operations—into dedicated subsystems, allowing targeted development of specialized components without interfering with the core engine.

A key advantage of pyGANDALF is its Pythonic extensibility. For the purposes of this thesis, a dedicated PRT module was developed within the framework, responsible for all radiance transfer computations, including SH projection, visibility sampling, and coefficient generation. This integration allowed seamless experimentation with both classical and neural PRT variants while preserving a consistent and reproducible workflow. The framework’s architecture also facilitated the addition of neural extensions, enabling MLP and GNN predictors to operate directly on scene entities.

In addition to its modularity, pyGANDALF includes a built-in scene editor resembling those found in modern game engines such as Unity and Unreal. As illustrated in Figure 39, this editor supports real-time visualization of meshes, materials, and lighting, making the framework particularly well-suited for developing interactive prototypes. The combination of real-time rendering, ECS-based data organization, and neural extensibility makes pyGANDALF an ideal testbed for bridging traditional physically based rendering with neural global illumination methods.

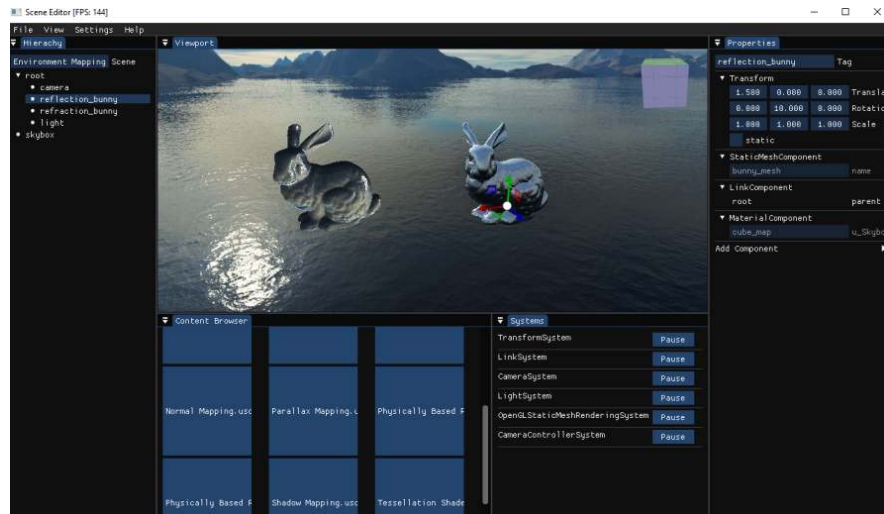


Figure 39: The pyGANDALF scene editor, providing an engine-like interface for real-time visualization and prototyping. [60]

A.3 Methodology

For the implementation of the PRT algorithm in pyGANDALF, we utilized one of the existing examples of the platform, specifically an introductory demonstration featuring only a simple cube and its coordinate axes. Recognizing this modular architecture, we determined that the most efficient approach was to develop a

new library within pyGANDALF, specifically designed to encapsulate all components necessary for PRT computations. This dedicated module would serve as an integrated extension of the framework, ensuring a structured and scalable implementation of radiance transfer techniques.

It is important to clarify that in this implementation, OpenGL was exclusively utilized for rendering and computation. WebGPU was not employed in the development of the PRT algorithm. However, given WebGPU’s capabilities for parallel processing and modern rendering optimizations, exploring its integration with PRT remains an interesting direction for future research.

A.3.1 3D Mesh Loading

To ensure high-quality and visually meaningful results from the PRT algorithm, it was necessary to load complex 3D models. While PRT can operate on simple geometric shapes, such as cubes, its effectiveness becomes significantly more apparent when applied to highly detailed models with intricate surfaces and varying normal orientations. Fortunately, pyGANDALF provides built-in support for loading 3D models, primarily in the Wavefront OBJ (.obj) format, which we also selected as our primary file format for compatibility and consistency.

Although pyGANDALF’s built-in model loader is efficient, we determined that developing a custom model loading system would offer greater flexibility and reliability. This decision was motivated by several factors. First, we sought to ensure full compatibility with all Wavefront OBJ models, as our experience indicated that certain models, despite being loadable, often contained hidden inconsistencies related to normals, face definitions, or, in rare cases, vertex data containing NaN values. While such issues may not necessarily affect standard rendering, they are particularly problematic for PRT computations, where accurate normal vectors are essential for light transport calculations. To address this, our custom loader automatically detects and corrects defective normals, recalculating them based on the model’s geometry.

Additionally, our custom model loader was designed to support multiple file formats beyond OBJ, ensuring broader compatibility with different 3D modeling workflows. In particular, we introduced support for Collada (.dae) files, allowing for greater flexibility in model selection and testing within the PRT framework.

To achieve these objectives, we developed the `PRTLoader` class (1), specifically designed for PRT preprocessing and computation. Its core functionality is encapsulated in the `load_model` method, which loads the target .obj and saves its vertices, normals and indices, which are of utmost importance for the algorithm. It is developed in a way that the loader can be easily extended to support additional mesh formats in the future, making it a scalable and adaptable solution

for PRT-based rendering applications.

A.3.2 High-Dynamic-Range (HDR) Skybox

A High Dynamic Range (HDR) image is essential for the correct operation of PRT because it accurately represents the full range of light intensities in a scene, from the darkest shadows to the brightest highlights. PRT relies on SH projections to approximate global illumination, and these computations require physically accurate lighting information to produce realistic results. Standard Low Dynamic Range (LDR) images lack sufficient precision, often clamping or compressing high-intensity values, which leads to loss of crucial lighting details necessary for indirect illumination and realistic shading. HDR images, on the other hand, preserve the entire luminance range, enabling PRT to correctly capture subtle lighting variations, smooth transitions in indirect light, and accurate reflections. This is particularly important when simulating soft shadows, interreflections, and diffuse global illumination, as any loss of dynamic range would introduce visual artifacts or incorrect light transport calculations. By ensuring an HDR environment map, PRT can properly reconstruct complex lighting interactions, leading to a more realistic and physically accurate rendering of the scene. A comparison between a LDR and a HDR image is illustrated in Figure 40.



Figure 40: An example of a LDR image (left) and a HDR image (right). In the HDR image, the snow keeps its texture, the trees in shadow are still visible, and both bright and dark regions have detail. [30]

To address this requirement, we performed two specific tasks. First, we developed a method to extract the necessary information from the given HDR image, specifically the *light-transfer* function values required for PRT computations. Second, we implemented a way to convert an HDR image into a skybox for use in pyGANDALF. Since the HDR images were in cross-HDRI format, we devised a technique to accurately extract the individual cubemap faces and render them as a cube around the camera. We also took special precautions to handle cases

where the cross-HDR image was formatted either horizontally or vertically, ensuring proper alignment and seamless integration into the rendering system.

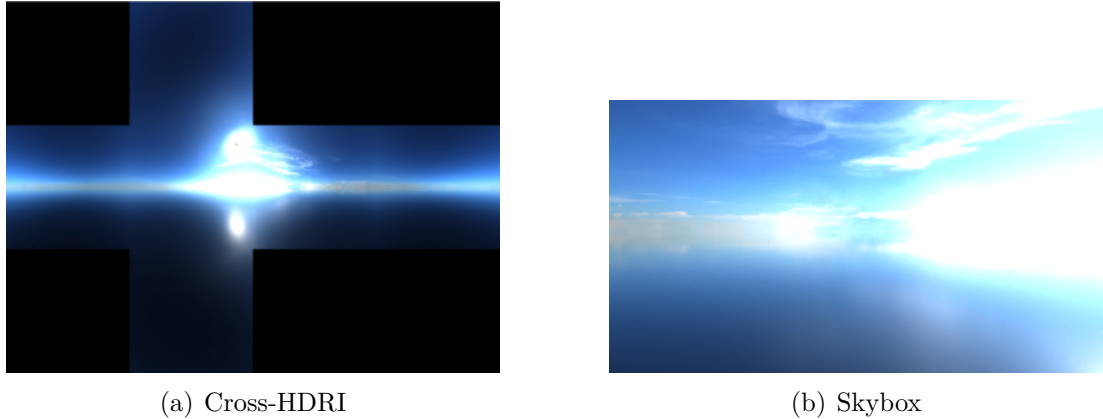


Figure 41: A cross-HDRI image (a) and the skybox (b) we create based on it. The HDRI is sourced from *Poly Haven* [62].

The process of converting a cross-HDRI image into a skybox follows a structured approach. First, we determine whether the given HDRI image is in a horizontal or vertical format, which is primarily inferred from its dimensions and aspect ratio. Once the orientation is identified, we parse the image row by row, isolating each non-black rectangular region corresponding to a cubemap face. These extracted regions are stored in a structured array, where each of the six elements represents one face of the cube.

After the extraction process, the six individual images are passed to the skybox generation algorithm within pyGANDALF, which ensures that they are correctly mapped and rendered as a coherent environment. When the application runs, the skybox is accurately displayed, providing a seamless background. Figure 42 demonstrates how a cross-HDRI image is divided with this logic.

It is important to emphasize that the assignment of cubemap faces to array indices follows a specific order, rather than being arbitrary. This structured mapping is illustrated in 2, where each array index is explicitly assigned to its corresponding cubemap face. The same methodology is applied when handling alternative orientations of the HDRI image, ensuring consistency regardless of the input format.

A.3.3 Main Implementation

For the core implementation, we followed the fundamental principles of PRT [75], as well as the existing C++ implementation of [9]. The Python PRT pipeline is illustrated in Figure 43.

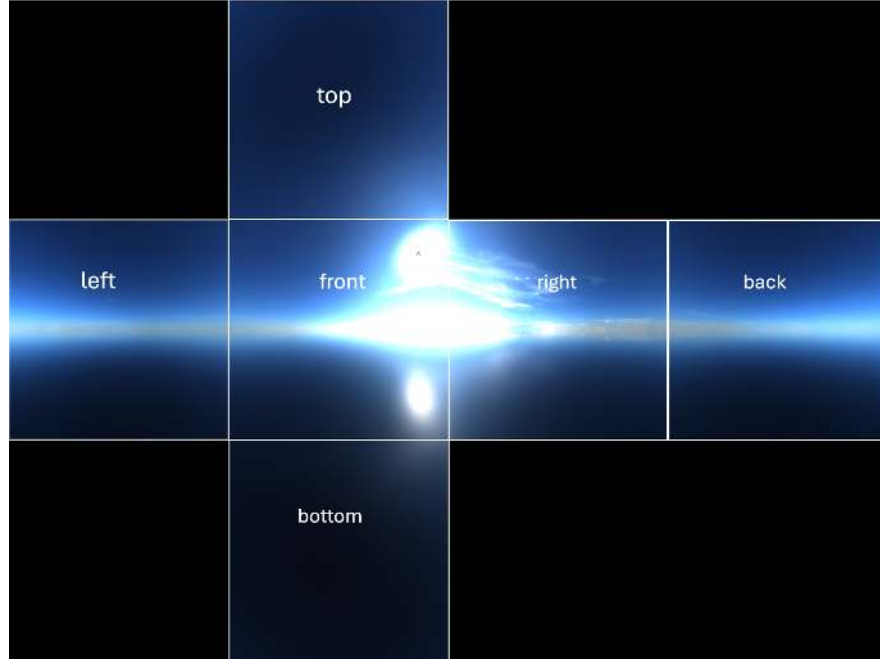


Figure 42: A cross-HDRI image with its sections marked. The HDRI is sourced from *Poly Haven* [62].

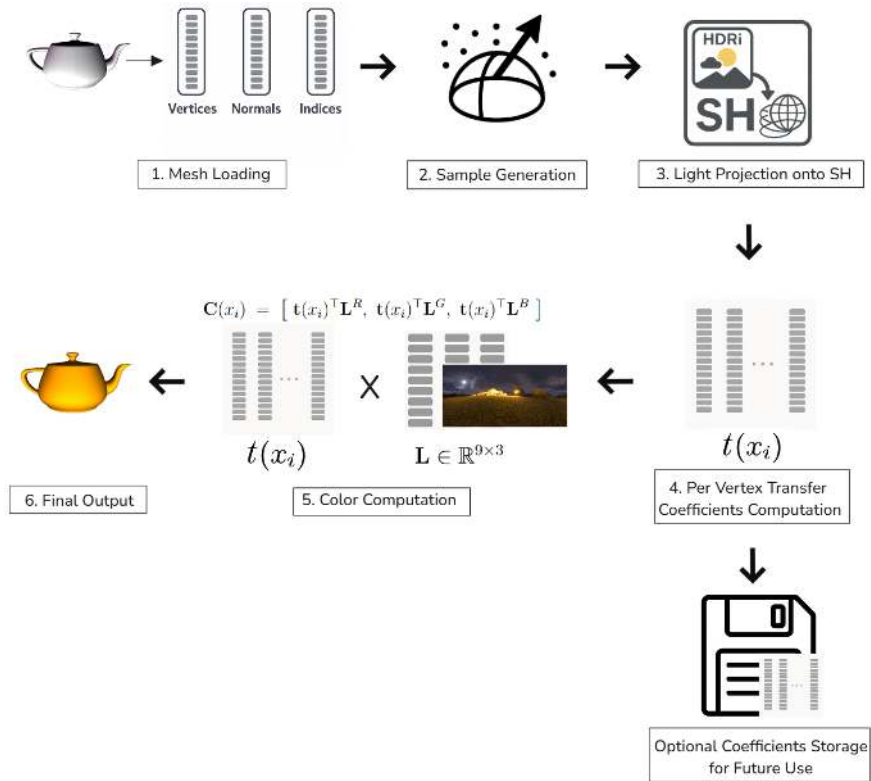


Figure 43: The main pipeline of our PRT implementation in Python.

Mesh Loading. Triangle meshes are loaded from `.obj` files. Vertex positions and per-vertex normals are parsed; polygonal faces are triangulated as needed. When normals are missing, they are estimated by accumulating ad-

jacent face normals and normalizing per vertex. The loader consolidates unique (position, normal) pairs, yielding dense arrays of vertices $\mathbf{v}_i \in \mathbb{R}^3$, normals $\mathbf{n}_i \in \mathbb{R}^3$, and triangle indices. Illumination is provided by a cross-layout HDR environment map, which is read once, split into cube faces, and sampled during light projection.

Sample Generation. We adopt diffuse PRT with three SH bands ($L = 2$). In SH theory, the band index L defines the maximum degree of the expansion. Each band $\ell = 0, \dots, L$ contains $(2\ell + 1)$ basis functions, giving a total of $B = L^2$ coefficients up to order L [75]. Thus, for $L = 2$, we obtain $B = 9$ basis functions $\{Y_l\}_{l=1}^9$. This choice is standard in diffuse PRT, as it captures low-frequency lighting effects with a compact representation while avoiding the computational overhead of higher-order expansions. For a specified resolution N , the hemisphere is divided into an $N \times N$ grid of cells. Within each cell, a sample is jittered by adding a small random offset, ensuring both uniform coverage and stochastic variation. Two stratified random variables $(a, b) \in [0, 1]^2$ are mapped to spherical coordinates via

$$\theta = 2 \arccos(\sqrt{1-a}), \quad \phi = 2\pi b,$$

which guarantees cosine-weighted sampling of the hemisphere. These angles are converted into Cartesian directions

$$\omega = (\sin \theta \cos \phi, \sin \theta \sin \phi, \cos \theta),$$

which are then used to evaluate SH basis functions and clamped cosine terms during transfer computation. This strategy balances uniformity and randomness, producing stable results while reducing variance compared to purely random sampling. SH values $Y_l(\omega_j)$ are precomputed for all samples.

Light Projection onto SH. The RGB environment $L(\omega) \in \mathbb{R}^3$ is projected to SH coefficients $\mathbf{L} \in \mathbb{R}^{9 \times 3}$ via

$$\mathbf{L}[l, :] \approx \frac{4\pi}{N} \sum_{j=1}^N L(\omega_j) Y_l(\omega_j), \quad l = 1, \dots, 9,$$

with the factor $4\pi/N$ accounting for uniform spherical integration.

Per Vertex Transfer Coefficients Computation.

Unshadowed transfer. For each vertex x_i , a transfer vector $\mathbf{t}(x_i) \in \mathbb{R}^9$ (clamped cosine only) is computed by summing over all sampled directions:

$$\mathbf{t}(x_i)[l] = \frac{4\pi}{N} \sum_{j=1}^N \max(\mathbf{n}_i \cdot \omega_j, 0) Y_l(\omega_j), \quad l = 1, \dots, 9.$$

Shadowed transfer. Self-occlusion is incorporated by testing visibility for each sample. Let $V(x_i, \omega_j) \in \{0, 1\}$ be the result of a ray-cast from x_i along ω_j (Möller–Trumbore on GPU), skipping triangles incident to x_i . The visibility-aware transfer becomes

$$\mathbf{t}(x_i)[l] = \frac{4\pi}{N} \sum_{j=1}^N V(x_i, \omega_j) \max(\mathbf{n}_i \cdot \omega_j, 0) Y_l(\omega_j), \quad l = 1, \dots, 9.$$

Color Computation. At runtime the RGB irradiance is reconstructed by reusing the same transfer across channels,

$$\mathbf{C}(x_i) = \begin{bmatrix} \mathbf{t}(x_i)^\top \mathbf{L}^R, & \mathbf{t}(x_i)^\top \mathbf{L}^G, & \mathbf{t}(x_i)^\top \mathbf{L}^B \end{bmatrix},$$

optionally modulated by albedo. Updating the HDRI only requires recomputing $\mathbf{L}$.

Practical notes. SH basis values for all samples are cached. Environment projection and transfer accumulation share the same sampling and normalization for energy consistency. The visibility kernel parallelizes ray–triangle tests on the GPU and early-outs on the first hit. For interactive use, lighting coefficients $\mathbf{L}$ are recomputed only when the HDRI changes, while per-vertex transfers can be cached to disk and reloaded to avoid recomputation.

A.3.4 Implementation Variants

Building on the baseline formulation, two variants of the algorithm were implemented: unshadowed and shadowed PRT. The unshadowed case serves as the starting point, since it is conceptually simpler and avoids the costly visibility evaluation. In this setting, transfer coefficients are accumulated using only the clamped cosine term, providing a clear baseline for verifying correctness and efficiency. During execution, a *progress* variable was added to report computation status, which is particularly useful for large-scale models where precomputation may take several minutes to hours.

The shadowed variant extends this baseline by incorporating self-occlusion tests for each sample direction. For every vertex and sampled direction, a ray–triangle intersection test is performed against the mesh to determine whether



Figure 44: Comparison of PRT variants on the monkey mesh. (a) Unshadowed transfer, where illumination is accumulated without accounting for occlusion. (b) Shadowed transfer, where ray-triangle visibility tests capture self-occlusion, producing darker regions in crevices and shadowed areas. Red boxes highlight regions with notable differences due to shadow integration.

the direction is occluded. Triangles incident to the vertex are skipped to avoid self-intersection artifacts. Visibility tests are parallelized using CUDA, enabling thousands of rays to be traced simultaneously and significantly reducing runtime compared to a sequential CPU implementation. Specifically, for each fixed vertex x_i and sampled direction ω_j , a kernel is launched where each thread is responsible for testing a single triangle against the ray (x_i, ω_j) via the Möller-Trumbore intersection algorithm [52]. Triangle indices are stored in a contiguous `int32` array and vertex positions in a `float32` array organized in a structure-of-arrays layout, ensuring coalesced global memory accesses when threads within a warp access adjacent triangles. Triangles incident to the query vertex are skipped to avoid self-intersections. Each thread writes a hit flag into a device buffer; after kernel execution, a reduction over this buffer determines the binary visibility $V(x_i, \omega_j)$. To further optimize performance, results can be cached to disk as precomputed transfer coefficients and reloaded when needed, thereby avoiding redundant recomputation.

This distinction between the unshadowed and shadowed implementations highlights the trade-off between accuracy and computational cost. While the unshadowed version is efficient but lacks occlusion effects, the shadowed version produces more realistic shading at the expense of increased precomputation time. Example results for both cases are shown in Figure 44.

B Appendix

B.1 Traditional PRT in Python

```

1 class PRTLoader:
2     """
3     Class responsible for loading 3D models in .obj format for processing by the PRT algorithm.
4     """
5
6     def __init__(self):
7         self.vertices = []
8         self.faces = []
9         self.normals = []
10        self.texcoords = []
11        self.joints = {}
12        self.weights = []
13        self.animations = {}
14
15    def load_model(self, filename, auto_calculate_normals=False):
16        """
17        Load a 3D model from a file (.obj).
18        """
19        # Ensure filename is a string
20        if not isinstance(filename, str):
21            filename = str(filename)
22
23        file_extension = filename.split('.')[-1].lower()
24        if file_extension == 'obj':
25            self._load_obj(filename)
26        else:
27            raise ValueError(f"Unsupported file format: {file_extension}")
28
29    def _load_obj(self, filename):
30        """
31        Load an OBJ file.
32        """
33        # Regex for positions, normals, and faces
34        vertex_re = re.compile(r'^v\s+([\d\.\-eE]+\s+)([\d\.\-eE]+\s+)([\d\.\-eE]+\s+)')
35        normal_re = re.compile(r'^vn\s+([\d\.\-eE]+\s+)([\d\.\-eE]+\s+)([\d\.\-eE]+\s+)')
36        face_re = re.compile(r'^f\s+(.+)'')
37
38        # Raw data read from the OBJ file (temporary lists)
39        raw_vertices = []
40        raw_normals = []
41
42        # Processed data that will be stored in the final arrays
43        final_vertices = []
44        final_normals = []
45        faces = []
46
47        # A dictionary to map unique (vertex_idx, normal_idx) pairs to a new final index.
48        vertex_map = {}
49
50        with open(filename, 'r') as file:
51            for line in file:
52                line = line.strip()
53                if not line or line.startswith('#'):
54                    continue
55
56                # Match vertex positions and add to raw list
57                vertex_match = vertex_re.match(line)
58                if vertex_match:
59                    raw_vertices.append([float(v) for v in vertex_match.groups()])
60                    continue
61
62                # Match vertex normals and add to raw list
63                normal_match = normal_re.match(line)
64                if normal_match:
65                    raw_normals.append([float(n) for n in normal_match.groups()])
66                    continue
67
68                # Match faces
69                face_match = face_re.match(line)
70                if face_match:
71                    face_corner_strings = face_match.group(1).split()
72                    current_face_final_indices = []
73
74                    # Check if this face has normals defined in the v/vt/vn or v//vn format.

```

```

75         has_normals = ('/' in face_corner_strings[0] or (face_corner_strings[0].count('/') ==
76                     2))
77
78         if not has_normals or not raw_normals:
79             # Fallback for simple faces without normals (e.g., "f 1 2 3")
80             face_v_indices = [int(p.split('/')[0]) - 1 for p in face_corner_strings]
81             if len(face_v_indices) > 3: # Triangulate if it's a quad or polygon
82                 for i in range(1, len(face_v_indices) - 1):
83                     faces.append([face_v_indices[0], face_v_indices[i], face_v_indices[i+1]])
84             else:
85                 faces.append(face_v_indices)
86             continue
87
88         # Process faces that have normals
89         for corner_str in face_corner_strings:
90             indices = corner_str.split('/')
91             v_idx = int(indices[0]) - 1
92             n_idx = int(indices[-1]) - 1 # Normal index is always the last one
93
94             vertex_key = (v_idx, n_idx)
95
96             if vertex_key in vertex_map:
97                 final_idx = vertex_map[vertex_key]
98             else:
99                 final_idx = len(final_vertices)
100                 vertex_map[vertex_key] = final_idx
101                 final_vertices.append(raw_vertices[v_idx])
102                 final_normals.append(raw_normals[n_idx])
103
104             current_face_final_indices.append(final_idx)
105
106         # Triangulate if the face is a polygon
107         if len(current_face_final_indices) > 3:
108             for i in range(1, len(current_face_final_indices) - 1):
109                 faces.append([
110                     current_face_final_indices[0],
111                     current_face_final_indices[i],
112                     current_face_final_indices[i + 1]
113                 ])
114             else:
115                 faces.append(current_face_final_indices)
116
117         if raw_normals:
118             self.vertices = np.array(final_vertices, dtype=np.float32)
119             self.normals = np.array(final_normals, dtype=np.float32)
120             self.faces = np.array(faces, dtype=np.int32)
121
122         elif not raw_normals and raw_vertices and faces:
123             np_vertices = np.array(raw_vertices, dtype=np.float32)
124             np_faces = np.array(faces, dtype=np.int32)
125
126             computed_normals = np.zeros_like(np_vertices)
127
128             v0 = np_vertices[np_faces[:, 0]]
129             v1 = np_vertices[np_faces[:, 1]]
130             v2 = np_vertices[np_faces[:, 2]]
131
132             face_normals = np.cross(v1 - v0, v2 - v0)
133
134             np.add.at(computed_normals, np_faces[:, 0], face_normals)
135             np.add.at(computed_normals, np_faces[:, 1], face_normals)
136             np.add.at(computed_normals, np_faces[:, 2], face_normals)
137
138             norms = np.linalg.norm(computed_normals, axis=1, keepdims=True)
139
140             np.divide(computed_normals, norms, out=computed_normals, where=norms != 0)
141
142             self.vertices = np_vertices
143             self.normals = computed_normals
144             self.faces = np_faces
145
146         else:
147             self.vertices = np.array(raw_vertices, dtype=np.float32)
148             self.normals = np.array([], dtype=np.float32)
149             self.faces = np.array(faces, dtype=np.int32)
150
151         def get_vertices(self):
152             return self.vertices
153
154         def get_faces(self):

```

```

154         return self.faces
155
156     def get_normals(self):
157         return self.normals
158
159     def get_texcoords(self):
160         return np.array(self.texcoords, dtype='float32')
161
162     def get_joints(self):
163         return self.joints
164
165     def get_weights(self):
166         return self.weights
167
168     def get_animations(self):
169         return self.animations
170
171
172     def save_vertices_and_normals(self, filename):
173         """
174         Save vertices and normals to separate text files.
175         """
176         # Create a folder to save the data if it doesn't exist
177         output_folder = "."
178         if not os.path.exists(output_folder):
179             os.makedirs(output_folder)
180
181         # Save vertices to file
182         if not os.path.exists(os.path.join(output_folder, str(filename).split('/')[1].replace('.obj', '')
183                                           + "_vertices.txt")):
184             vertex_file = os.path.join(output_folder, str(filename).split('/')[1].replace('.obj', '') + "_vertices.txt")
185             np.savetxt(vertex_file, self.get_vertices(), delimiter=' ')
186             print(f"Saved vertices to {vertex_file}")
187
188         # Save normals to file
189         if len(self.normals) > 0:
190             normal_file = os.path.join(output_folder, str(filename).split('/')[1].replace('.obj', '')
191                                       + "_normals.txt")
192             np.savetxt(normal_file, self.get_normals(), delimiter=' ')
193             print(f"Saved normals to {normal_file}")
194         else:
195             print("No normals to save.")

```

Listing 1: The PRTLoader class responsible for loading a mesh to be processed by the PRT algorithm.

```

1 class HDRIFrame:
2     """
3     Class responsible for loading an HDR/Cross-HDR image.
4     """
5     def __init__(self):
6         """
7         Initializes the HDRIFrame with default dimensions and an empty cubemap.
8         """
9         self.dimx = 0
10        self.dimy = 0
11        self.cubemap = [None] * 6
12
13    def read_hdr(self, filename):
14        """
15        Reads an HDR image file and loads it into the class.
16
17        Args:
18            filename (str): The path to the HDR image file to be loaded.
19
20        Returns:
21            hdrResult: An object containing the loaded HDR image data.
22        """
23        self.hdrResult = HDRLoader.load(filename)
24        self.dimx = self.hdrResult.width
25        self.dimy = self.hdrResult.height
26
27        return self.hdrResult
28
29    def create_cube_map_faces(self, hdrResult):
30        """
31        Creates cubemap faces from the loaded HDR image.

```

```

32
33     Args:
34         hdrResult: The HDR image data from which to create the cubemap faces.
35     """
36     if(self.dimx < self.dimy):
37         h = self.dimx // 3
38         w = self.dimy // 4
39
40         for i in range(6):
41             self.cubemap[i] = np.zeros((h, w, 3), dtype=np.uint8)
42
43             self.cubemap[2][:,:,] = hdrResult.cols[0:h, w:2*w, :]
44             self.cubemap[3][:,:,] = hdrResult.cols[2*h:3*h, w:2*w, :]
45             self.cubemap[0][:,:,] = hdrResult.cols[h:2*h, 2*w:3*w, :]
46             self.cubemap[1][:,:,] = hdrResult.cols[h:2*h, 0:w, :]
47             self.cubemap[4][:,:,] = hdrResult.cols[h:2*h, w:2*w, :]
48             self.cubemap[5][:,:,] = hdrResult.cols[3*h:4*h, w:2*w, :][::-1, :-1, :]
49
50     else:
51         isRightFaced = False
52         w = self.dimx // 4
53         h = self.dimy // 3
54
55         for i in range(6):
56             self.cubemap[i] = np.zeros((h, w, 3), dtype=np.uint8)
57
58             self.cubemap[2][:,:,] = hdrResult.cols[0:h, 2*w:3*w, :]
59
60         for i in range(h):
61             for j in range(w):
62                 for k in range(3):
63                     if(self.cubemap[2][i,j,k] != 0):
64                         isRightFaced = True
65                         break
66
67         if(isRightFaced):
68             self.cubemap[3][:,:,] = hdrResult.cols[2*h:3*h, 2*w:3*w, :]
69             self.cubemap[0][:,:,] = hdrResult.cols[h:2*h, 3*w:4*w, :]
70             self.cubemap[1][:,:,] = hdrResult.cols[h:2*h, w:2*w, :]
71             self.cubemap[4][:,:,] = hdrResult.cols[h:2*h, 2*w:3*w, :]
72             self.cubemap[5][:,:,] = hdrResult.cols[h:2*h, 0:w, :][::-1, :-1, :]
73         else:
74             self.cubemap[2][:,:,] = hdrResult.cols[0:h, w:2*w, :]
75             self.cubemap[3][:,:,] = hdrResult.cols[2*h:3*h, w:2*w, :]
76             self.cubemap[0][:,:,] = hdrResult.cols[h:2*h, 2*w:3*w, :]
77             self.cubemap[1][:,:,] = hdrResult.cols[h:2*h, 0:w, :]
78             self.cubemap[4][:,:,] = hdrResult.cols[h:2*h, w:2*w, :]
79             self.cubemap[5][:,:,] = hdrResult.cols[h:2*h, 3*w:4*w, :][::-1, :-1, :]
80
81
82
83 class HDRLoader:
84     """
85     Class responsible for loading an HDR/Cross-HDR image.
86     """
87     @staticmethod
88     def load(file_name):
89         """
90         Loads an HDR image from the specified file.
91
92         Args:
93             file_name (str): The path to the HDR image file to be loaded.
94
95         Returns:
96             HDRLoaderResult: An object containing the loaded HDR image data.
97
98         Raises:
99             SystemExit: If the image cannot be loaded or is not in the cross HDRI format.
100     """
101     result = imageio.imread(file_name)
102
103     if result is None:
104         print("Cross HDRI image not found.")
105         exit(0)
106
107     x, y, n = result.shape
108
109     if x == y:
110         print("Image must be cross HDRI format.")
111         exit(0)

```

```

112         width = y
113         height = x
114
115         res = HDRLoaderResult(width, height, result)
116
117         print("\nLoading cross HDRI successful.\n")
118
119         return res
120
121     class HDRLoaderResult:
122         """
123         Class to hold the results of loading an HDR image.
124         """
125         def __init__(self, width, height, cols):
126             """
127             Initializes the HDRLoaderResult with the given dimensions and image data.
128
129             Args:
130                 width (int): The width of the loaded HDR image.
131                 height (int): The height of the loaded HDR image.
132                 cols (ndarray): The image data of the loaded HDR image.
133             """
134             self.width = width
135             self.height = height
136             self.cols = cols
137

```

Listing 2: Code responsible for loading an HDRI map and creating the surrounding skybox.

```

1 class Sample:
2     """
3     Class responsible for keeping the information of a PRT sample.
4     """
5     def __init__(self):
6         """
7         Initializes the Sample with default spherical and cartesian coordinates,
8         and sets the spherical harmonics functions to None.
9         """
10        self.spherical_coord = (0, 0) # (theta, phi)
11        self.cartesian_coord = (0, 0, 0) # (x, y, z)
12        self.sh_functions = None # Spherical harmonics functions

```

Listing 3: The Sample class responsible for initializing a sample.

```

1 class Sampler:
2     """
3     Class responsible for keeping the information of the PRT sampler.
4     """
5     def __init__(self, N):
6         """
7         Initializes the Sampler with an empty list of samples and sets the number of samples to zero.
8         """
9         self.samples = []
10        self.number_of_samples = 0
11        self.generate_samples(N)
12
13    def generate_samples(self, N):
14        """
15        Generates N x N samples and stores them in the samples list. Each sample's spherical and
16        cartesian coordinates are calculated based on stratified sampling.
17
18        Args:
19            N (int): The square root of the number of samples to generate.
20        """
21        self.samples = [Sample() for _ in range(N * N)]
22        self.number_of_samples = N * N
23
24        for i in range(N):
25            for j in range(N):
26                a = (i + random.random()) / 32767 / N
27                b = (j + random.random()) / 32767 / N
28                theta = 2 * math.acos(math.sqrt(1 - a))
29                phi = 2 * math.pi * b
30                x = math.sin(theta) * math.cos(phi)
31                y = math.sin(theta) * math.sin(phi)
32                z = math.cos(theta)

```

```

33         k = i * N + j
34
35         self.samples[k].spherical_coord = (theta, phi)
36         self.samples[k].cartesian_coord = (x, y, z)
37         self.samples[k].sh_functions = None # Placeholder for spherical harmonics functions
38

```

Listing 4: The Sampler class responsible for keeping the information of the PRT sampler.

```

1 class SphericalHarmonics:
2     """
3     Class responsible for the computation of Spherical Harmonics for the PRT algorithm.
4     """
5     def __init__(self, bands):
6         """
7         Initializes the SphericalHarmonics object with the given number of bands.
8
9         Args:
10             bands (int): The number of spherical harmonics bands to use.
11         """
12         self.bands = bands
13         self.coeefs = None
14         self.image = None
15         self.lightCoeefs = None
16
17     @staticmethod
18     def legendre(l, m, x):
19         """
20         Computes the associated Legendre polynomial  $P_l^m(x)$ .
21
22         Args:
23             l (int): Degree of the polynomial.
24             m (int): Order of the polynomial.
25             x (float): Input value.
26
27         Returns:
28             float: The value of the associated Legendre polynomial.
29         """
30         if l == m + 1:
31             return x * (2 * m + 1) * SphericalHarmonics.legendre(m, m, x)
32         elif l == m:
33             return math.pow(-1, m) * SphericalHarmonics.double_factorial(2 * m - 1) * math.pow((1 - x * x),
34                                                     m / 2)
35         else:
36             return (x * (2 * l - 1) * SphericalHarmonics.legendre(l - 1, m, x) - (l + m - 1) *
37                     SphericalHarmonics.legendre(l - 2, m, x)) / (l - m)
38
39     @staticmethod
40     def double_factorial(n):
41         """
42         Computes the double factorial of n ( $n!!$ ).
43
44         Args:
45             n (int): The input value.
46
47         Returns:
48             int: The double factorial of n.
49         """
50         if n <= 1:
51             return 1
52         else:
53             return n * SphericalHarmonics.double_factorial(n - 2)
54
55     @staticmethod
56     def spherical_harmonic(l, m, theta, phi):
57         """
58         Computes the spherical harmonic  $Y_l^m(\theta, \phi)$ .
59
60         Args:
61             l (int): Degree of the harmonic.
62             m (int): Order of the harmonic.
63             theta (float): Polar angle.
64             phi (float): Azimuthal angle.
65
66         Returns:
67             float: The value of the spherical harmonic.
68         """

```

```

67         if m > 0:
68             result = math.sqrt(2) * SphericalHarmonics.k(l, m) * math.cos(m * phi) * SphericalHarmonics.
                legendre(l, m, math.cos(theta))
69         elif m < 0:
70             result = math.sqrt(2) * SphericalHarmonics.k(l, m) * math.sin(-m * phi) * SphericalHarmonics.
                legendre(l, -m, math.cos(theta))
71         else:
72             result = SphericalHarmonics.k(l, m) * SphericalHarmonics.legendre(l, 0, math.cos(theta))
73
74         return result if abs(result) > 0 else 0.0
75
76     @staticmethod
77     def k(l, m):
78         """
79         Computes the normalization constant for the spherical harmonic.
80
81         Args:
82             l (int): Degree of the harmonic.
83             m (int): Order of the harmonic.
84
85         Returns:
86             float: The normalization constant.
87         """
88         if m == 0:
89             return math.sqrt((2 * l + 1) / (4 * math.pi))
90
91         num = (2 * l + 1) * SphericalHarmonics.factorial(l - abs(m))
92         denom = 4 * math.pi * SphericalHarmonics.factorial(l + abs(m))
93         return math.sqrt(num / denom)
94
95     @staticmethod
96     def factorial(n):
97         """
98         Computes the factorial of n (n!).
99
100        Args:
101            n (int): The input value.
102
103        Returns:
104            int: The factorial of n.
105        """
106        if n <= 1:
107            return 1
108        else:
109            return n * SphericalHarmonics.factorial(n - 1)
110
111     def precompute_sh_functions(self, sampler, bands):
112         """
113         Precomputes the spherical harmonic functions for the given samples and bands.
114
115        Args:
116            sampler (Sampler): The sampler containing the samples.
117            bands (int): The number of bands.
118        """
119        for i in range(len(sampler.samples)):
120            sampler.samples[i].sh_functions = np.zeros(bands * bands, dtype=np.float32)
121            theta, phi = sampler.samples[i].spherical_coord
122            for l in range(bands):
123                for m in range(-l, l + 1):
124                    j = l * (l + 1) + m
125                    sampler.samples[i].sh_functions[j] = SphericalHarmonics.spherical_harmonic(l, m, theta,
                        phi)

```

Listing 5: The SphericalHarmonics class responsible for the computation of Spherical Harmonics for the PRT algorithm..

```

1 def light_probe_access(self, imageName, direction):
2     """
3     Accesses the light probe and retrieves the color for a given direction.
4
5     Args:
6         imageName (str): The name of the image file.
7         direction (tuple): The direction vector.
8
9     Returns:
10        np.ndarray: The color value retrieved from the light probe.
11    """
12    if self.image is None:

```

```

13         self.image = imageio.imread(imageName)
14         width, height, _ = self.image.shape
15
16         d = math.sqrt(direction[0] * direction[0] + direction[1] * direction[1])
17         r = (0.0 if d == 0 else (1.0 / math.pi / 2.0) * math.acos(direction[2]) / d)
18         tex_coord = [0.5 + direction[0] * r, 0.5 + direction[1] * r]
19         pixel_coord = [int(tex_coord[0] * width), int(tex_coord[1] * height)]
20         return self.image[pixel_coord[0], pixel_coord[1], :3]
21
22 def project_light_function(self, sampler, light, bands):
23     """
24     Projects the light function onto the spherical harmonics basis.
25
26     Args:
27         sampler (Sampler): The sampler containing the samples.
28         light (str): The path to the light probe image.
29         bands (int): The number of bands.
30     """
31     self.lightCoeffs = np.zeros((bands * bands, 3), dtype=np.float32)
32     self.precompute_sh_functions(sampler, bands)
33
34     for i in range(sampler.number_of_samples):
35         direction = sampler.samples[i].cartesian_coord
36         color = self.light_probe_access(light, direction)
37         for j in range(bands * bands):
38             sh_function = sampler.samples[i].sh_functions[j]
39             self.lightCoeffs[j] += color * sh_function
40
41     weight = 4.0 * math.pi
42     scale = weight / sampler.number_of_samples
43     self.lightCoeffs *= scale

```

Listing 6: Functions responsible computing the light SH coefficients from an HDRI map.

```

1
2 def ProjectUnshadowed(self, sampler, vertices, normals, bands, lightprobeName, meshName):
3     """
4     Projects the unshadowed irradiance for the given vertices and normals.
5
6     Args:
7         sampler (Sampler): The sampler containing the samples.
8         vertices (np.ndarray): The vertices of the mesh.
9         normals (np.ndarray): The normals of the vertices.
10        bands (int): The number of bands.
11
12    Returns:
13        np.ndarray: The resulting colors for each vertex.
14    """
15
16    self.coeffs = np.zeros((len(vertices), bands * bands, 3), dtype=np.float32)
17
18    self.project_light_function(sampler, TEXTURES_PATH / 'skybox' / 'probes' / lightprobeName, bands)
19
20    progress = 0.00
21    for i in range(len(vertices)):
22        for j in range(sampler.number_of_samples):
23            sample = sampler.samples[j]
24            cosine_term = max(np.dot(normals[i], sample.cartesian_coord), 0.0)
25            for k in range(bands * bands):
26                sh_function = sample.sh_functions[k]
27                progress = (i / len(vertices)) * 100
28                print(f"Computing coefficients... {progress:.2f}%", end="\r")
29                self.coeffs[i][k] += 0.2 * sh_function * cosine_term
30
31    print(f"Computing coefficients... 100.00%", end="\r")
32
33    weight = 4.0 * math.pi
34    scale = weight / sampler.number_of_samples
35    self.coeffs *= scale
36
37
38    colors = np.zeros((len(vertices), 3), dtype=np.float32)
39    for i in range(len(vertices)):
40        for j in range(9):
41            colors[i] += self.lightCoeffs[j] * self.coeffs[i][j]
42

```

43 `return colors / 255`

Listing 7: Computation of unshadowed PRT SH coefficients and final color of mesh.

```

1 def ProjectShadowed(self, sampler, vertices, normals, bands, indices, lightprobeName, meshName):
2     """
3     Projects the shadowed irradiance for the given vertices and normals.
4
5     Args:
6         sampler (Sampler): The sampler containing the samples.
7         vertices (np.ndarray): The vertices of the mesh.
8         normals (np.ndarray): The normals of the vertices.
9         bands (int): The number of bands.
10        indices (np.ndarray): The indices of the mesh triangles.
11
12    Returns:
13        np.ndarray: The resulting colors for each vertex.
14    """
15    self.coeffs = np.zeros((len(vertices), bands * bands), dtype=np.float32)
16    self.project_light_function(sampler, TEXTURES_PATH / 'skybox' / 'probes'/lightprobeName, bands)
17
18    if os.path.exists(meshName + ".txt"):
19        with open(meshName + ".txt", 'r') as f:
20            for i in range(len(self.coeffs)):
21                line = np.loadtxt(f, max_rows=1, delimiter=' ')
22
23                if len(line) != 27:
24                    raise ValueError(f"Unexpected data length for vertex {i}: {len(line)}")
25
26                self.coeffs[i] = line.reshape(9, 3)
27    else:
28        progress = 0.00
29        start_time = time.time()
30        for i in range(len(vertices)):
31            for j in range(sampler.number_of_samples):
32                sample = sampler.samples[j]
33                progress = (i/len(vertices)) * 100
34                print(f"Computing coefficients.... {progress:.2f}%", end="\r")
35                if self.Visibility(vertices, i, sample.cartesian_coord, indices):
36                    cosine_term = max(np.dot(normals[i], sample.cartesian_coord), 0.0)
37                    for k in range(bands * bands):
38                        sh_function = sample.sh_functions[k]
39                        self.coeffs[i][k] += 0.2 * sh_function * cosine_term
40        end_time = time.time()
41        print(f"Computing coefficients.... 100.00%", end="\r")
42        elapsed_time = end_time - start_time
43        print(f"Elapsed time: {elapsed_time:.6f} seconds")
44
45        weight = 4.0 * math.pi
46        scale = weight / sampler.number_of_samples
47        self.coeffs *= scale
48
49    colors = np.zeros((len(vertices), 3), dtype=np.float32)
50    for i in range(len(vertices)):
51        for j in range(9):
52            colors[i] += self.lightCoeffs[j] * self.coeffs[i][j]
53
54    return colors / 255

```

Listing 8: Computation of shadowed PRT SH coefficients and final color of mesh.

```

1 @cuda.jit(device=True)
2 def _ray_intersects_triangle(origin, direction, v0, v1, v2):
3     """
4     Device function to check if a ray intersects a triangle.
5
6     Args:
7         origin (float32[:]): Ray origin point.
8         direction (float32[:]): Ray direction vector.
9         v0, v1, v2 (float32[:]): Triangle vertices.
10
11    Returns:
12        bool: True if the ray intersects the triangle, False otherwise.
13    """
14    epsilon = 1e-6
15
16    # Edges

```

```

17     edge1 = cuda.local.array(3, float32)
18     edge2 = cuda.local.array(3, float32)
19     for i in range(3):
20         edge1[i] = v1[i] - v0[i]
21         edge2[i] = v2[i] - v0[i]
22
23     # Cross product h = cross(direction, edge2)
24     h = cuda.local.array(3, float32)
25     h[0] = direction[1] * edge2[2] - direction[2] * edge2[1]
26     h[1] = direction[2] * edge2[0] - direction[0] * edge2[2]
27     h[2] = direction[0] * edge2[1] - direction[1] * edge2[0]
28
29     # Dot product a = dot(edge1, h)
30     a = edge1[0] * h[0] + edge1[1] * h[1] + edge1[2] * h[2]
31     if -epsilon < a < epsilon:
32         return False # Ray is parallel to triangle
33
34     f = 1.0 / a
35     s = cuda.local.array(3, float32)
36     for i in range(3):
37         s[i] = origin[i] - v0[i]
38
39     # Dot product u = f * dot(s, h)
40     u = f * (s[0] * h[0] + s[1] * h[1] + s[2] * h[2])
41     if u < 0.0 or u > 1.0:
42         return False
43
44     # Cross product q = cross(s, edge1)
45     q = cuda.local.array(3, float32)
46     q[0] = s[1] * edge1[2] - s[2] * edge1[1]
47     q[1] = s[2] * edge1[0] - s[0] * edge1[2]
48     q[2] = s[0] * edge1[1] - s[1] * edge1[0]
49
50     # Dot product v = f * dot(direction, q)
51     v = f * (direction[0] * q[0] + direction[1] * q[1] + direction[2] * q[2])
52     if v < 0.0 or u + v > 1.0:
53         return False
54
55     # Dot product t = f * dot(edge2, q)
56     t = f * (edge2[0] * q[0] + edge2[1] * q[1] + edge2[2] * q[2])
57     return t > epsilon
58
59
60 @cuda.jit
61 def _visibility_kernel(vertices, vertexidx, direction, indices, results, n_triangles):
62     """
63     GPU kernel to determine visibility of a vertex in a given direction.
64
65     Args:
66         vertices (float32[:, :]): The vertices of the mesh (device array).
67         vertexidx (int): The index of the vertex.
68         direction (float32[:]): The direction vector (device array).
69         indices (int32[:, :]): The indices of the mesh triangles (device array).
70         results (int32[:]): Array to store results (1 if intersects, 0 otherwise).
71         n_triangles (int): Number of triangles.
72     """
73     idx = cuda.grid(1)
74     if idx >= n_triangles:
75         return
76
77     # Calculate triangle vertices
78     i0 = indices[idx * 3]
79     i1 = indices[idx * 3 + 1]
80     i2 = indices[idx * 3 + 2]
81
82     # Skip triangles containing the vertex itself
83     if vertexidx == i0 or vertexidx == i1 or vertexidx == i2:
84         return
85
86     v0 = vertices[i0]
87     v1 = vertices[i1]
88     v2 = vertices[i2]
89     origin = vertices[vertexidx]
90
91     # Check for intersection
92     if _ray_intersects_triangle(origin, direction, v0, v1, v2):
93         results[idx] = 1
94
95 @staticmethod
96 def Visibility(vertices, vertexidx, direction, indices):

```

```

97     Checks the visibility of a vertex in a given direction using GPU acceleration.
98
99     Args:
100         vertices (np.ndarray): The vertices of the mesh.
101         vertexidx (int): The index of the vertex.
102         direction (np.ndarray): The direction vector.
103         indices (np.ndarray): The indices of the mesh triangles.
104
105     Returns:
106         bool: True if the vertex is visible in the given direction, False otherwise.
107     """
108     # Convert inputs to GPU-compatible data
109     d_vertices = cuda.to_device(vertices)
110     d_direction = cuda.to_device(direction)
111     d_indices = cuda.to_device(indices)
112
113     n_triangles = indices.shape[0] // 3
114     results = np.zeros(n_triangles, dtype=np.int32)
115     d_results = cuda.to_device(results)
116
117     threads_per_block = 256
118     blocks_per_grid = (n_triangles + threads_per_block - 1) // threads_per_block
119
120     _visibility_kernel[blocks_per_grid, threads_per_block](
121         d_vertices, vertexidx, d_direction, d_indices, d_results, n_triangles
122     )
123
124     # Copy results back to host and determine visibility
125     results = d_results.copy_to_host()
126     return not np.any(results)

```

Listing 9: Functions responsible for computing the visibility samples of a mesh, used in the shadowed PRT version.

B.2 GNN Training

```

1  import os
2  import numpy as np
3  import torch
4  import torch.nn as nn
5  import torch.optim as optim
6  from torch.utils.data import Dataset, DataLoader
7  from torch_geometric.data import Data, Batch
8  import torch.nn.functional as F
9  from sklearn.model_selection import train_test_split
10 from pyGandalf.utilities.GA.CGA_Multivector_Processing import build_motor_from_vertex_normal
11 from pyGandalf.utilities.definitions import NN_PATH
12
13 from torch_geometric.nn import radius_graph
14 from sklearn.metrics import f1_score
15 from tqdm import tqdm
16
17 # =====
18 # CONFIGURATION FOR VISIBILITY GNN
19 # =====
20 VISIBILITY_INPUT_SIZE = 32 # Motor features (vertices + normals)
21 VISIBILITY_OUTPUT_SIZE = 25 # 25 binary visibility samples per vertex
22 VISIBILITY_HIDDEN_CHANNELS = 64 # Hidden dimensions for Visibility GNN
23 VISIBILITY_NUM_LAYERS = 3 # Number of GNN layers for Visibility GNN
24 VISIBILITY_HEADS = 1 # Attention heads for Visibility GNN
25
26 NUM_EPOCHS_VISIBILITY = 200 # Epochs for visibility training
27 MESH_DIR_VISIBILITY = '...'
28 PROCESSED_SUBDIR_VISIBILITY = "processed_visibility_graphs" # Separate cache for visibility data
29
30 # =====
31 # UTILITY FUNCTIONS
32 # =====
33 def load_large_txt(file_path):
34     with open(file_path, "r") as f:
35         for line in f:
36             yield np.array([float(x) for x in line.split()], dtype=np.float32)
37
38 # =====
39 # VISIBILITY GNN MODEL

```

```

40 # =====
41 from torch_geometric.nn import TransformerConv
42
43 class VisibilityGNN(nn.Module):
44     def __init__(self, input_size, output_size, hidden_channels=64, num_layers=5, heads=1):
45         super(VisibilityGNN, self).__init__()
46         self.num_layers = num_layers
47         self.hidden_channels = hidden_channels
48
49         self.initial_lin = nn.Linear(input_size, hidden_channels)
50
51         self.convs = nn.ModuleList()
52         self.norms = nn.ModuleList()
53         self.dropouts = nn.ModuleList()
54
55         for _ in range(num_layers):
56             conv = TransformerConv(hidden_channels, hidden_channels, heads=heads, concat=False)
57             self.convs.append(conv)
58             self.norms.append(nn.LayerNorm(hidden_channels))
59             self.dropouts.append(nn.Dropout(0.1))
60
61         self.fc = nn.Sequential(
62             nn.Dropout(0.2),
63             nn.Linear(hidden_channels, output_size)
64         )
65
66     def forward(self, data):
67         x, edge_index = data.x, data.edge_index
68
69         x = F.relu(self.initial_lin(x))
70
71         if not x.requires_grad:
72             x.requires_grad_(True)
73
74         for i in range(self.num_layers):
75             identity = x
76
77             def block(x_in, edge_index_in):
78                 out = self.convs[i](x_in, edge_index_in)
79                 out = self.norms[i](out)
80                 return F.relu(out)
81
82             x = torch.utils.checkpoint.checkpoint(block, x, edge_index, use_reentrant=False)
83             x = self.dropouts[i](x)
84
85             if x.shape == identity.shape:
86                 x = x + identity
87         return self.fc(x)
88
89 # =====
90 # VISIBILITY DATASET
91 # =====
92 class VisibilityDataset(Dataset):
93     def __init__(self, mesh_directory, processed_subdir="processed_visibility_graphs"):
94         self.mesh_directory = mesh_directory
95         self.processed_dir = os.path.join(mesh_directory, processed_subdir)
96         self.graphs = []
97
98         if self._check_processed_files():
99             print(f"Loading processed visibility graphs from {self.processed_dir}...")
100             self._load_processed_files()
101         else:
102             print(f"Processed visibility graphs not found or incomplete. Starting fresh processing and saving to {self.processed_dir}...")
103             self._process_and_save_graphs()
104
105     def _check_processed_files(self):
106         if not os.path.exists(self.processed_dir):
107             return False
108
109         processed_files_count = len([f for f in os.listdir(self.processed_dir) if f.endswith('.pt')])
110
111         all_files = os.listdir(self.mesh_directory)
112         model_names = {
113             f.replace("_vertices.txt", "")
114             for f in all_files if f.endswith("_vertices.txt")
115         }
116         num_expected_models = 0
117         for model_name in model_names:
118             vertices_file = os.path.join(self.mesh_directory, f"{model_name}_vertices.txt")

```

```

119         normals_file = os.path.join(self.mesh_directory, f"{model_name}_normals.txt")
120         vis_file = os.path.join(self.mesh_directory, f"{model_name}.obj_visibility_samples.txt")
121
122         if os.path.exists(vertices_file) and os.path.exists(normals_file) and os.path.exists(vis_file):
123             num_expected_models += 1
124
125         return processed_files_count == num_expected_models
126
127     def _load_processed_files(self):
128         processed_files = sorted([f for f in os.listdir(self.processed_dir) if f.endswith('.pt')])
129         for i, filename in enumerate(processed_files):
130             file_path = os.path.join(self.processed_dir, filename)
131             try:
132                 data = torch.load(file_path)
133                 self.graphs.append(data)
134                 if (i + 1) % 100 == 0:
135                     print(f" Loaded {i+1} graphs...")
136             except Exception as e:
137                 print(f"Error loading processed file {filename}: {e}. Deleting {self.processed_dir} and re-
                    processing.")
138                 import shutil
139                 if os.path.exists(self.processed_dir):
140                     shutil.rmtree(self.processed_dir)
141                 self.graphs = []
142                 self._process_and_save_graphs()
143                 return
144
145     def _process_and_save_graphs(self):
146         if not os.path.exists(self.processed_dir):
147             os.makedirs(self.processed_dir)
148
149         all_files = os.listdir(self.mesh_directory)
150         model_names = {
151             f.replace("_vertices.txt", "")
152             for f in all_files if f.endswith("_vertices.txt")
153         }
154
155         all_input_features_temp = []
156
157         raw_data_items = []
158
159         print("First pass: Loading raw data for visibility GNN...")
160         for model_idx, model_name in enumerate(model_names):
161             vertices_file = os.path.join(self.mesh_directory, f"{model_name}_vertices.txt")
162             normals_file = os.path.join(self.mesh_directory, f"{model_name}_normals.txt")
163             vis_file = os.path.join(self.mesh_directory, f"{model_name}.obj_visibility_samples.txt")
164
165             if not (os.path.exists(vertices_file) and os.path.exists(normals_file) and os.path.exists(
                vis_file)):
166                 print(f"Skipping {model_name}: missing required files for visibility training.")
167                 continue
168
169             try:
170                 vertices = np.loadtxt(vertices_file, dtype=np.float32)
171                 normals = np.loadtxt(normals_file, dtype=np.float32)
172                 visibilities = np.loadtxt(vis_file, dtype=np.float32) # Target for this GNN
173                 num_vertices = vertices.shape[0]
174                 assert num_vertices == normals.shape[0] == visibilities.shape[0], \
175                     f"Mismatched vertex counts for {model_name}"
176
177                 normals = np.nan_to_num(normals, nan=0.0, posinf=0.0, neginf=0.0)
178                 visibilities = np.nan_to_num(visibilities, nan=0.0, posinf=0.0, neginf=0.0)
179
180                 cleaned_normals = np.array([
181                     np.array([0.0, 1.0, 0.0], dtype=np.float32) if np.linalg.norm(n_val) == 0 else n_val /
182                     np.linalg.norm(n_val)
183                     for n_val in normals
184                 ])
185
186                 # Input features for Visibility GNN (motor features only)
187                 input_features_list = []
188                 for v, n_clean in zip(vertices, cleaned_normals):
189                     motor_f = build_motor_from_vertex_normal(v, n_clean)
190                     motor_f = np.nan_to_num(motor_f, nan=0.0, posinf=0.0, neginf=0.0)
191                     input_features_list.append(motor_f)
192
193                 input_raw_tensor = torch.tensor(np.array(input_features_list), dtype=torch.float32)
194                 target_vis_tensor = torch.tensor(visibilities, dtype=torch.float32) # Target is visibility
195
196                 # Store raw data for second pass after global stats are known

```

```

196         raw_data_items.append({
197             "model_name": model_name,
198             "input_features_raw": input_raw_tensor,
199             "target_vis_raw": target_vis_tensor,
200             "raw_pos": torch.tensor(vertices, dtype=torch.float32), # Store raw pos for graph
                           construction
201         })
202         all_input_features_temp.append(input_raw_tensor.numpy())
203
204     except Exception as e:
205         print(f"Error processing {model_name}: {e}. Skipping.")
206         continue
207
208     # --- GLOBAL INPUT NORMALIZATION STATS CALCULATION ---
209     self.global_input_mean = np.zeros(VISIBILITY_INPUT_SIZE, dtype=np.float32)
210     self.global_input_std = np.ones(VISIBILITY_INPUT_SIZE, dtype=np.float32)
211     if all_input_features_temp:
212         all_features_concatenated = np.vstack(all_input_features_temp)
213         self.global_input_mean = np.mean(all_features_concatenated, axis=0, dtype=np.float32)
214         self.global_input_std = np.std(all_features_concatenated, axis=0, dtype=np.float32)
215         self.global_input_std[self.global_input_std == 0] = 1e-6
216
217     print(f"Global Input Features Mean (for normalization): {self.global_input_mean}")
218     print(f"Global Input Features Std (for normalization): {self.global_input_std}")
219
220     # Save normalization stats to disk
221     normalization_stats = {
222         'global_input_mean': self.global_input_mean,
223         'global_input_std': self.global_input_std,
224     }
225     np.savez(os.path.join(self.processed_dir, 'normalization_stats.npz'), **normalization_stats)
226     print(f"Saved global normalization statistics to {self.processed_dir}/normalization_stats.npz")
227
228
229     # Second pass: Apply normalization and save processed graphs
230     print("Second pass: Applying global input normalization and saving processed graphs...")
231     for i, data_item in enumerate(raw_data_items):
232         model_name = data_item["model_name"]
233         raw_input_tensor = data_item["input_features_raw"]
234         target_vis_raw = data_item["target_vis_raw"]
235         raw_pos = data_item["raw_pos"]
236
237         # Apply input normalization (Z-score)
238         normalized_input_features = (raw_input_tensor - torch.tensor(self.global_input_mean, dtype=
            torch.float32)) / \
239                                     torch.tensor(self.global_input_std, dtype=torch.float32)
240
241         # Normalize position for radius_graph (consistent with other GNNs)
242         pos_mean = raw_pos.mean(dim=0)
243         pos_centered = raw_pos - pos_mean
244         pos_scale = pos_centered.abs().max()
245         if pos_scale > 1e-6:
246             pos_normalized = pos_centered / pos_scale
247         else:
248             pos_normalized = pos_centered
249
250         if pos_normalized.shape[0] > 1:
251             sample_size = min(pos_normalized.shape[0], 5000)
252             sample_indices = torch.randperm(pos_normalized.shape[0])[:sample_size]
253             avg_distance = torch.cdist(pos_normalized[sample_indices], pos_normalized[sample_indices]).
                mean().item()
254         else:
255             avg_distance = 1.0
256         radius_multiplier = 0.1
257         radius_val = avg_distance * radius_multiplier
258
259         edge_index = radius_graph(pos_normalized.cpu(), r=radius_val, loop=False)
260
261         processed_data = Data(
262             x=normalized_input_features,
263             y=target_vis_raw,
264             edge_index=edge_index,
265             pos=raw_pos,
266             pos_normalized=pos_normalized,
267             model_name=model_name,
268         )
269         self.graphs.append(processed_data)
270
271         save_path = os.path.join(self.processed_dir, f"graph_{i}.pt")
272         torch.save(processed_data, save_path)

```

```

273         if (i + 1) % 10 == 0:
274             print(f" Saved {i+1} graphs to {self.processed_dir}")
275
276             print(f"Total processed graphs saved: {len(self.graphs)}")
277
278         def __len__(self):
279             return len(self.graphs)
280
281         def __getitem__(self, idx):
282             data = self.graphs[idx]
283             return data, data.y, data.model_name
284
285 # =====
286 # CUSTOM COLLATE FUNCTION FOR VISIBILITY GNN
287 # =====
288 def custom_collate_fn_visibility(batch):
289     data_list = [item[0] for item in batch]
290     targets = [item[1] for item in batch]
291     model_names = [item[2] for item in batch]
292
293     student_graph_batch = Batch.from_data_list(data_list)
294
295     # Concatenate targets from individual graphs
296     concatenated_targets = torch.cat(targets, dim=0)
297
298     return student_graph_batch, concatenated_targets, model_names
299
300 # =====
301 # MAIN TRAINING LOOP FOR VISIBILITY GNN
302 # =====
303 from sklearn.metrics import f1_score
304 from tqdm import tqdm
305 from transformers import get_cosine_schedule_with_warmup
306
307 def compute_f1(preds, targets):
308     preds_np = preds.cpu().numpy().flatten()
309     targets_np = targets.cpu().numpy().flatten()
310     return f1_score(targets_np, preds_np, average='macro')
311
312 def main_visibility_training():
313     device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
314
315     dataset = VisibilityDataset(MESH_DIR_VISIBILITY, processed_subdir=PROCESSED_SUBDIR_VISIBILITY)
316     train_indices, val_indices = train_test_split(range(len(dataset)), test_size=0.2, random_state=42)
317
318     train_loader = DataLoader(torch.utils.data.Subset(dataset, train_indices), batch_size=4, shuffle=True,
319                               collate_fn=custom_collate_fn_visibility)
320     val_loader = DataLoader(torch.utils.data.Subset(dataset, val_indices), batch_size=4, shuffle=False,
321                             collate_fn=custom_collate_fn_visibility)
322
323     model = VisibilityGNN(
324         input_size=VISIBILITY_INPUT_SIZE,
325         output_size=VISIBILITY_OUTPUT_SIZE,
326         hidden_channels=VISIBILITY_HIDDEN_CHANNELS,
327         num_layers=VISIBILITY_NUM_LAYERS,
328         heads=VISIBILITY_HEADS
329     ).to(device)
330
331     # Compute pos_weight
332     all_targets = torch.cat([g.y for g in dataset.graphs], dim=0)
333     pos_ratio_per_label = all_targets.mean(dim=0)
334     pos_weight_tensor = (1.0 - pos_ratio_per_label) / (pos_ratio_per_label + 1e-6)
335     criterion = nn.BCEWithLogitsLoss(pos_weight=pos_weight_tensor.to(device))
336
337     optimizer = optim.AdamW(model.parameters(), lr=1e-4, weight_decay=1e-4)
338     total_steps = len(train_loader) * NUM_EPOCHS_VISIBILITY
339     scheduler = get_cosine_schedule_with_warmup(
340         optimizer,
341         num_warmup_steps=int(0.1 * total_steps),
342         num_training_steps=total_steps
343     )
344
345     best_val_f1 = 0.0
346     best_epoch = 0
347     patience = 20
348
349     print("Starting Visibility GNN training...")
350     for epoch in range(NUM_EPOCHS_VISIBILITY):
351         model.train()
352         train_loss = 0.0

```

```

351     correct_train = 0
352     total_train = 0
353     all_train_preds = []
354     all_train_targets = []
355
356     for inputs_batch, targets, _ in tqdm(train_loader, desc=f"Epoch {epoch+1}/{NUM_EPOCHS_VISIBILITY} -
357         Train"):
358         inputs_batch = inputs_batch.to(device)
359         targets = targets.to(device)
360
361         optimizer.zero_grad()
362         outputs = model(inputs_batch)
363         loss = criterion(outputs, targets)
364         loss.backward()
365         optimizer.step()
366         scheduler.step()
367
368         train_loss += loss.item()
369         predicted_probs = torch.sigmoid(outputs)
370         predicted_classes = (predicted_probs > 0.5).float()
371
372         correct_train += (predicted_classes == targets).sum().item()
373         total_train += targets.numel()
374         all_train_preds.append(predicted_classes.detach())
375         all_train_targets.append(targets.detach())
376
377     avg_train_loss = train_loss / len(train_loader)
378     train_accuracy = correct_train / total_train
379     train_f1 = compute_f1(torch.cat(all_train_preds), torch.cat(all_train_targets))
380
381     # Validation
382     model.eval()
383     val_loss = 0.0
384     correct_val = 0
385     total_val = 0
386     all_val_preds = []
387     all_val_targets = []
388
389     with torch.no_grad():
390         for inputs_batch, targets, _ in tqdm(val_loader, desc=f"Epoch {epoch+1}/{NUM_EPOCHS_VISIBILITY}
391             - Val"):
392             inputs_batch = inputs_batch.to(device)
393             targets = targets.to(device)
394
395             outputs = model(inputs_batch)
396             loss = criterion(outputs, targets)
397             val_loss += loss.item()
398
399             predicted_probs = torch.sigmoid(outputs)
400             predicted_classes = (predicted_probs > 0.5).float()
401
402             correct_val += (predicted_classes == targets).sum().item()
403             total_val += targets.numel()
404             all_val_preds.append(predicted_classes)
405             all_val_targets.append(targets)
406
407     avg_val_loss = val_loss / len(val_loader)
408     val_accuracy = correct_val / total_val
409     val_f1 = compute_f1(torch.cat(all_val_preds), torch.cat(all_val_targets))
410
411     print(f"Epoch [{epoch+1}/{NUM_EPOCHS_VISIBILITY}] Train Loss: {avg_train_loss:.6f}, Acc: {
412         train_accuracy:.4f}, F1: {train_f1:.4f}")
413     print(f"                               Val Loss: {avg_val_loss:.6f}, Acc: {val_accuracy:.4f}, F1: {
414         val_f1:.4f}")
415
416     # Save best model
417     if val_f1 > best_val_f1:
418         best_val_f1 = val_f1
419         best_epoch = epoch
420         torch.save(model.state_dict(), NN_PATH / 'VisibilityGNN_best.pth')
421         print(f"Saved new best model (Val F1: {val_f1:.4f})")
422
423     elif epoch - best_epoch >= patience:
424         print(f"Early stopping at epoch {epoch+1} (no F1 improvement for {patience} epochs)")
425         break
426
427 if __name__ == "__main__":
428     main_visibility_training()

```

Listing 10: Code responsible for training the GNN that predicts the visibility samples.

```

1 import torch
2 import torch.nn as nn
3 import torch.nn.functional as F
4 from torch_geometric.nn import GATv2Conv
5 from torch_geometric.data import Data
6 from torch.utils.data import Dataset
7 from torch_geometric.loader import DataLoader
8 from sklearn.model_selection import train_test_split
9 import numpy as np
10 import os
11 from torch.nn import Sequential as Seq, Linear as Lin, ReLU
12 from torch.utils.checkpoint import checkpoint
13 import gc
14 from torch.optim.lr_scheduler import ReduceLROnPlateau
15 from torch_geometric.nn import radius_graph
16
17 def load_large_txt(file_path):
18     with open(file_path, "r") as f:
19         for line in f:
20             yield np.array([float(x) for x in line.split()], dtype=np.float32)
21
22 class SHModel_GNN(nn.Module):
23     def __init__(self, input_size=57, output_size=9, hidden_channels=96, num_layers=3, heads=2):
24         super().__init__()
25         self.num_layers = num_layers
26         self.hidden_channels = hidden_channels
27
28         self.initial_lin = nn.Linear(input_size, hidden_channels)
29
30         self.convs = nn.ModuleList()
31         self.norms = nn.ModuleList()
32         self.dropouts = nn.ModuleList()
33
34         for i in range(num_layers):
35             self.convs.append(GATv2Conv(hidden_channels, hidden_channels, heads=heads, concat=False))
36             self.norms.append(nn.LayerNorm(hidden_channels))
37             self.dropouts.append(nn.Dropout(0.2))
38
39         self.fc = nn.Linear(hidden_channels, output_size)
40
41     def forward(self, data):
42         x, edge_index = data.x, data.edge_index
43
44         x = F.relu(self.initial_lin(x))
45
46         if not x.requires_grad:
47             x.requires_grad_(True)
48
49         for i in range(self.num_layers):
50             conv_layer = self.convs[i]
51             norm_layer = self.norms[i]
52             dropout_layer = self.dropouts[i]
53
54             identity = x
55
56             def gnn_block(x_in, edge_index_in):
57                 out = conv_layer(x_in, edge_index_in)
58                 out = norm_layer(out)
59                 out = F.relu(out)
60                 return out
61
62             x = checkpoint(gnn_block, x, edge_index, use_reentrant=False)
63             x = dropout_layer(x)
64
65             if x.shape == identity.shape:
66                 x = x + identity
67             else:
68                 print(f"Warning: Dimensions mismatch for residual connection at layer {i}. Skipping residual.")
69
70         final_output = self.fc(x)
71
72         if torch.isnan(final_output).any() or torch.isinf(final_output).any():
73             print("NaN/Inf detected in final model output!")
74
75         return final_output
76
77 class MeshGraphDataset(Dataset):
78     def __init__(self, mesh_directory, radius=0.05, processed_subdir="processed_graphs"):
79         self.mesh_directory = mesh_directory

```

```

80     self.radius = radius
81     self.processed_dir = os.path.join(mesh_directory, processed_subdir)
82     self.graphs = []
83
84     if self._check_processed_files():
85         print(f"Loading processed graphs from {self.processed_dir}...")
86         self._load_processed_files()
87     else:
88         print(f"Processed graphs not found or incomplete. Starting fresh processing and saving to {self}
89             .processed_dir}...")
90         self._process_and_save_graphs()
91
92     def _check_processed_files(self):
93         if not os.path.exists(self.processed_dir):
94             return False
95
96         processed_files_count = len([f for f in os.listdir(self.processed_dir) if f.endswith('.pt')])
97
98         all_files = os.listdir(self.mesh_directory)
99         model_names = {
100             file.replace("_vertices.txt", "")
101             for file in all_files if file.endswith("_vertices.txt")
102         }
103         num_expected_models = 0
104         for model_name in model_names:
105             vertices_path = os.path.join(self.mesh_directory, f"{model_name}_vertices.txt")
106             normals_path = os.path.join(self.mesh_directory, f"{model_name}_normals.txt")
107             visibility_path = os.path.join(self.mesh_directory, f"{model_name}.obj_visibility_samples.txt")
108             coeffs_path = os.path.join(self.mesh_directory, f"{model_name}.obj.txt")
109
110             # All four files must exist to consider the model valid for processing
111             if os.path.exists(vertices_path) and os.path.exists(normals_path) and \
112                 os.path.exists(visibility_path) and os.path.exists(coeffs_path):
113                 num_expected_models += 1
114
115         return processed_files_count == num_expected_models
116
117     def _load_processed_files(self):
118         processed_files = sorted([f for f in os.listdir(self.processed_dir) if f.endswith('.pt')])
119         for i, filename in enumerate(processed_files):
120             file_path = os.path.join(self.processed_dir, filename)
121             try:
122                 data = torch.load(file_path)
123                 self.graphs.append(data)
124                 if (i + 1) % 100 == 0:
125                     print(f"Loaded {i+1} graphs...")
126             except Exception as e:
127                 print(f"Error loading processed file {filename}: {e}. Deleting {self.processed_dir} and re-
128                     processing.")
129                 import shutil
130                 if os.path.exists(self.processed_dir):
131                     shutil.rmtree(self.processed_dir)
132                 self.graphs = []
133                 self._process_and_save_graphs()
134                 return
135
136     def _process_and_save_graphs(self):
137         if not os.path.exists(self.processed_dir):
138             os.makedirs(self.processed_dir)
139
140         all_files = os.listdir(self.mesh_directory)
141         model_names = {
142             file.replace("_vertices.txt", "")
143             for file in all_files if file.endswith("_vertices.txt")
144         }
145
146         for i, model_name in enumerate(model_names):
147             print(f"Processing {model_name} ({i+1}/{len(model_names)})...")
148
149             vertices_path = os.path.join(self.mesh_directory, f"{model_name}_vertices.txt")
150             normals_path = os.path.join(self.mesh_directory, f"{model_name}_normals.txt")
151             visibility_path = os.path.join(self.mesh_directory, f"{model_name}.obj_visibility_samples.txt")
152             # New path
153             coeffs_path = os.path.join(self.mesh_directory, f"{model_name}.obj.txt")
154
155             if not (os.path.exists(vertices_path) and os.path.exists(normals_path) and \
156                 os.path.exists(visibility_path) and os.path.exists(coeffs_path)):
157                 print(f"Skipping {model_name}: Missing one or more required files (including visibility).")
158                 continue

```

```

156
157         try:
158             vertices = np.loadtxt(vertices_path, dtype=np.float32)
159             normals = np.loadtxt(normals_path, dtype=np.float32)
160             visibilities = np.loadtxt(visibility_path, dtype=np.float32)
161             coeffs = np.vstack(list(load_large_txt(coeffs_path)))
162
163             if np.isnan(coeffs).any() or np.isinf(coeffs).any():
164                 print(f"    WARNING: NaN/Inf detected in target coefficients (coeffs) for {model_name}.
165                     Replacing with zeros.")
166                 coeffs = np.nan_to_num(coeffs, nan=0.0, posinf=0.0, neginf=0.0)
167
168             if np.isnan(visibilities).any() or np.isinf(visibilities).any():
169                 print(f"    WARNING: NaN/Inf detected in visibility samples for {model_name}. Replacing
170                     with zeros.")
171                 visibilities = np.nan_to_num(visibilities, nan=0.0, posinf=0.0, neginf=0.0)
172
173         except Exception as e:
174             print(f"    Error loading files for {model_name}: {e}. Skipping.")
175             continue
176
177         features = []
178         for v, n in zip(vertices, normals):
179             n = np.nan_to_num(n, nan=0.0, posinf=0.0, neginf=0.0)
180             if np.linalg.norm(n) == 0:
181                 n = np.array([0, 1, 0], dtype=np.float32)
182             f = build_motor_from_vertex_normal(v, n)
183             f = np.nan_to_num(f, nan=0.0, posinf=0.0, neginf=0.0)
184
185             if np.isnan(f).any() or np.isinf(f).any():
186                 print(f"    WARNING: NaN/Inf detected in motor features 'f' for {model_name}. Setting
187                     to zeros.")
188                 f = np.zeros_like(f)
189
190             f_norm = np.linalg.norm(f)
191             if f_norm != 0:
192                 f = f / f_norm
193             features.append(f)
194
195         motor_features = torch.tensor(np.array(features), dtype=torch.float32)
196         y = torch.tensor(coeffs, dtype=torch.float32)
197         raw_pos = torch.tensor(vertices, dtype=torch.float32)
198
199         visibility_samples_tensor = torch.tensor(visibilities, dtype=torch.float32)
200
201         if visibility_samples_tensor.shape[0] != motor_features.shape[0] or \
202             visibility_samples_tensor.shape[1] != 25:
203             print(f"    ERROR: Visibility samples shape mismatch for {model_name}. Expected ({
204                 motor_features.shape[0]}, 25), got {visibility_samples_tensor.shape}. Skipping model."
205                 )
206             continue
207
208         pos_mean = raw_pos.mean(dim=0)
209         pos_centered = raw_pos - pos_mean
210         pos_scale = pos_centered.abs().max()
211         if pos_scale > 1e-6:
212             pos_normalized = pos_centered / pos_scale
213         else:
214             pos_normalized = pos_centered
215
216         x = torch.cat([motor_features, visibility_samples_tensor], dim=-1)
217
218         if torch.isnan(x).any() or torch.isinf(x).any():
219             print(f"    WARNING: NaN/Inf detected in combined input features 'x' for {model_name}. This
220                 graph might cause issues.")
221
222         if pos_normalized.shape[0] > 1:
223             sample_size = min(pos_normalized.shape[0], 5000)
224             sample_indices = torch.randperm(pos_normalized.shape[0])[:sample_size]
225             avg_distance = torch.cdist(pos_normalized[sample_indices], pos_normalized[sample_indices]).
226                 mean().item()
227         else:
228             avg_distance = 1.0
229
230         radius_multiplier = 0.1
231         radius_val = avg_distance * radius_multiplier
232
233         print(f"    Calculated radius for {model_name}: {radius_val:.6f}")

```

```

229         edge_index = radius_graph(pos_normalized.cpu(), r=radius_val, loop=False)
230
231     data = Data(x=x, edge_index=edge_index, y=y, pos=raw_pos, pos_normalized=pos_normalized,
232               model_name=model_name)
233     self.graphs.append(data)
234     print(f"  Processed {model_name}: {x.shape[1]} features per vertex, {edge_index.shape[1] // 2}
235           edges")
236
237     save_path = os.path.join(self.processed_dir, f"graph_{i}.pt")
238     torch.save(data, save_path)
239     print(f"  Saved processed graph to {save_path}")
240
241     def __len__(self):
242         return len(self.graphs)
243
244     def __getitem__(self, idx):
245         return self.graphs[idx]
246
247 def weighted_log_mse_loss(predicted, target):
248     # Ensure both tensors are of float type for calculations
249     predicted = predicted.float()
250     target = target.float()
251
252     # Calculate the squared difference between predicted and target
253     squared_error = (predicted - target) ** 2
254
255     # Calculate the mean of the squared errors
256     loss_val = torch.mean(squared_error)
257
258     # Basic check for NaN/Inf in the output, which can indicate issues upstream
259     if torch.isnan(loss_val).any() or torch.isinf(loss_val).any():
260         print(f"Warning: NaN/Inf detected in default_mse_loss output!")
261         print(f"Predicted min/max: {predicted.min().item():.4e}/{predicted.max().item():.4e}")
262         print(f"Target min/max: {target.min().item():.4e}/{target.max().item():.4e}")
263
264     return loss_val
265
266 def train_gnn():
267     device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
268     print(f"Using device: {device}")
269     torch.cuda.empty_cache()
270     gc.collect()
271
272     dataset = MeshGraphDataset(
273         mesh_directory='...',
274         radius=0.05,
275         processed_subdir="processed_graphs"
276     )
277
278     train_dataset, val_dataset = train_test_split(dataset, test_size=0.2, random_state=42)
279     train_loader = DataLoader(train_dataset, batch_size=1, shuffle=True)
280     val_loader = DataLoader(val_dataset, batch_size=1)
281
282     model = SHModel_GNN(input_size=57, output_size=9, hidden_channels=96, num_layers=5, heads=2).to(device)
283
284     # Increased learning rate back to a more common value for MSE
285     optimizer = torch.optim.AdamW(model.parameters(), lr=0.0005)
286     scaler = torch.cuda.amp.GradScaler()
287
288     scheduler = ReduceLROnPlateau(optimizer, mode='min', factor=0.5, patience=5, verbose=True)
289
290     print("Starting training...")
291     for epoch in range(200):
292         model.train()
293         train_loss = 0.0
294         for i, data in enumerate(train_loader):
295             current_model_name = data.model_name if hasattr(data, 'model_name') else f"graph_idx_{i}"
296
297             data = data.to(device)
298
299             # Keep NaN/Inf checks for data integrity
300             if torch.isnan(data.x).any() or torch.isinf(data.x).any():
301                 print(f"NaN/Inf in data.x for Epoch {epoch+1}, Batch {i+1} (Model: {current_model_name}).
302                       Skipping batch.")
303                 continue
304             if torch.isnan(data.y).any() or torch.isinf(data.y).any():
305                 print(f"NaN/Inf in data.y (target) for Epoch {epoch+1}, Batch {i+1} (Model: {

```

```

        current_model_name)). Skipping batch.")
306     continue
307     if torch.isnan(data.edge_index).any() or torch.isinf(data.edge_index).any():
308         print(f"NaN/Inf in data.edge_index for Epoch {epoch+1}, Batch {i+1} (Model: {
            current_model_name)). Skipping batch.")
309     continue
310
311
312     optimizer.zero_grad()
313     with torch.cuda.amp.autocast():
314         out = model(data)
315         loss = weighted_log_mse_loss(out, data.y)
316
317     if torch.isnan(loss).any() or torch.isinf(loss).any():
318         print(f"NaN/Inf loss detected for Epoch {epoch+1}, Batch {i+1} (Model: {current_model_name
            })). Skipping backprop for this batch.")
319     continue
320
321     scaler.scale(loss).backward()
322
323     scaler.unscale_(optimizer)
324     torch.nn.utils.clip_grad_norm_(model.parameters(), max_norm=1.0)
325
326     scaler.step(optimizer)
327     scaler.update()
328
329     train_loss += loss.item()
330     if (i + 1) % 10 == 0:
331         print(f" [Epoch {epoch+1}, Batch {i+1}] Train Loss: {loss.item():.4f}")
332
333     avg_train_loss = train_loss / len(train_loader)
334     print(f"[Epoch {epoch+1}] Train Loss: {avg_train_loss:.4f}")
335
336     if np.isnan(avg_train_loss):
337         print("Training loss became NaN. Stopping training.")
338         break
339
340     model.eval()
341     val_loss = 0.0
342     shadow_loss = 0.0
343     num_shadow_points = 0
344
345     print("Starting validation...")
346     with torch.no_grad():
347         for i, data in enumerate(val_loader):
348             current_model_name = data.model_name if hasattr(data, 'model_name') else f"graph_idx_{i}"
349
350             data = data.to(device)
351
352             if torch.isnan(data.x).any() or torch.isinf(data.x).any() or \
353                 torch.isnan(data.y).any() or torch.isinf(data.y).any() or \
354                 torch.isnan(data.edge_index).any() or torch.isinf(data.edge_index).any():
355                 print(f"NaN/Inf in validation data for Epoch {epoch+1}, Batch {i+1} (Model: {
                    current_model_name)). Skipping validation batch.")
356                 continue
357
358             with torch.cuda.amp.autocast():
359                 out = model(data)
360                 loss_val_batch = weighted_log_mse_loss(out, data.y)
361
362             if torch.isnan(loss_val_batch).any() or torch.isinf(loss_val_batch).any():
363                 print(f"NaN/Inf validation loss detected for Epoch {epoch+1}, Batch {i+1} (Model: {
                    current_model_name)). Skipping this batch.")
364                 continue
365
366             val_loss += loss_val_batch.item()
367
368             # The brightness calculation still needs to handle potential negative SH coeffs
369             # For a "brightness" mask, you generally want to consider positive light contributions.
370             # Clamping to 0.0 here before averaging for brightness is reasonable.
371             brightness = torch.clamp(data.y.float(), min=0.0).mean(dim=-1)
372             shadow_mask = brightness < 0.2
373             if shadow_mask.any():
374                 shadow_diff = ((out.float() - data.y.float()) ** 2)[shadow_mask]
375                 if not (torch.isnan(shadow_diff).any() or torch.isinf(shadow_diff).any()):
376                     shadow_loss += shadow_diff.mean().item()
377                     num_shadow_points += 1
378             else:
379                 print(f"NaN/Inf in shadow_diff for Epoch {epoch+1}, Batch {i+1} (Model: {
                    current_model_name)). Skipping shadow loss for this batch.")

```

```

380
381     val_loss /= len(val_loader) if len(val_loader) > 0 else 1
382     shadow_loss /= max(num_shadow_points, 1)
383
384     print(f"Validation Loss: {val_loss:.4f}")
385     print(f"Shadow Region MSE: {shadow_loss:.4f}")
386
387     scheduler.step(val_loss)
388
389
390     torch.save(model.state_dict(), "GNN_SH.pth")
391     print("Model saved to: GNN_SH.pth")
392
393 if __name__ == "__main__":
394     train_gnn()

```

Listing 11: Code responsible for training the GNN that predicts the PRT SH coefficients.

B.3 Flask Server

```

1 from flask import Flask, request, jsonify, make_response
2 import numpy as np
3 import torch
4 import torch.nn as nn
5 import torch.nn.functional as F
6
7 from torch_geometric.nn import GATv2Conv, knn_graph # Added knn_graph import
8 from torch_geometric.nn import TransformerConv
9 from torch_geometric.data import Data
10 from waitress import serve
11
12 import time
13 from pathlib import Path
14
15 # --- CONFIGURATION ---
16 VISIBILITY_INPUT_SIZE = 32 # Still the size of the multivectors
17 VISIBILITY_OUTPUT_SIZE = 25
18 VISIBILITY_HIDDEN_CHANNELS = 64
19 VISIBILITY_NUM_LAYERS = 3
20 VISIBILITY_HEADS = 1
21
22 COEFFICIENT_INPUT_SIZE = VISIBILITY_INPUT_SIZE + VISIBILITY_OUTPUT_SIZE # 32 + 25 = 57
23 COEFFICIENT_OUTPUT_SIZE = 9
24 COEFFICIENT_HIDDEN_CHANNELS = 96
25 COEFFICIENT_NUM_LAYERS = 5
26 COEFFICIENT_HEADS = 2
27
28 NN_PATH = Path('.')
29 visNN = "VisibilityGNN_best.pth"
30 coeffNN = "GNN_SH.pth"
31
32 # --- GNN MODEL DEFINITIONS ---
33 class VisibilityGNN(nn.Module):
34     def __init__(self, input_size, output_size, hidden_channels=64, num_layers=5, heads=1):
35         super(VisibilityGNN, self).__init__()
36         self.num_layers = num_layers
37         self.hidden_channels = hidden_channels
38         self.initial_lin = nn.Linear(input_size, hidden_channels)
39         self.convs = nn.ModuleList()
40         self.norms = nn.ModuleList()
41         self.dropouts = nn.ModuleList()
42         for _ in range(num_layers):
43             conv = TransformerConv(hidden_channels, hidden_channels, heads=heads, concat=False)
44             self.convs.append(conv)
45             self.norms.append(nn.LayerNorm(hidden_channels))
46             self.dropouts.append(nn.Dropout(0.1))
47         self.fc = nn.Sequential(nn.Dropout(0.2), nn.Linear(hidden_channels, output_size))
48     def forward(self, data):
49         x, edge_index = data.x, data.edge_index
50         x = F.relu(self.initial_lin(x))
51         for i in range(self.num_layers):
52             identity = x
53             out = self.convs[i](x, edge_index)
54             out = self.norms[i](out)

```

```

55         x = F.relu(out)
56         x = self.dropouts[i](x)
57         if x.shape == identity.shape:
58             x = x + identity
59         return self.fc(x)
60
61 class CoefficientGNN(nn.Module):
62     def __init__(self, input_size=57, output_size=9, hidden_channels=96, num_layers=5, heads=2):
63         super().__init__()
64         self.num_layers = num_layers
65         self.hidden_channels = hidden_channels
66         self.heads = heads
67         self.initial_lin = nn.Linear(input_size, hidden_channels)
68         self.convs = nn.ModuleList()
69         self.norms = nn.ModuleList()
70         self.dropouts = nn.ModuleList()
71         for i in range(num_layers):
72             self.convs.append(GATv2Conv(hidden_channels, hidden_channels, heads=heads, concat=False))
73             self.norms.append(nn.LayerNorm(hidden_channels))
74             self.dropouts.append(nn.Dropout(0.2))
75         self.fc = nn.Linear(hidden_channels, output_size)
76     def forward(self, data):
77         x, edge_index = data.x, data.edge_index
78         x = F.relu(self.initial_lin(x))
79         for i in range(self.num_layers):
80             identity = x
81             conv_layer = self.convs[i]
82             norm_layer = self.norms[i]
83             dropout_layer = self.dropouts[i]
84             out = conv_layer(x, edge_index)
85             out = norm_layer(out)
86             x = F.relu(out)
87             x = dropout_layer(x)
88             if x.shape == identity.shape:
89                 x = x + identity
90         final_output = self.fc(x)
91         return final_output, final_output
92
93
94 # --- GLOBAL NORMALIZATION STATS LOADING ---
95 global_input_mean_vis_np = np.zeros(VISIBILITY_INPUT_SIZE, dtype=np.float32)
96 global_input_std_vis_np = np.ones(VISIBILITY_INPUT_SIZE, dtype=np.float32)
97 global_input_mean_coeff_np = np.zeros(COEFFICIENT_INPUT_SIZE, dtype=np.float32)
98 global_input_std_coeff_np = np.ones(COEFFICIENT_INPUT_SIZE, dtype=np.float32)
99
100 try:
101     vis_stats_path = NN_PATH / 'processed_visibility_graphs' / 'normalization_stats.npz'
102     if vis_stats_path.exists():
103         vis_stats = np.load(vis_stats_path)
104         global_input_mean_vis_np = vis_stats['global_input_mean']
105         global_input_std_vis_np = vis_stats['global_input_std']
106         print(f"Loaded Visibility GNN normalization stats from {vis_stats_path}")
107     else:
108         print(f"WARNING: Visibility GNN normalization stats not found at {vis_stats_path}. Using default (zeros/ones).")
109
110     coeff_stats_path = NN_PATH / 'processed_coefficient_graphs' / 'normalization_stats.npz'
111     if coeff_stats_path.exists():
112         coeff_stats = np.load(coeff_stats_path)
113         global_input_mean_coeff_np = coeff_stats['global_input_mean']
114         global_input_std_coeff_np = coeff_stats['global_input_std']
115         print(f"Loaded Coefficient GNN normalization stats from {coeff_stats_path}")
116     else:
117         print(f"WARNING: Coefficient GNN normalization stats not found at {coeff_stats_path}. Using default (zeros/ones).")
118
119 except Exception as e:
120     print(f"Error loading normalization stats: {e}. Ensure paths are correct and files exist.")
121
122 global_input_mean_vis_t = torch.tensor(global_input_mean_vis_np, dtype=torch.float32)
123 global_input_std_vis_t = torch.tensor(global_input_std_vis_np, dtype=torch.float32)
124 global_input_mean_coeff_t = torch.tensor(global_input_mean_coeff_np, dtype=torch.float32)
125 global_input_std_coeff_t = torch.tensor(global_input_std_coeff_np, dtype=torch.float32)
126
127
128 # --- MODEL INITIALIZATION AND LOADING ---
129 device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
130 print(f"CUDA available: {torch.cuda.is_available()}")
131 print(f"Using device: {device}")
132

```

```

133 visibility_model = VisibilityGNN(
134     input_size=VISIBILITY_INPUT_SIZE,
135     output_size=VISIBILITY_OUTPUT_SIZE,
136     hidden_channels=VISIBILITY_HIDDEN_CHANNELS,
137     num_layers=VISIBILITY_NUM_LAYERS,
138     heads=VISIBILITY_HEADS
139 ).to(device)
140
141 coefficient_model = CoefficientGNN(
142     input_size=COEFFICIENT_INPUT_SIZE,
143     output_size=COEFFICIENT_OUTPUT_SIZE,
144     hidden_channels=COEFFICIENT_HIDDEN_CHANNELS,
145     num_layers=COEFFICIENT_NUM_LAYERS,
146     heads=COEFFICIENT_HEADS
147 ).to(device)
148
149 try:
150     visibility_model.load_state_dict(torch.load(visNN, map_location=device))
151     visibility_model.eval()
152     print(f"Visibility GNN model '{visNN}' loaded successfully.")
153 except Exception as e:
154     print(f"Error loading Visibility GNN model '{visNN}': {e}.")
155
156 try:
157     coefficient_model.load_state_dict(torch.load(coeffNN, map_location=device))
158     coefficient_model.eval()
159     print(f"Coefficient GNN model '{coeffNN}' loaded successfully.")
160 except Exception as e:
161     print(f"Error loading Coefficient GNN model '{coeffNN}': {e}.")
162
163 global_input_mean_vis_t = global_input_mean_vis_t.to(device)
164 global_input_std_vis_t = global_input_std_vis_t.to(device)
165 global_input_mean_coeff_t = global_input_mean_coeff_t.to(device)
166 global_input_std_coeff_t = global_input_std_coeff_t.to(device)
167
168 def radius_graph_pytorch(pos, radius):
169     """
170     pos: [V,3], r: radius threshold
171     returns edge_index: [2, E]
172     """
173     dists = torch.cdist(pos, pos, p=2)
174     mask = (dists <= radius) & (dists > 0)
175     src, dst = mask.nonzero(as_tuple=True)
176     return torch.stack([src, dst], dim=0)
177
178 app = Flask(__name__)
179
180 @app.route('/process_mesh', methods=['POST'])
181 def process_mesh_data():
182
183     if request.method == 'POST':
184         try:
185             start_time_total = time.perf_counter()
186             start_time_receive = time.perf_counter()
187             binary_data = request.data
188             end_time_receive = time.perf_counter()
189             print(f"Received {len(binary_data)} bytes in {(end_time_receive - start_time_receive)*1000:.2f}
190                   ms.")
191
192             start_time_deserialize = time.perf_counter()
193             num_vertices = int.from_bytes(binary_data[0:4], byteorder='little')
194
195             EXPECTED_FLOATS_PER_VERTEX_INPUT = 3 + 3 + VISIBILITY_INPUT_SIZE
196             raw_float_data = np.frombuffer(binary_data[4:], dtype=np.float32)
197
198             if raw_float_data.shape[0] != num_vertices * EXPECTED_FLOATS_PER_VERTEX_INPUT:
199                 raise ValueError(f"Received data size mismatch. Expected {num_vertices *
200                                     EXPECTED_FLOATS_PER_VERTEX_INPUT} floats, got {raw_float_data.shape[0]}.")
201
202             received_data_reshaped = raw_float_data.reshape(-1, EXPECTED_FLOATS_PER_VERTEX_INPUT)
203
204             pos_np = received_data_reshaped[:, :3]
205             norm_np = received_data_reshaped[:, 3:6]
206             multivectors_np = received_data_reshaped[:, 6:]
207
208             end_time_deserialize = time.perf_counter()
209             print(f"Deserialization & Data Separation in {(end_time_deserialize - start_time_deserialize)
210                   *1000:.2f} ms.")
211
212             start_time_input_norm = time.perf_counter()

```

```

210     input_multivectors_t = torch.tensor(multivectors_np, dtype=torch.float32, device=device)
211     input_multivectors_normalized_vis = (input_multivectors_t - global_input_mean_vis_t) /
        global_input_std_vis_t
212     end_time_input_norm = time.perf_counter()
213     print(f"Input Normalization (Visibility NN) in {(end_time_input_norm - start_time_input_norm)
        *1000:.2f} ms.")
214
215     start_time_graph = time.perf_counter()
216     pos_tensor = torch.tensor(pos_np, dtype=torch.float32, device=device)
217
218     pos_mean = pos_tensor.mean(dim=0)
219     pos_centered = pos_tensor - pos_mean
220     pos_scale = pos_centered.abs().max()
221     if pos_scale > 1e-6:
222         pos_normalized = pos_centered / pos_scale
223     else:
224         pos_normalized = pos_centered
225
226
227     edge_index = torch.empty((2, 0), dtype=torch.long, device=device)
228
229     k_val = 15
230     edge_index = knn_graph(pos_normalized.cpu(), k=k_val, loop=False).to(device)
231
232     input_data_nn1 = Data(x=input_multivectors_normalized_vis, edge_index=edge_index, pos=
        pos_tensor)
233     end_time_graph = time.perf_counter()
234     print(f"Graph Construction in {(end_time_graph - start_time_graph)*1000:.2f} ms.")
235
236     start_time_nn1 = time.perf_counter()
237     with torch.no_grad():
238         visibility_logits = visibility_model(input_data_nn1)
239         visibility_predictions = (torch.sigmoid(visibility_logits) > 0.5).float()
240     end_time_nn1 = time.perf_counter()
241     print(f"NN1 Inference in {(end_time_nn1 - start_time_nn1)*1000:.2f} ms.")
242
243     start_time_prepare_nn2 = time.perf_counter()
244     combined_features_for_nn2 = torch.cat((input_multivectors_t, visibility_predictions), dim=1)
245
246     combined_features_for_nn2_normalized = (combined_features_for_nn2 - global_input_mean_coeff_t)
        / global_input_std_coeff_t
247
248     input_data_nn2 = Data(x=combined_features_for_nn2_normalized,
        edge_index=edge_index, pos=pos_tensor)
249     end_time_prepare_nn2 = time.perf_counter()
250     print(f"Prepare NN2 Input in {(end_time_prepare_nn2 - start_time_prepare_nn2)*1000:.2f} ms.")
251
252     start_time_nn2 = time.perf_counter()
253     with torch.no_grad():
254         coefficient_outputs, _ = coefficient_model(input_data_nn2)
255     end_time_nn2 = time.perf_counter()
256     print(f"NN2 Inference in {(end_time_nn2 - start_time_nn2)*1000:.2f} ms.")
257
258     start_time_ao = time.perf_counter()
259
260
261
262     pred = coefficient_outputs.squeeze(0)
263     pos = pos_tensor
264     norm = torch.tensor(norm_np, device=device)
265     if(num_vertices < 45000):
266         bbox = pos.max(dim=0).values - pos.min(dim=0).values
267         diag = bbox.norm()
268         r = diag * 0.1
269         edge_index = radius_graph_pytorch(pos, r)
270     src, dst = edge_index
271     V = pos.size(0)
272     dirs = pos[dst] - pos[src]
273     dots = (dirs * norm[src]).sum(dim=1)
274     if(num_vertices < 45000):
275         vis = (dots < 0).float()
276     else:
277         vis = (dots < 0.005).float()
278
279
280     counts = torch.zeros(V, device=pos.device).index_add(0, src, vis)
281     deg = torch.zeros(V, device=pos.device).index_add(0, src, torch.ones_like(vis))
282     ao = counts / (deg + 1e-6)
283     ao = ao * 0.8
284     pred_occluded = pred * ao.unsqueeze(1)
285     end_time_ao = time.perf_counter()

```

```

286         print(f"A0 in {(end_time_ao - start_time_ao)*1000:.2f} ms.")
287
288
289         start_time_serialize = time.perf_counter()
290         coefficients_np = pred_occluded.cpu().numpy().astype(np.float32)
291         response_binary_data = coefficients_np.tobytes()
292         end_time_serialize = time.perf_counter()
293         print(f"Serialization & Send Prep in {(end_time_serialize - start_time_serialize)*1000:.2f} ms.
294             ")
295
296         response = make_response(response_binary_data)
297         response.headers.set('Content-Type', 'application/octet-stream')
298
299         end_time_total = time.perf_counter()
300         print(f"Total Request Processing Time: {(end_time_total - start_time_total)*1000:.2f} ms.\n")
301         return response
302
303     except Exception as e:
304         print(f"Server error: {e}")
305         return jsonify({"error": str(e)}), 500
306
307 if __name__ == '__main__':
308     print("Starting Flask server with Waitress...")
309     serve(app, host='0.0.0.0', port=5000)

```

Listing 12: The main code of the Flask server. It processes the requests from Unity and returns the predicted SH coefficients.

B.4 Unity

```

1
2 using System;
3 using System.Collections;
4 using System.Collections.Generic;
5 using System.Globalization;
6 using System.IO;
7 using System.Linq;
8 using System.Text;
9 using R41;
10 using Unity.Barracuda;
11 using Unity.Collections;
12 using Unity.Jobs;
13 using UnityEngine;
14
15 public class NGAShModel : MonoBehaviour
16 {
17     public NNModel modelAsset;
18     private IWorker worker;
19     public ComputeShader shColorShader;
20
21     public GameObject Model;
22     private CGA_Transformations cga;
23
24     // Input data
25     private Vector3[] prevnormals;
26     private List<Vector3> prevLightCoeffs;
27     [HideInInspector]
28     public List<Vector3> lightCoeffs;
29     [HideInInspector]
30     public List<Vector3> shFunctions;
31
32     public float[,] predictedCoeffsReshaped;
33     [HideInInspector]
34
35     float[] prevCoefficients = null;
36     [HideInInspector]
37     public float[] predictedCoeffs = null;
38
39     Vector3 [] meshVertices, meshNormals;
40
41     private ComputeBuffer predictedCoeffsBuffer;
42     private ComputeBuffer colorOutputBuffer;
43     private int kernelSH;
44     private int bufferSize = -1;
45     [HideInInspector]

```

```

46 public bool lightOnlyChanged = false;
47 public Mesh bakedMesh;
48 private float[] inputData;
49 public byte[] serializedBuffer;
50 bool coeffsReceived = false;
51 public bool geometryChanged = false;
52 public bool exportCoeffs = false;
53
54
55
56 private Vector3 lastPosition;
57 private float movementThreshold = 0.2f;
58
59 public void ExportToTxt(string filePath, int numVertices)
60 {
61     if (predictedCoeffs == null || predictedCoeffs.Length != numVertices * 9)
62     {
63         Debug.LogError("Coefficient array length must be numVertices * 9.");
64         return;
65     }
66
67     StringBuilder sb = new StringBuilder();
68
69     for (int v = 0; v < numVertices; v++)
70     {
71         int baseIndex = v * 9;
72         for (int j = 0; j < 9; j++)
73         {
74             sb.Append(predictedCoeffs[baseIndex + j].ToString("G6", CultureInfo.InvariantCulture)); //
              compact float format
75             if (j < 8)
76                 sb.Append(" ");
77         }
78         sb.AppendLine();
79     }
80
81     File.WriteAllText(filePath, sb.ToString());
82     Debug.Log("Exported PRT coefficients to: " + filePath);
83 }
84
85 bool IsMovingFast()
86 {
87     Vector3 currentPosition = Model.transform.position;
88     float movementDelta = (currentPosition - lastPosition).magnitude;
89     lastPosition = currentPosition;
90
91     Quaternion currentRotation = Model.transform.rotation;
92     float rotationDelta = Quaternion.Angle(lastRotation, currentRotation);
93     lastRotation = currentRotation;
94
95     return (movementDelta > movementThreshold) || (rotationDelta > movementThreshold);
96 }
97
98
99 void Start()
100 {
101     kernelSH = shColorShader.FindKernel("CSMain");
102
103     Model runtimeModel = ModelLoader.Load(modelAsset);
104     worker = WorkerFactory.CreateWorker(WorkerFactory.Type.ComputePrecompiled, runtimeModel);
105     cga = GameObject.Find("CGA Setup").GetComponent<CGA_Transformations>();
106
107     lightCoeffs = LoadFromFile("LightData/Solitude Night/LightCoefficients");
108     shFunctions = LoadFromFileSH("LightData/Solitude Night/SHFunctions");
109
110     Mesh bakedMesh = new Mesh();
111
112     if (Model.GetComponent<SkinnedMeshRenderer>()) {
113         Model.GetComponent<SkinnedMeshRenderer>().BakeMesh(bakedMesh);
114
115         meshVertices = bakedMesh.vertices;
116         meshNormals = bakedMesh.normals;
117     }
118     else
119     {
120         Mesh mesh = Model.GetComponent<MeshFilter>().mesh;
121
122         Transform modelTransform = Model.transform;
123
124         // Transform vertices to world space

```

```

125         Vector3[] worldVertices = mesh.vertices
126             .Select(v => modelTransform.TransformPoint(v))
127             .ToArray();
128
129         // Transform normals to world space
130         Vector3[] worldNormals = mesh.normals
131             .Select(n => modelTransform.TransformDirection(n))
132             .ToArray();
133
134
135         meshVertices = worldVertices;
136         meshNormals = worldNormals;
137
138     }
139
140 }
141 public R410 EncodeMatrixAsMultivectorFastReturn(Matrix4x4 M)
142 {
143     Quaternion q = M.rotation;
144     Vector3 t = M.GetColumn(3);
145
146     R410 rotor = R410_Helper.Quaternion_to_rotor2(q);
147     R410 translator = R410_Helper.Generate_translation_rotor(t);
148     R410 motor = translator * rotor;
149
150     R410 result = motor.Normalized();
151
152     rotor.Dispose();
153     translator.Dispose();
154     motor.Dispose();
155
156     return result;
157 }
158
159
160 void Update()
161 {
162     if (exportCoeffs)
163     {
164         string filePath = Path.Combine(Application.dataPath, "coeffs.txt");
165         ExportToTxt(filePath, meshVertices.Length);
166         exportCoeffs = false;
167     }
168     geometryChanged = false;
169     UpdateMesh();
170
171     bool lightChanged = false;
172
173     if (prevnormals == null)
174     {
175         geometryChanged = true;
176     }
177     else if (prevCoefficients == null)
178     {
179         geometryChanged = true;
180     }
181     else
182     {
183         if (!ArraysEqual(prevnormals, meshNormals) || !prevCoefficients.SequenceEqual(predictedCoeffs))
184         {
185             geometryChanged = true;
186         }
187     }
188
189     if (prevLightCoeffs == null || !lightCoeffs.SequenceEqual(prevLightCoeffs))
190     {
191         lightChanged = true;
192     }
193     if (geometryChanged || lightChanged || !coeffsReceived)
194     {
195         if (IsMovingFast())
196         {
197             return;
198         }
199
200         lightOnlyChanged = !geometryChanged && lightChanged;
201         coeffsReceived = ComputeColorsGPU(predictedCoeffs);
202     }
203
204     prevnormals = meshNormals.ToArray();

```

```

205     prevCoefficients = predictedCoeffs;
206     prevLightCoeffs = new List<Vector3>(lightCoeffs);
207 }
208
209 void UpdateMesh()
210 {
211     if (Model.GetComponent<SkinnedMeshRenderer>())
212     {
213         bakedMesh = new Mesh();
214         Model.GetComponent<SkinnedMeshRenderer>().BakeMesh(bakedMesh);
215
216         meshNormals = bakedMesh.normals;
217         meshVertices = bakedMesh.vertices;
218         geometryChanged = true;
219     }
220     else
221     {
222         Mesh mesh = Model.GetComponent<MeshFilter>().sharedMesh;
223         Transform t = Model.transform;
224
225         meshVertices = mesh.vertices.Select(v => t.TransformPoint(v)).ToArray();
226         meshNormals = mesh.normals.Select(n => t.TransformDirection(n)).ToArray();
227     }
228 }
229
230
231 bool ArraysEqual(Vector3[] a, Vector3[] b)
232 {
233     if (a == null || b == null || a.Length != b.Length) return false;
234     for (int i = 0; i < a.Length; i++)
235         if (a[i] != b[i]) return false;
236     return true;
237 }
238
239 public void ResetModel(string lightName)
240 {
241     lightCoeffs = LoadFromFile("LightData/" + lightName + "/LightCoefficients");
242     shFunctions = LoadFromFileSH("LightData/" + lightName + "/SHFunctions");
243 }
244
245 void OnDestroy()
246 {
247     worker.Dispose(); // Clean up
248 }
249
250 IEnumerator PrepareInputJobWrapper(float[] outputArray)
251 {
252     int numVertices = meshVertices.Length;
253     int floatsPerVertex = 3 + 3 + 32; // pos + normal + multivector
254     int totalFloats = numVertices * floatsPerVertex;
255
256     serializedBuffer = new byte[4 + totalFloats * sizeof(float)];
257     using MemoryStream ms = new MemoryStream(serializedBuffer);
258     using BinaryWriter writer = new BinaryWriter(ms);
259
260     writer.Write(numVertices); // First 4 bytes: number of vertices
261
262     var verts = new NativeArray<Vector3>(meshVertices, Allocator.TempJob);
263     var norms = new NativeArray<Vector3>(meshNormals, Allocator.TempJob);
264     var output = new NativeArray<float>(outputArray.Length, Allocator.TempJob);
265     var motors = new NativeArray<Matrix4x4>(numVertices, Allocator.TempJob);
266
267     var motorJob = new PrepareMotorMatrixJob
268     {
269         vertices = verts,
270         normals = norms,
271         motorMatrices = motors
272     };
273
274     JobHandle motorHandle = motorJob.Schedule(numVertices, 64);
275     yield return new WaitUntil(() => motorHandle.IsCompleted);
276     motorHandle.Complete();
277
278     float[] multivectorTemp = new float[32];
279
280     for (int i = 0; i < numVertices; i++)
281     {
282         Vector3 pos = meshVertices[i];
283         Vector3 norm = meshNormals[i];
284

```

```

285         writer.Write(pos.x);
286         writer.Write(pos.y);
287         writer.Write(pos.z);
288         writer.Write(norm.x);
289         writer.Write(norm.y);
290         writer.Write(norm.z);
291
292         Matrix4x4 motorMatrix = motors[i];
293         Quaternion q = motorMatrix.rotation;
294         Vector3 t = motorMatrix.GetColumn(3);
295
296         using R410 rotor = R410_Helper.Quaternion_to_rotor2(q);
297         using R410 translator = R410_Helper.Generate_translation_rotor(t);
298         using R410 motor = translator * rotor;
299         using R410 normalized = motor.Normalized();
300
301         int baseOffset = i;
302         for (int j = 0; j < 32; j++)
303         {
304             float val = normalized[j];
305             output[baseOffset + j * numVertices] = val;
306             writer.Write(val);
307         }
308     }
309
310     output.CopyTo(outputArray);
311
312     verts.Dispose();
313     norms.Dispose();
314     output.Dispose();
315     motors.Dispose();
316     while (!Model.GetComponent<FlaskServerComm>())
317         yield return null;
318     Model.GetComponent<FlaskServerComm>().GetCoeffs(this);
319 }
320
321
322 bool ComputeColorsGPU(float[] predictedCoeffs)
323 {
324     int numVertices = meshVertices.Length;
325     if (inputData == null || inputData.Length != numVertices * 32)
326         inputData = new float[numVertices * 32];
327
328     StartCoroutine(PrepareInputJobWrapper(inputData));
329     float[] flatLC = FlattenVector3List(lightCoeffs);
330     float[,] reshapedLC = ReshapeTo2D(flatLC, 9, 3);
331     float[,] lightCoeffsMatrix = reshapedLC;
332
333     if (bufferSize != predictedCoeffs.Length)
334     {
335         predictedCoeffsBuffer?.Dispose();
336         colorOutputBuffer?.Dispose();
337
338         predictedCoeffsBuffer = new ComputeBuffer(predictedCoeffs.Length, sizeof(float));
339         colorOutputBuffer = new ComputeBuffer(numVertices, sizeof(float) * 3);
340         bufferSize = predictedCoeffs.Length;
341     }
342
343     shColorShader.SetBuffer(kernelSH, "predictedCoeffs", predictedCoeffsBuffer);
344     shColorShader.SetBuffer(kernelSH, "outColors", colorOutputBuffer);
345     shColorShader.SetInt("numVertices", numVertices);
346
347     predictedCoeffsBuffer.SetData(predictedCoeffs);
348
349     Vector4[] flatLight = new Vector4[9];
350     for (int i = 0; i < 9; i++)
351     {
352         flatLight[i] = new Vector4(
353             lightCoeffsMatrix[i, 0],
354             lightCoeffsMatrix[i, 1],
355             lightCoeffsMatrix[i, 2],
356             0
357         );
358     }
359     shColorShader.SetVectorArray("lightCoeffs", flatLight);
360
361     int groups = Mathf.CeilToInt(numVertices / 64f);
362     shColorShader.Dispatch(kernelSH, groups, 1, 1);
363
364     if (predictedCoeffs == null || predictedCoeffs.Length != meshVertices.Length * 9)

```

```

365     {
366         return false;
367     }
368
369     Vector3[] rawColors = new Vector3[numVertices];
370     colorOutputBuffer.GetData(rawColors);
371
372     Color[] finalColors = new Color[numVertices];
373     for (int i = 0; i < numVertices; i++)
374     {
375         finalColors[i] = new Color(rawColors[i].x, rawColors[i].y, rawColors[i].z, 1);
376     }
377
378     if (Model.GetComponent<SkinnedMeshRenderer>())
379     {
380         Model.GetComponent<SkinnedMeshRenderer>().sharedMesh.colors = finalColors;
381     }
382     else
383     {
384         Model.GetComponent<MeshFilter>().mesh.colors = finalColors;
385     }
386
387     return true;
388 }
389
390
391 List<Vector3> LoadFromFile(string path)
392 {
393     List<Vector3> data = new List<Vector3>();
394     try
395     {
396         TextAsset textAsset = Resources.Load<TextAsset>(path);
397
398         if (textAsset == null)
399         {
400             UnityEngine.Debug.LogError($"Error: File '{path}' not found in Resources.");
401         }
402
403         string[] lines = textAsset.text.Split('\n');
404         foreach (string line in lines)
405         {
406             string[] tokens = line.Split(' ');
407             if (tokens.Length >= 3)
408             {
409                 float x = float.Parse(tokens[0], CultureInfo.InvariantCulture);
410                 float y = float.Parse(tokens[1], CultureInfo.InvariantCulture);
411                 float z = float.Parse(tokens[2], CultureInfo.InvariantCulture);
412
413                 data.Add(new Vector3(x, y, z));
414             }
415         }
416     }
417     catch (System.Exception e)
418     {
419         UnityEngine.Debug.LogError($"Error loading vertices: {e.Message}");
420     }
421
422     return data;
423 }
424
425 List<Vector3> LoadFromFileSH(string path)
426 {
427     List<Vector3> data = new List<Vector3>();
428     try
429     {
430         // Load the text file from Resources folder
431         TextAsset textAsset = Resources.Load<TextAsset>(path);
432
433         if (textAsset == null)
434         {
435             UnityEngine.Debug.LogError($"Error: File '{path}' not found in Resources.");
436         }
437
438         string[] lines = textAsset.text.Split('\n');
439         foreach (string line in lines)
440         {
441             string[] tokens = line.Split(new[] { ' ', '\t' }, System.StringSplitOptions.
442                                     RemoveEmptyEntries);
443             int numTokens = tokens.Length;

```

```

444         // Parse all groups of x, y, z values in the line
445         for (int i = 0; i < numTokens; i += 3)
446         {
447             if (i + 2 < numTokens) // Ensure there are 3 components
448             {
449                 float x = float.Parse(tokens[i], CultureInfo.InvariantCulture);
450                 float y = float.Parse(tokens[i + 1], CultureInfo.InvariantCulture);
451                 float z = float.Parse(tokens[i + 2], CultureInfo.InvariantCulture);
452
453                 data.Add(new Vector3(x, y, z));
454             }
455         }
456     }
457 }
458 catch (System.Exception e)
459 {
460     UnityEngine.Debug.LogError($"Error loading data from {path}: {e.Message}");
461 }
462
463 return data;
464 }
465
466
467 float[] FlattenVector3List(List<Vector3> vectors)
468 {
469     float[] flattened = new float[vectors.Count * 3];
470     for (int i = 0; i < vectors.Count; i++)
471     {
472         flattened[i * 3 + 0] = vectors[i].x;
473         flattened[i * 3 + 1] = vectors[i].y;
474         flattened[i * 3 + 2] = vectors[i].z;
475     }
476     return flattened;
477 }
478
479 // Function to reshape 1D array to 2D array
480 public float[,] ReshapeTo2D(float[] flatArray, int numRows, int numCols)
481 {
482     float[,] reshapedArray = new float[numRows, numCols];
483     for (int i = 0; i < numRows; i++)
484     {
485         for (int j = 0; j < numCols; j++)
486         {
487             reshapedArray[i, j] = flatArray[i * numCols + j];
488         }
489     }
490     return reshapedArray;
491 }
492
493 public void UpdateLight(string lightName)
494 {
495     lightCoeffs = LoadFromFile("LightData/" + lightName + "/LightCoefficients");
496     shFunctions = LoadFromFileSH("LightData/" + lightName + "/SHFunctions");
497 }
498 }

```

Listing 13: The code of the NGAShModel component, which is the backbone of the GNN-based NeuralGASh in Unity.

C Appendix

C.1 Introduction to Conformal Geometric Algebra (CGA)

Conformal Geometric Algebra (CGA) extends traditional Euclidean geometry by enabling both flat and round geometric primitives to be expressed as multivectors. Unlike classical vector spaces, where only lines and planes are directly represented, CGA provides a unified framework in which points, spheres, circles, lines, and planes all coexist within the same algebraic structure. Moreover, rigid-body transformations such as translations and rotations, as well as uniform scalings (dilations), can be expressed as multivectors and applied through the so-called *sandwich operation*. This dual capability—representing both geometry and transformations algebraically—makes CGA especially attractive for computer graphics and physics.

For a comprehensive introduction to CGA and its applications in computer graphics and engineering, readers are referred to standard texts such as Hildenbrand et al. foundational works on geometric algebra computing [31, 32]. Recent surveys and advances illustrating the growing role of geometric algebra in applied mathematics, physics, and graphics can be found in Hitzer et al. [35, 36]. For more applied and technically oriented treatments of CGA within computer graphics and related computational frameworks, the reader is referred to [41, 54, 42], which illustrate practical implementations and optimization strategies for geometric algebra in rendering, simulation, and visualization contexts.

C.2 Vector Objects in CGA

To construct the 3D conformal model, the Euclidean basis vectors $\{e_1, e_2, e_3\}$ of $\mathbb{R}^3$ are extended with two additional basis elements, e_+ and e_- , of positive and negative signature, respectively, such that $e_+^2 = 1$ and $e_-^2 = -1$. The resulting space is the 5D vector space $\mathbb{R}^{4,1}$, whose Clifford algebra is denoted $\mathcal{G}(4, 1)$. It is convenient to introduce a *null basis*:

$$e_0 = \frac{1}{2}(e_- - e_+), \quad e_\infty = e_- + e_+,$$

for which $e_0^2 = e_\infty^2 = 0$. These null vectors provide the foundation for encoding Euclidean points and higher-order primitives in conformal space.

A general vector $Y \in \mathbb{R}^{4,1}$ can be expressed as

$$Y = y_1 e_1 + y_2 e_2 + y_3 e_3 + y_\infty e_\infty + y_0 e_0, \quad y_i \in \mathbb{R}.$$

Due to projective equivalence ($Y \sim \lambda Y$ for any scalar λ), the representation is

unique up to scaling. Within this framework:

- A point $x = (x_1, x_2, x_3) = x_1e_1 + x_2e_2 + x_3e_3$ of $\mathbb{R}^3$ is *up-projected* into the conformal vector

$$X = x + \frac{1}{2}x^2e_\infty + e_0 = x_1e_1 + x_2e_2 + x_3e_3 + \frac{1}{2}(x_1^2 + x_2^2 + x_3^2)e_\infty + e_0.$$

- A sphere centered at x with radius r is represented by

$$S = X - \frac{1}{2}r^2e_\infty.$$

- A plane with normal $\mathbf{n}$ and distance d from the origin is encoded as

$$\Pi = \mathbf{n} + de_\infty.$$

C.3 Products in $\mathbb{R}_{4,1}$

In CGA, three products are fundamental: the *inner product*, the *outer product*, and the *geometric product*. These are first defined on the basis vectors $\{e_1, e_2, e_3, e_+, e_-, e_0, e_\infty\}$ and then extended by linearity to arbitrary multivectors. Below we summarize their key properties.

Inner product. The inner product (denoted by $\cdot$) encodes metric information. For the basis elements it holds:

- $e_i \cdot e_j = \delta_{ij}, \quad i, j \in \{1, 2, 3, +\},$
- $e_- \cdot e_- = -1,$
- $e_- \cdot e_j = 0, \quad j \in \{1, 2, 3, +\},$
- $e_0 \cdot e_0 = e_\infty \cdot e_\infty = 0,$
- $e_0 \cdot e_\infty = -1,$
- $e_i \cdot e_j = 0, \quad i \in \{1, 2, 3, +\}, j \in \{0, \infty\}.$

Outer product. The outer product (denoted $\wedge$) generalizes the cross product to higher dimensions. It is associative and can be applied to any number of basis vectors:

$$e_i \wedge e_j, \quad e_i \wedge e_j \wedge e_k, \quad \dots$$

The outer product of k basis vectors is called a *k-blade*, and sums of blades of varying grades form general *multivectors*. Geometrically, the outer product is crucial because it encodes intersections:

- a circle is the intersection ($\wedge$) of two spheres,
- a line is the intersection of two planes,
- three intersecting spheres yield a point pair.

Geometric product. The most important operator in CGA is the geometric product, which combines the inner and outer products:

$$e_i e_j = e_i \cdot e_j + e_i \wedge e_j.$$

It is associative and distributive, and it underpins the entire algebra. For instance, if $i \neq j$ and $\{i, j\} \neq \{0, \infty\}$, then

$$e_i e_j = e_i \wedge e_j.$$

The geometric product generalizes the familiar complex and quaternion multiplications, and it enables the compact representation of rotations, translations, and other transformations as multivectors.

C.4 Rotations, Translations, and Dilations

Beyond representing points, planes, circles, spheres, lines, and point pairs as multivectors, CGA can also express *transformations* such as rotations, translations, and dilations in a unified form. Each of these operations can be applied directly to objects via the *sandwich operator*, i.e.,

$$O' = M O M^{-1},$$

where M is the transformation multivector.

Rotation. A rotation in CGA is encoded as the multivector

$$R = \exp\left(-b \frac{\phi}{2}\right) = \exp\left(-I_3 u \frac{\phi}{2}\right) = \cos\left(\frac{\phi}{2}\right) - u I_3 \sin\left(\frac{\phi}{2}\right),$$

where:

- ϕ is the rotation angle,
- b is the normalized rotation plane,
- u is the normalized rotation axis,
- $I_3 = e_1 e_2 e_3$ is the unit pseudoscalar of Euclidean 3-space.

Its inverse is given by

$$R^{-1} = \exp\left(b\frac{\phi}{2}\right) = \cos\left(\frac{\phi}{2}\right) + uI_3 \sin\left(\frac{\phi}{2}\right).$$

Translation. A translation by a Euclidean vector $t = t_1e_1 + t_2e_2 + t_3e_3$ is represented as

$$T = \exp\left(-\frac{1}{2}te_\infty\right) = 1 - \frac{1}{2}te_\infty,$$

with inverse

$$T^{-1} = \exp\left(\frac{1}{2}te_\infty\right) = 1 + \frac{1}{2}te_\infty.$$

Dilation. A dilation with scale factor $d > 0$, relative to the origin, is expressed as

$$D = 1 + \frac{1-d}{1+d}e_\infty \wedge e_0,$$

whose inverse is

$$D^{-1} = \frac{(1+d)^2}{4d} + \frac{d^2-1}{4d}e_\infty \wedge e_0.$$

Remark: for $d = 0$, this operator reduces to $D = 1 + e_\infty \wedge e_0 = 1 + e_+e_-$, which is not invertible in $\mathbb{R}_{4,1}$ since $(e_+e_-)^2 = 1$.

Composition of transformations. Multiple transformations can be composed by multiplying their respective multivectors:

$$M = M_n M_{n-1} \cdots M_1,$$

and applying the sandwich operator:

$$O' = M O M^{-1}.$$

This formulation allows unified handling of rigid motions and scalings within the same algebraic framework.

C.5 Remarks

Through this algebraic structure, CGA provides a unified representation for geometric primitives, transformations, and their combinations. Its expressiveness makes it a powerful tool in computer graphics, robotics, and physics, where both the description of geometry and the manipulation of objects are essential.